Master of Science in Software Engineering
June 2025

All Work and No Flow: Experiences on Automating Product Owners' Workflow using Generative AI

Gauri Vinsha Singh

Faculty of Computing, Blekinge Institute of Technology, 371 79 Karlskrona, Sweden

This thesis is submitted to the Faculty of Computing at Blekinge Institute of Technology in partial fulfillment of the requirements for the degree of Master of Science in Software Engineering. The thesis is equivalent to 20 weeks of full-time studies.

The authors declare that they are the sole authors of this thesis and that they have not used any sources other than those listed in the bibliography and identified as references. They further declare that they have not submitted this thesis at any other institution to obtain a degree.

Contact Information:
Author(s):
Gauri Vinsha Singh
E-mail: gasi22@student.bth.se

University advisor:
Dr. Mikael Svahnberg
Department of Software Engineering

Faculty of Computing
Blekinge Institute of Technology
SE–371 79 Karlskrona, Sweden

Internet : www.bth.se
Phone : +46 455 38 50 00
Fax : +46 455 38 50 57

Abstract

Background. Generative Artificial Intelligence (GenAI) is rapidly transforming businesses by automating their operations. The potential of GenAI for code generation tasks has been widely explored, yet its application in non-coding activities within software development remains relatively unexplored. Product owner tasks are not necessarily code-related because they manage both the business and development sides. Automating these repetitive or time-consuming activities presents an opportunity to streamline and improve the workflow.
Objectives. The goal of this study is to explore the perceived potential of GenAI to automate and improve non-coding tasks in software development. The non-coding tasks refer to the activities performed by product owners in their workflow from receiving the requirements for a product to delivering them. The study aims to identify non-coding tasks that are suitable candidates for automation based on existing evidence and explore the perceived potential of GenAI implementation along with associated limitations and challenges.
Methods. A technical action research is conducted at Ericsson for this study. Semi-structured interviews and observations are conducted to identify time-consuming tasks in the process, expectations of product owners from automation of workflow, and perceptions about GenAI use. A scoping literature is carried out to identify suitable candidates for automation that are then automated in a prototype developed to streamline the workflow of product owners. The prototype is evaluated by product owners and exit-interviews and surveys are conducted to collect product owner feedback on the prototype and automated support for tasks.
Results. The findings identify time-consuming tasks conducted by product owners are documentation, artefact management, communication, compiling information, and requirement analysis. The literature suggests that GenAI can be effectively used to automate articulation tasks such as documentation and requirement analysis. Product owners have a positive preference for using the prototype for a streamlined workflow and GenAI automations within it as support for workflow tasks.
Conclusions. The study establishes the importance of streamlined, integrated workflows with as few tool chain distractions as possible. The prototype's development and evaluation demonstrate that streamlined processes save time and reduce context switching, while GenAI-based automations improve these streamlined processes.

Keywords: Generative Artificial Intelligence, Non-Coding Tasks, Process Automation, Product Owner

Acknowledgments

They say it takes a village to raise a child; turns out it also takes one to complete a thesis.

First and foremost, I want to extend my deepest gratitude to my university supervisor, **Dr. Mikael Svahnberg**, who has been much more than just an advisor. Thank you for guiding me not only through the thesis but also offering wisdom for life that often came with perspective shifts. You taught me how to think better, work smarter, and take myself a little less seriously in the process.

A special thank you to **Prof. Dr. Daniel Mendez Fernandez** and **Dr. Julian Frattini** for helping me shape what was once an ad post on LinkedIn into a clear research plan. Your thoughtful feedback, sharp questions, and generous time helped lay the groundwork for everything that followed.

A huge thank you to the team at Ericsson, my supervisors **Ruwan Lakmal Silva** and **David Mollersten**, and manager **Peter Raneskold** for consistently supporting me in this journey. Your trust and encouragement made me want to do better, be better, and believe that "just one more feature" makes all the difference.

To my teammate at Ericsson, **Angelo Prado Junior**, you are a true MVP. Thank you for enduring my roller coaster of emotions, late-night rants, and deadline-induced coding binges. Your moral support and friendship are the reason this thesis did not end up written entirely in caps lock.

I am also immensely thankful to all the **participants** at Ericsson who lent their time, opinions, and honesty. Your contributions are the heart of this research. Thank you for sharing your insights and perspectives with honesty and openness. I am genuinely grateful for your involvement in filling this thesis with meaningful stories and data.

Lastly, to my **friends and family**, thank you for your infinite patience and the noble act of not asking, "Are you done yet?" every time we spoke. I am grateful for your support during this time and for not kicking me out of the group chats anytime I sent a hundred texts, stressing over nothing and everything at once.

Contents

Abstract **i**

Acknowledgments **iii**

1 Introduction **1**
1.1 Ethical, societal and sustainability aspects 2
1.1.1 Ethical Considerations 2
1.1.2 Societal Considerations 3
1.1.3 Sustainability Considerations 3
1.2 Aim and objectives 4
1.3 Scope 4
1.4 Outline 4

2 Background and Related Work **7**
2.1 Background 7
2.1.1 Non-Coding Software Development Tasks 7
2.1.2 Machine and Deep Learning 8
2.1.3 GenAI and LLMs 8
2.1.4 Prompt Engineering 9
2.2 Related Works 9
2.2.1 Agile Software Development and Scrum 10
2.2.2 Role and Responsibilities of Product Owners 11
2.2.3 Productivity and Work Satisfaction 12
2.2.4 GenAI in Software Development 12
2.2.5 Automation of Software Development Tasks 14
2.3 Study Motivation and Research Gap 15

3 Method **17**
3.1 Study Design 18
3.1.1 Participants and Timeline 19
3.1.2 Alternate Study Designs 20
3.1.3 Ethical Considerations 20
3.2 Initial Interviews and Observation 21
3.2.1 Data Collection 21
3.2.2 Data Analysis 22
3.2.3 Data Validation 23
3.3 Scoping Review 23
3.3.1 Review Protocol 23

3.3.2 Search Strategy . . . 25
3.3.3 Eligibility Criteria . . . 27
3.3.4 Paper Screening . . . 28
3.4 Implementation . . . 28
3.4.1 Script-Based Automations . . . 28
3.4.2 GenAI-Based Automations . . . 29
3.5 Exit Interviews and Survey . . . 32
3.5.1 Data Collection . . . 32
3.5.2 Data Analysis . . . 33
3.5.3 Data Validation . . . 33

4 Results and Analysis **35**
4.1 Participant Demographic . . . 35
4.2 Initial Interviews and Observations . . . 35
4.2.1 Thematic Analysis . . . 36
4.2.2 Findings from Initial Interviews . . . 38
4.2.3 Findings from Observations . . . 47
4.3 Scoping Review . . . 48
4.3.1 Findings from Scoping Review . . . 49
4.4 Exit-Interviews and Surveys . . . 51
4.4.1 Findings from Survey . . . 51
4.4.2 Thematic Analysis . . . 52
4.4.3 Findings from Exit-Interviews . . . 54
4.5 Answering Research Questions . . . 57

5 Discussion **61**
5.1 The Evolving Role of Product Owner . . . 61
5.2 Rethinking Processes, Not Just Tasks . . . 62
5.3 The Productivity Paradox: When More Tools Mean Less Flow . . . 63
5.4 Automation Dreams and Reality Checks . . . 64
5.5 Automation Meets Articulation . . . 65
5.6 If Everything's AI, What's Left for Us? A Philosophical Pause . . . 66
5.7 Threats to Validity . . . 67
5.7.1 Construct Validity . . . 68
5.7.2 Internal Validity . . . 68
5.7.3 External Validity . . . 69
5.7.4 Reliability . . . 69

6 Conclusions and Future Work **71**
6.1 Conclusions . . . 71
6.2 Future Work . . . 72

References **73**

A Coding Scheme for Initial Interviews **85**

B Results from Scoping Review **89**

C Coding Scheme for Exit Interviews **95**

List of Figures

3.1 Study Design - Technical Action Research 19
3.2 Example Automation Flow for Generation of Main Report Text Suggestions from Pre-Study Report or a Requirement 32

4.1 Thematic map showing themes and sub-themes identified from initial interviews 38
4.2 Process Workflow of Product Owners 39
4.3 PRISMA Diagram for Scoping Literature Review 48
4.4 Thematic map showing themes and sub-themes identified from initial interviews 53

List of Tables

3.1 Problem Formulation 17
3.2 Participants involved in different phases of the study and the associated timeline. *P1, P2 ... P5*: are product owners at Ericsson. *S1, S2*: supervisors at Ericsson. *R1, R2*: reviewers at university. *T1, T2*: thesis students who conducted the study. Phases: *(D)* Diagnosing, *(AP)* Action Planning, *(AT)* Action Taking, *(EV)* Evaluation, *(SL)* Specifying Learning 20
3.3 Observation data recorded for 15 minutes during a full day observation with a product owner. The task IDs are denoted as follows: 2 refers to replying to messages on Teams chat, 5 refers to taking down notes, 7 refers to finding and reading information, and 8 refers to attending meeting with the design team. 22
3.4 Protocol for Scoping Review 25
3.5 Concept Map for Brainstorming Keyterms for Search String 26
3.6 Search Strings Used for Scoping Review 27
3.7 Prompt for Text Generation for Functional Requirements for Report . 30
3.8 Prompt for Text Explanation of Terminology in Reports 31
3.9 Prompt for Language Review of Reports 31

4.1 Participant Demographic for the Study 35
4.2 Time Duration for Initial Interviews and Observations 36
4.3 Themes, Description, and Codes Identified in Thematic Analysis . . . 37
4.4 Results for non-coding tasks automated using GenAI models in literature 51
4.5 Time Duration for Exit-Interviews and Survey 51
4.6 Mean and p-values for prototype and automation set from t-test . . . 51
4.7 Data Analysis of Survey 52
4.8 Themes, Description, and Codes Identified in Thematic Analysis . . . 53

A.1 Coding Scheme, Description, and Examples Identified in Thematic Analysis of Initial Interviews 88

B.1 Results from scoping review for automation and implementation of non-coding tasks using GenAI 94

C.1 Coding Scheme, Description, and Examples Identified in Thematic Analysis of Exit Interviews 96

List of Acronyms

AI Artificial Intelligence

ANN Artificial Neural Network

API Application Programming Interface

CoT Chain of Thought

DL Deep Learning

DoD Definition of Done

GDPR General Data Protection Regulation

GenAI Generative Artificial Intelligence

GPT Generative Pre-Trained Transformer

GPU Graphics Processing Unit

LLM Large Language Models

ML Machine Learning

RAG Retrieval Augmented Generation

SUS System Usability Scale

UML Unified Modelling Language

XML Extensible Markup Language

Chapter 1

Introduction

Generative Artificial Intelligence (GenAI) creates new content such as text and images by using a machine learning (ML) model to learn patterns and relationships in a dataset of human- or machine-generated content and then using the learned patterns to generate new content [55]. GenAI presents new opportunities for organisations to rethink their business processes and capabilities, one such application being task automation [11]. GenAI tools utilise Large Language Models (LLMs) and machine learning techniques to automate repetitive or complex tasks [41].

In software development, the new research on GenAI has focused on team productivity, centred on using generative AI-based tools to improve work efficiency, lower task errors, and speed up software development [25]. Research shows that companies often struggle to properly apply generative AI, with most programs failing to gain traction or yield financial gains [19]. However, software professionals generally have a positive perception of generative AI tools, particularly valuing them for their ability to streamline workflows and content generation [24]. Some other applications of GenAI for developers include receiving instant code feedback from AI coding assistants and improved testing and documentation using automated content generation [10].

For example, a previous study finds that development time is significantly reduced for three projects under consideration using GenAI for code generation [85]. The code quality is higher since the bug frequency is reduced by close to 40%, and the time spent on code reviews is reduced by almost 30% using GenAI. A specific research focus of GenAI applications is agile software development, where software products are developed in iterations with a feedback loop fostering team collaboration and knowledge sharing [7]. Other studies indicate that GenAI may enhance developers' productivity in agile software development by automating repetitive tasks so that they can focus on innovation [89].

An important part of driving the software development process in agile environments are product owners responsible for analysing customer requirements and guiding development teams [54]. The most common responsibilities of product owners include breaking down high-level requirements into technical requirements for implementation, business meetings, project tracking, requirement prioritisation and sharing technical knowledge [59]. Moreover, their tasks, such as understanding business requirements, organising meetings, and tracking progress are often time-consuming or repetitive [87].

Product owners spend significant time on non-coding tasks such as meetings with stakeholders or development teams [90]. They can spend around 65% of their time in

meetings and scrum events interacting with up to 15 different types of roles [90]. The role of the product owner is complex and challenging since they need to manage the needs of the stakeholders, the technical constraints of the product, and the business goals of the organisation [87], [9], [48]. Moreover, the existing research has shown that product owners are important for the success of agile software projects [59], and their tasks can be time-consuming [90], [59]. For example, product owners can spend up to 20% of their time on maintenance and management of the product backlog [87].

While research exists about GenAI for code automation and its benefits (see e.g. [85], [45], [72]), the benefits and challenges of automating non-coding tasks are relatively unexplored. A major challenge discussed in the studies is the reliability of the AI-generated responses with practitioners claiming they have to verify the generated responses and spend extra time on refining the response [24], [51]. Therefore, the research gap identified is to study the potential of GenAI tools to automate product owners' non-coding software development tasks and their associated limitations and challenges. This is an important research topic because software processes rely heavily on several non-coding development tasks such as requirement analysis, backlog prioritisation, and project management for their success [39].

Furthermore, the current notion among researchers is that additional research is needed to explore the capabilities of GenAI further for non-coding tasks in software development processes. This is due to two main reasons, either non-coding tasks have been studied in the backdrop of coding tasks and thus, the results are not emphasised [25], [89] or the context and scope of the study are limited and more studies are needed to generalise the results [24], [78], [51]. This makes this research topic an important one to explore to corroborate previous research's findings and contribute new findings in the understanding of the use of GenAI for automation of non-coding software development tasks.

1.1 Ethical, societal and sustainability aspects

This study is part of an industrial thesis with Ericsson and involves interviews, observations, and surveys with product owners at the company. The implementation for this study involves use of GenAI for automation of non-coding tasks. The ethical, societal, and sustainability aspects are considered in this regard and are discussed here.

1.1.1 Ethical Considerations

The study complies with GDPR rules [99] during the proposed research's data collection and analysis stages. The study also complies with the company's Business Code of Ethics and Non-Disclosure Agreement since participants for this study are company employees. Participation in the study was voluntary and the participants could withdraw consent at anytime without giving any reasons.

There is no personal data collected that can be used to identify a person. Personal information such as the names of the participants are collected at any stage because it does not serve any purpose for the research. The participants, therefore, stay

anonymous during the thesis to maintain confidentiality and all results are presented as an aggregate to ensure that a specific person can not be identified.

The data collected during the study is stored locally on the work computer issued by the company, cloud storage services are avoided. Data access is only available to students, supervisors, and examiners at the university. The data is processed according to GDPR rules and deleted after the completion of the thesis. No personal data and sensitive company data is used as input for AI tools. GenAI models are used solely for automating tasks using scripts. Only company-controlled AI models are utilised for implementation during the thesis.

1.1.2 Societal Considerations

Introducing AI in the workplace can cause unnecessary stress and pressure as employees might feel judgement from AI [21]. For instance, employees may overthink if they are being efficient or not or why is an AI determining their performance [21]. There might also be a lack of trust or scepticism about integrating AI automated tasks in the workflow [2]. The study is done transparently and documented properly to clarify any discrepancies and address user concerns as the automated tasks get rolled out to product owners.

The study introduces an AI-supported automation which may not work as expected. The subjects might end up having to spend more time to fix the mistakes The same has been a concern in the previous research as well [24], [89]. The implementation is tested before product owners interact with it to ensure a basic level of reliability. In addition, the exit interviews also include questions related to challenges faced by the product owner while using the automated support for tasks and any limitations they perceive for the automated task.

While GenAI has the potential to augment content generation, there is a social aspect of people being flooded by generated text such that they do not read anything anymore. The sheer volume of content can be overwhelming making it difficult to identify and engage with high-quality, relevant information [73]. In this study, the suggestions made by GenAI automations are not added to reports directly so product owners have the opportunity to review the generated content and consciously add it to their documents. In addition, all suggestions for different sections of reports are generated separately instead of a large AI-generated response covering the entire report at once.

1.1.3 Sustainability Considerations

From a sustainability perspective, the non-coding tasks selected for automation should be feasible. This is why the tasks suitable for automation are identified based on existing evidence in research. The aim is to have automations that utilise only necessary computing power and resources, and are optimised so they do not take an exceptionally long time to execute [1].

1.2 Aim and objectives

The goal of this study is to explore the perceived potential of GenAI to automate and improve non-coding tasks in software development. The non-coding tasks refer to the activities performed by product owners in their workflow from receiving the requirements for a product to delivering them. This may include tasks like requirement analysis, meetings with stakeholders, project tracking, backlog management, etc. The study aims to identify non-coding tasks that are suitable candidates for automation based on existing evidence and explore the perceived potential of GenAI implementation along with associated limitations and challenges. The objectives of the proposed study are as follows:

1. Identify non-coding tasks performed by product owners in their roles and daily workflow.

2. Identify non-coding tasks that may be suitable for automation based on existing evidence.

3. Automate a selection of important and time-consuming non-coding tasks using GenAI.

4. Explore the perceived potential and limitations of GenAI in automating these tasks.

1.3 Scope

The scope of the study is to explore the potential and limitations of GenAI for automating non-coding software development tasks of product owners. The study is conducted at Ericsson, a large telecommunications company, at one of the product development units. The goal of this study is to improve the workflow followed by system managers, who perform a subset of traditional product owner tasks, at Ericsson with a focus on non-coding tasks. System managers are referred to as product owners for rest of this thesis. They are responsible for analysing requirements, documenting the requirements, guiding development teams through implementation, and tracking project status. The aim is to automate parts of the workflow using GenAI such that the automated support for non-coding tasks reduces time spent on time-consuming, repetitive, and monotonous tasks in order to streamline the workflow and improve its efficiency to alleviate pain points of product owners.

1.4 Outline

The thesis report is organised to provide an overview of the research problem identified, aim and objectives, and scope of the study in Chapter-1. The background and related works in Chapter 2 present a critical analysis of existing literature related to the research problem and identify gaps in the existing literature to motivate the need for this study. The research questions and research method employed to

answer them are detailed in Chapter-3 along with data collection, analysis, and validation measures. The results and analysis for each research question are presented in Chapter-4. The discussion in Chapter-5 interprets the results and relates them to the research questions. The conclusion chapter summarises the main findings and recommendations for future research in Chapter-6.

Chapter 2

Background and Related Work

2.1 Background

The Background section presents an overview of the foundational concepts that underpin this study. The section begins with an overview of non-coding activities in the software development lifecycle, with a focus on tasks performed by product owners. The section then delves into the principles of machine learning and its evolution into deep learning, followed by the emergence of generative AI with a particular focus on large language models. The section also introduces prompt engineering, a growing discipline that explores how to effectively interact with LLMs to achieve desired outputs.

2.1.1 Non-Coding Software Development Tasks

Software projects involve many critical activities beyond writing code and the product owner role is highly involved in these non-coding tasks [88]. A product owner's primary non-coding responsibility is requirement analysis, where they engage with stakeholders to elicit and refine requirements [46]. They analyse the impact of requirements and negotiate trade-offs to balance conflicting requirements. They also do a variety of administrative tasks to support the software development lifecycle. A primary task performed by product owners is writing and maintaining detailed and up-to-date documentation for requirements which may involve epics, user stories, impact analysis, and decision logs [46]. Product owners also frequently plan and facilitate important meetings, such as project progress meetings, and product demos.

They lead discussions about product direction, and stakeholder feedback [88]. This includes creating agendas, gathering input, and ensuring that participants have access to necessary information prior to meetings. Product owners are often in charge of configuring and managing tools like Jira for generating and managing epics, stories, and tasks, and keeping sprint boards up to current [23]. Product owners serve as cross-functional coordinators, collaborating with external teams and roles. These tasks are also administrative in nature and are critical to mapping requirements with organisational constraints [39]. In some organisations, especially those operating under scaled agile frameworks, they may also contribute to resource planning and budget forecasting [14]. This can include estimating the cost of the requirement and planning man-hours to align expenditures with available budgets. While not always within the traditional product owner role, these tasks are commonly observed in

scaled or hybrid agile contexts [14].

2.1.2 Machine and Deep Learning

For computers and machines to act intelligently, they need to learn [47]. Machine learning is a subset of AI that allows computers to learn automatically without any human intervention or assistance [81]. It is a set of algorithms and techniques that allow machines to learn from data using two main types of techniques: supervised and unsupervised machine learning [47]. In supervised learning, the machines learn from labelled data, i.e., the outcome or target variable is already known [81]. In unsupervised learning, there is no labelled data [81]. The machine is responsible for analysing the data and finding patterns which it then classifies input data based on these learned patterns [81].

Deep learning is a subset of machine learning inspired by the human brain and utilises artificial neural networks (ANN) [47]. ANN mimic the neural structure of the brain using learning, memory, and generalisation. Neural networks can therefore capture highly complex relationships in data and are used as building block of deep learning [32]. An ANN comprises multiple layers such as input output layer and several hidden layers between the input and output layers [8]. Deep learning differs from machine learning because machine learning generally uses structured data [47]. On the other hand, deep learning utilises unstructured data without pre-processing [47]. DL models often require large datasets for training in comparison to ML models and consequently a longer training time [47]. The outputs from ML models are usually numeric whereas outputs from DL models can be of different formats [47].

2.1.3 GenAI and LLMs

GenAI refers to a set of algorithms that can be used to generate new content such as text, images, videos, audio, code, and simulations from input data commonly known as prompts [20]. The building blocks of GenAI models are artificial neural networks. The GenAI models can be built on different types or combination of ANNs, such as large language models and transformers [52]. A large language model allows to predict what words are likely to come next in a text [47] whereas a transformer model is a type of artificial neural network that is designed to process sequential data, like sentences in natural language, and can look at all parts of the input simultaneously [92]

A Large Language Model (LLM) is a deep neural network usually trained on extensive textual data, including books, code, articles, and websites, to identify patterns and relationships in specific languages [47]. The trained models seek to construct an internal representation of the training data by analysing language structure, rules, and patterns [6]. LLMs employ massive datasets to learn billions of parameters during training, requiring large computing resources for both the training and operating stages [92]. In contrast to conventional LLMs, modern language models (OpenAI tools like GPT-4.1) can analyse large amounts of data at once using transformer neural network technology, which leads to them having the ability to perform tasks like generating code snippets and creative texts [92].

2.1.4 Prompt Engineering

Prompt engineering refers to a set of techniques to improve the instructions a user provides to GenAI models to get desired outputs [96]. Designing appropriate prompts is important to get more relevant and accurate results, therefore, it is suggested to avoid generic and broad queries and instead narrow down the scope of the prompts to specific instructions [96]. Several prompting techniques exist that can be used depending on different context such as zero-shot and few-shot prompting for new tasks without extensive training, chain of thought prompting for reasoning and logic, and retrieval augmented generation (RAG) to give the model access to data it has not been trained on [82].

The one-shot prompting technique removes the need for extensive data training of the model by sending it a task description in the prompt but without labeled data for training on specific input-output mappings [96]. The model then leverages its pre-existing knowledge to generate predictions based on the given prompt for the new task. Unlike zero-shot prompting, where no examples are supplied, few-shot prompting provides models with a few input-output examples to induce an understanding of a given task [96]. However, few-shot prompting requires more tokens which can be a hindrance for large-text requests.

Chain-of-Thought (CoT) prompting is a technique where the prompt additionally includes a rationale for each example, encouraging the model to verbalise the intermediate reasoning steps for solving the task [95]. But today this is more or less superseded or inbuilt in the models themselves by the new class of reasoning models. RAG analyses user input, formulates a targeted query, and searches a pre-built knowledge base for relevant resources [96]. Retrieved snippets are added to the original prompt, adding background context to make it better. The LLM is empowered by this augmented prompt to produce creative, factually correct answers.

2.2 Related Works

The related work section presents work on agile software development and widespread adoption of the Scrum methodology in industry, and the importance of the product owner role within it. Although Ericsson is not fully-agile or employs Scrum, the work is included to establish where product owner role originates from and the motivation for looking into the product owner role as context for this study. This is followed by related work on the role and responsibilities of product owners as described in literature and how it is applied in industry. The section also highlights how the role is adopted in industries to validate that the role and responsibilities of product owners can be split among several roles in practice as project manager, system manager, etc. as was the case in this study. System managers at Ericsson only perform a subset of the tasks discussed in this section.

The chapter continues with related work on productivity and work satisfaction to emphasise the relationship between them and how adoption of new technologies such as GenAI can be used to improve workplaces. The section relates the concept of productivity and work satisfaction from a more generic context of software industries to the specific context of agile teams that is more directly related to the scope of

this study. This then leads to introducing related work on use of GenAI in software development, covering the perceptions of software practitioners on use of GenAI in software development tasks and the challenges associated with its adoption. This section is included to add awareness on the perceived potential and limitations of use of GenAI in software development.

The section also presents literature on how various tasks, both coding and non-coding related, can be automated using GenAI and the potential they have for improving software processes. The section discusses automation of tasks in a software development lifecycle to show how GenAI models can be utilised in different use cases and relate to processes instead of standalone tasks. More specific work on non-coding tasks of product owners are discussed as part of RQ2 results, and therefore not included here to avoid repetition.

2.2.1 Agile Software Development and Scrum

Agile software development emphasises flexibility, leanness, teamwork, communication, incremental development, quality, and adaptability. Agile software development methods typically involve collaboration, reviews, small teams, short release deadlines, time-boxing, and continuous testing [7]. Scrum is one such methodology that has emerged as the most popular agile methodology in recent times, as more businesses continue to implement it [79]. Scrum teams are small, cross-functional, self-organised, and empowered, with a focus on communication and collaboration and a team typically includes a product owner, Scrum master, and developers [83].

Raunak et al. present survey data from 99 software engineering managers and developers about their preferred methods, tools, and processes as well as their perceptions of the aspects that contribute to a project's success or failure, with a focus on the patterns of agile practice adoption [75]. While agile adoption is on the rise, traditional methods remain prevalent in some areas. Scrum is the most widely used agile technique, although firms use it differently to meet their unique needs. The report shows that incomplete or unclear requirements continue to be a big issue in the software business. Moreover, the lack of accurate time and expense estimation is prevalent and could explain the high occurrence of software failures and delays.

Sekitoleko et al. investigate the scalability of agile approaches. The authors mention five issues, including planning, task prioritisation, knowledge exchange, code quality, and integration [84]. The authors suggest that the issues interact with one another, perhaps creating a vicious circle where struggling with one raises the likelihood of problems with others. The study recommends improving planning and information sharing (e.g., scrum-of-scrums) to enhance team communication and prevent vicious cycles. The report recommends better planning and knowledge sharing, including Scrum-of-Scrums, to increase team communication and prevent vicious loops.

Kadenic et al. carry out a large-scale cross-sectional survey with 182 Scrum team members to investigate characteristics and practices that make a team successful at Scrum [46]. Their findings suggest that the Scrum framework's success relies on all three roles: product owner, Scrum master, and developers. Prioritisation by product owners is crucial for effective backlog management and accountability, and a product owner's inability to prioritise responsibilities hinders teamwork. Moreover, effective

teamwork relies on developers' capacity to modify daily plans towards the sprint goal. Conversely, a lack of active involvement in planning procedures can inhibit this ability.

2.2.2 Role and Responsibilities of Product Owners

Sverrisdottir et al. present findings from interview with five product owners who have had the role for at least a year within five different organisations [87]. The interviews are intended to determine how product owners' roles and responsibilities are defined in the management of Scrum projects. The findings reflect that the product owner role is to combine the understanding of the business and technical aspects of a product. They are the owners of the product and are responsible for defining and prioritising requirements to be implemented in cooperation with the stakeholders involved in the project. The vital characteristics identified for a product owner to have were communication and planning to create a common understanding between different stakeholders.

In another study by Matturro et al., an exploratory study is carried out to study how the product owner role is performed in industry in comparison to theory [62]. Six product owners from four participating companies are interviewed about their background, responsibilities, and characteristics of a product owner. The results show that product owners participate in sprint planning, sprint review, and backlog refinement meetings always or almost always. The characteristics desired in product owners are communication skills, teamwork, and customer orientation.

Bass et al. employ a mixed-method approach to identify product owner activities comprising two case studies across large and small medium enterprises [9]. Their findings suggest that the product owner is responsible for requirement analysis, backlog prioritisation, communicating between stakeholders and developers, and, as a mentor, providing domain knowledge and solving conflicts. The product owner is also responsible for clarifying details of product backlog items and the management and approval of release plans.

Magnussen et al. interview 18 professionals to identify common tasks and responsibilities associated with product owners [59]. Their findings also identify product owners responsible for communicating with stakeholders and the development team, prioritising backlogs, and sharing their technical knowledge. In addition, the tasks commonly performed and faced challenges by product owners are creating specifications, project tracking, business meetings, and backlog management.

Unger-Windeler et al. conduct a multi-mapping study and a case study to identify the role of product owners [90]. The study reveals that product owners' major role is as a communicator and they can spend up to 65% of their time in meetings. Rather than simply providing the team with the requirements for the product under development, product owners require this time to synchronise and align their work, streamline the agile process of large-scale Scrum, discuss team-related topics, and resolve upcoming issues.

Kadenic et al. conduct a systematic literature review in combination with a focus group to highlight that the role of product owner is tailored to the specific organisational context at the team level and involves managing the organisational environment and culture [46]. Organisational factors determine the job of a product

owner, but human traits have a significant impact on performance. For example, communication skills play a crucial role in establishing and managing connections within an organisation.

2.2.3 Productivity and Work Satisfaction

A conscious effort is required to offer an attractive workplace when competing for talent [67]. The outcome of the product is directly related to the ability to balance the needs and nature of the workdays for different team members [88]. Meyer et al. outline a framework for software developers that describes what makes a good workday, containing three main factors: value creation, efficient use of time, and perception [65]. Value creation refers to whether or not developers feel they are creating something, efficient use of time refers to meeting expectations and the ability to stay focused on work, and perception is about how developers perceive their own productivity for the day.

The opinion of whether a workday was good or terrible is heavily influenced by the expectations for the day. Even if a day is mostly spent in meetings, it can still be deemed productive. However, if one expects a day of focused work and the day is packed with meetings, it is regarded as a terrible workday. Interruptions from coworkers have a detrimental impact on developers' ability to focus and work efficiently. However, helping a coworker is viewed positively and is satisfying.

Fatema and Sakib conduct a study to identify and analyse how individual behaviour and productivity are affected by teamwork within an agile team [27]. The four most perceived factors influencing the productivity of agile team members are team effectiveness, team management, motivation, and customer satisfaction, while lack of team management support is considered to be the most mentioned reason for failed agile projects. Some additional factors positively affecting productivity are skillfulness, motivation, and project management efficiency whereas factors affecting productivity negatively are exhaustion, external dependency, culture, and resource constraints.

Melo et al. conduct a multiple-case study in three large Brazilian companies that have been using agile methods for over two years to provide a better understanding of factors and mediators that affect agile team productivity [64]. They analyse the major factors influencing agile teamwork productivity using the team's perception as one potential dimension and find that team management is the most influential factor on agile team productivity. Their findings suggest that companies may need to review their organisational structures and determine the fit between their structure and agile teams. Company structure decisions, therefore, directly influence the coordination process selection among teams.

2.2.4 GenAI in Software Development

Ebert and Louridas present a discussion about GenAI applications from their case study in the software industry [25]. Their primary focus is on how GenAI can streamline software development processes by automating tasks such as testing, debugging, and deployment. However, they also present a shorter discussion on how GenAI can

be utilised in non-coding development tasks such as idea generation, summarising documentation, reviews, and meeting minutes, similar to the product owner tasks.

Ulfsnes et al. conduct an empirical study interviewing 13 data scientists, managers, developers, designers, and front-end developers to investigate the usage of GenAI [89]. They find the common usage of GenAI in software development is asking for help when stuck with coding tasks or non-coding tasks, such as overcoming writer's block. The developers mainly use GenAI for code-related activities such as refactoring, testing, and explaining the code. The interview participants claim that using GenAI for repetitive tasks frees up their time for creative endeavours. However, they agree that AI-generated content rarely constitutes a finished product and usually needs additional refinement to be useful in a practical setting.

Coutinho et al. conduct a pilot case study involving software practitioners from different roles to understand the effect of integration of GenAI tools in their daily workflows [24]. They find a positive perception of these tools among participants since they improve their productivity but also have certain limitations. For example, the findings show that participants report difficulties in ensuring the reliability and functionality of the generated responses, especially when they use multiple questions simultaneously. One of the participants also notes security as a challenge because of their inability to use the tools with sensitive project data. Since the results found are preliminary, the authors argue there is a need for more research to understand the efficacy and impact of GenAI tools on the productivity of practitioners in the software industry.

Cabrero-Daniel et al. conduct four experiments to assess how different GenAI tools perform software management tasks as compared to humans and how they can be used to enhance the tasks, and their impact on human decisions and agency [21]. The findings show that GenAI can perform story point estimation better than humans, while it needs human oversight for other tasks, such as defining the Definition of Done (DoD) for tasks, analysing if requirements have been met and creating checklists from task titles, especially because it does not take the complete context into account. In addition, the authors highlight that engineers may feel discomfort or unnecessary stress when being judged by an AI as its integration might be accompanied by expectations of increased efficiency.

Kim evaluates ChatGPT and Bard's software development capabilities, one of the focuses being requirement analysis of an online tour reservation system [51]. The findings imply that both tools can identify ambiguities and clarify them in the system description, but both had limitations in fully integrating the provided clarifications. Another finding of the study is that there are overlaps in the identification of functional requirements from the description, and some additional and more general non-functional requirements are added by the tools that are not part of the system description. In addition, both tools generate use case specifications adhering to commonly used forms, but one of the tools always misses some details that the other one has. The findings imply that GenAI tools may not always provide the correct or complete solution to tasks.

2.2.5 Automation of Software Development Tasks

Sami et al. explore the use of LLMs to automate requirement elicitation by developing a multi-agent system using GPT models 3.5 and 4o to generate and prioritise user stories from an initial project description [80]. The results from the experiment on a real-world project show that GPT-3.5 achieved a higher semantic similarity score than GPT-4o for user story generation, but they both have consistent performance in prioritisation of requirements and correctly identified core features of the project. Although more research is needed to improve the prompts, the results show that this approach can improve productivity and accuracy in requirements engineering tasks. The authors also propose that human experts, like product owners and quality assurance specialists, should be included in the interaction loop with autonomous agents for improved performance.

Jahan et al. present an automated approach for generating behavioural models, specifically sequence diagrams, from natural language requirements expressed as user stories [44]. The user stories are preprocessed to extract actors, action and benefit and refined further. The information collected from user stories is used to generate a text script compatible with PlantUML. The generated text file is then fed to PlantUML API to create the sequence diagram as an image. This automation is compared to using ChatGPT for the same task using zero-shot prompting. The results find that the automated approach effectively produces relevant, simplified diagrams for straightforward user stories. However, ChatGPT outperforms the automated support implemented by the authors, but the LLM tends to create complex diagrams that are sometimes beyond the simplicity of the original user stories.

Sherje studies the potential of AI-powered code generation techniques to improve software development efficiency and assessed its benefits, limitations and challenges [85]. The study finds that development time is significantly reduced for three projects under consideration using GenAI for code generation. In addition, the code quality is higher since the bug frequency is reduced by close to 40%. The study also finds that time spent on code reviews is reduced by almost 30% using GenAI. The paper explores the advantages of AI-powered code production, such as faster development cycles, less human error, and easier quick prototyping and experimentation. The paper acknowledges use of AI-powered code generation is not without obstacles and restrictions, such as concerns around data quality, model interpretability, domain-specific expertise, and ethical considerations that should be addressed to ensure the appropriate and effective use of AI in software development.

Naimi et al. propose an automated approach for generating test cases from UML diagrams [69]. The proposed approach aims to streamline the process of test case generation, which starts with giving an XML file as input, which has structured use cases that are retrieved and sent as parameters in an API request to GenAI model along with a prompt. The generated response is then saved as an output file. The authors argue that the approach can improve test coverage when compared to manual test case generation, but it can pose challenges with the cost associated with APIs and tokens. The approach can also be further extended to other UML diagrams, such as class and sequence diagrams.

2.3 Study Motivation and Research Gap

It can be established from related works in agile software development that Scrum is being adopted as the most popular agile methodology at companies in recent times. The literature shows that Scrum roles are adapted as per companies' and projects' needs instead of Scrum guidelines, nevertheless, the role of product owner remains important for the success of Scrum projects. Further literature presented details the roles and responsibilities of product owners as practised in the industry. The existing work shows the complexity of the role and the time-consuming tasks performed by product owners in their role. This serves as motivation for looking into the software development tasks from the perspective of product owners.

Work productivity is related to work satisfaction, meaning it needs to be an important consideration for software companies [31]. More satisfied workers may be more productive and vice-versa. Introducing new technologies to team members' environment, such as GenAI may have a positive effect on their productivity. At the same time, teamwork and workplace culture are critical to job satisfaction. The long-term job satisfaction and overall productivity may be impacted by a greater reliance on tools like GenAI, which may increase individual productivity while unintentionally decreasing inter-team collaboration. This adds to the motivation of why GenAI is selected as the focus of this study.

The existing literature on the use and perception of GenAI in software development processes shows that there is mainly a positive perception of such tools being integrated but several ethical concerns remain. The studies establish the potential of GenAI adoption in the software industry while highlighting the concerns that should be addressed to avoid any hindrances to GenAI's adoption. This serves as motivation for looking into the potential and limitations of GenAI for software development tasks to add to the body of knowledge of the field.

The related works on the automation of software development tasks are also presented, showcasing different applications of GenAI models for both coding and non-coding tasks that happen during a software development lifecycle. The existing work show the capabilities of different GenAI models to automate different kinds of tasks, yet some implementation limitations remain. Moreover, the tasks are automated using GenAI in isolation, and the literature about the potential of automation of workflows and processes using GenAI remains relatively unexplored.

The research gap identified is to explore the potential and limitations of GenAI for the automation of non-coding software development tasks within a software process from the perspective of product owners. The research topic is of importance because non-coding development tasks have been explored very little as candidates for automation using GenAI to improve the software development process. Moreover, a lot of the existing research has been experimental to automate standalone tasks or compare GenAI models for a task using hypothetical data in existing databases or data created for the study. This opens another opportunity to do a study in an industrial context with real first-hand data about software processes and feedback.

Chapter 3

Method

The research design employed for the study is technical action research (TAR), and a mixed-methods approach is used with a combination of interviews, observations, a survey, and a literature review as part of the action research. TAR is the use of an experimental artefact to help a client and learn about its effects in practice [98]. TAR is considered a suitable choice for this study since an artefact, i.e., automated support for non-coding tasks, is being evaluated to help an industrial partner, Ericsson, with identified problems in the workflow and learning about the effect of automated support on the workflow in practice.

The goal of this study is described following the design problem structure proposed by Wieringa [98]:

Improve the workflow followed by product owners at Ericsson organisation in the context of non-coding tasks conducted during a study, **By** automating the non-coding tasks using GenAI within the workflow, **Such that** the automated support for tasks leads to less time consumed on time-consuming, repetitive, and boring tasks, **In order to** streamline the workflow and improve its efficiency to alleviate product owner pain points.

Table 3.1: Problem Formulation

The following research questions are derived from the above problem formulation in Table 3.1.

RQ1.1: What are the most time-consuming non-coding tasks that product owners conduct?
Motivation: This research question is formulated to identify the workflow tasks of the product owners and the pain points within it. The emphasis is on non-coding tasks, which are time-consuming since automations are expected to make the workflow efficient and thereby save time for product owners.

RQ1.2: What are the perceptions and expectations of product owners about automation using GenAI?
Motivation: This research question is formulated to complement the findings of RQ1.1 and take into account the expectations of the product owners for the prioritisation of tasks to automate. The research question also aims to gain insights

into product owners' perceptions of using GenAI in their workflow, including their current use of GenAI and the quality criteria they expect from it.

RQ2: What non-coding tasks are suitable for automation based on existing evidence in the research using GenAI?
Motivation: The research question is formulated to identify suitable candidates for automation from the non-coding tasks identified in previous RQs based on existing evidence in the research. The purpose is to add to the body of knowledge what models are useful and optimal for a certain type of non-coding task, and how the existing GenAI models can be used to implement the automations.

RQ3: What are the perceived potential and limitations of product owners when using automated support for non-coding tasks?
Motivation: This research question is formulated to evaluate the experience of product owners with the streamlined workflow and the automations within it. The research question aims to discover both the potential and limitations of the automated support for non-coding tasks of product owners with a focus on GenAI implementations.

3.1 Study Design

A TAR study involves five iterative phases: diagnosing, action planning, action taking, evaluation, and specifying learning. The way each phase contributes to answering the proposed research questions and the methods used for each research question are described below, starting with a description of the context in which TAR is applied. The research context is characterised by the non-coding tasks conducted by product owners in their workflow from receiving the requirements to delivering them at Ericsson. The product owners are responsible for analysing incoming requirements, proposing solutions, and overseeing the implementation of the solution. The study design is visualised in Figure 3.1 below.

Diagnosing: During this phase, the focus was on identifying the non-coding tasks and pain points associated with them within a product owner's workflow. Semi-structured interviews and observations were conducted with product owners to answer RQ1.1. The interviews were also used to answer RQ1.2 to gain insights into expectations of product owners from automations and their perceptions of using GenAI in their workflow.

Action Planning: During this phase, the focus was on discussing and choosing solutions to improve the problems faced by the product owners. Apart from discussions with supervisors, a literature review was conducted to answer RQ2, specifically a scoping review. The scoping review was conducted to identify suitable candidates for automation using GenAI, coupled with discussions with company supervisors for prioritising non-coding tasks to automate.

Action Taking: During this phase, a prototype was developed to streamline the workflow of product owners. The non-coding tasks identified during the diagnosing phase and suitable candidates identified for automation within them during the action planning phase were automated within the prototype using GenAI. The prototype was developed to be used and evaluated by product owners in the next phase.

Evaluation: During this phase, the prototype was used by the product owners to solve a meaningful task. Their experiences with the automated support for tasks were explored using a survey and another semi-structured interview to gain insights into the potential and limitations of the prototype and the automations within it.

Specifying Learning: The lessons learned from the first cycle of action research and their implications are documented for the next iterations. At the time of this study, only the first cycle of iterations has been done.

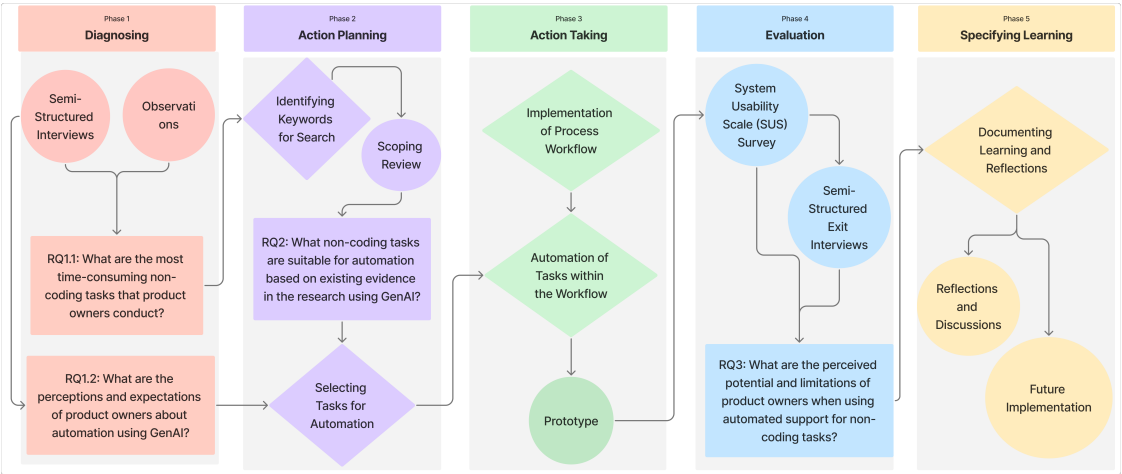

Figure 3.1: Study Design - Technical Action Research

3.1.1 Participants and Timeline

A total of eleven people participated in this TAR study which took place between January 2025 and May 2025: five product owners, two thesis students, two reviewers, and two company supervisors. The bachelor's thesis student was involved during the diagnosing and action taking phases while the master's thesis student took part in all phases for this thesis study and the research activities associated with it. The company supervisors and reviewers were involved during the action planning phase while product owners were involved during the diagnosing and evaluation phases. A summary of the involvement of different types of participants in different phases and timeline can be found below in Table 3.2.

Participant	Role in Thesis Study	D	AP	AT	EV	SL
P1	Product Owner	✓			✓	
P2	Product Owner	✓			✓	
P3	Product Owner	✓			✓	
P4	Product Owner	✓			✓	
P5	Product Owner	✓			✓	
S1	Company Supervisor		✓			
S2	Company Supervisor		✓			
R1	University Supervisor		✓			
R2	External Reviewer		✓			
T1	Bachelor's Thesis Student	✓		✓		
T2	Master's Thesis Student	✓	✓	✓	✓	✓
		Jan-Feb	Feb-Mar	Mar	Apr	Apr-May

Table 3.2: Participants involved in different phases of the study and the associated timeline. *P1, P2 ... P5*: are product owners at Ericsson. *S1, S2*: supervisors at Ericsson. *R1, R2*: reviewers at university. *T1, T2*: thesis students who conducted the study. Phases: *(D)* Diagnosing, *(AP)* Action Planning, *(AT)* Action Taking, *(EV)* Evaluation, *(SL)* Specifying Learning

3.1.2 Alternate Study Designs

An alternate study design that can be employed is a case study, which is usually done to observe and understand a phenomenon or artefact in a real-world context [97]. However, since the study involved an intervention to improve the process, a TAR study is considered more suitable. TAR also allows for building and deploying artefacts, the tool prototype in this case, within the research context, in contrast to case studies.

Another alternate is design science methodology that relies on the development of innovative solutions to solve a real-world problem and evaluate its effectiveness to solve the problem [97]. This alternative is not considered a suitable choice because a novel tool or technology is not being developed in the scope of this study. The aim is to have a tool to streamline the workflow and automate tasks within it based on existing evidence.

Technology transfer research is another alternative study design which focuses on moving technology from research labs into real-world practice [97]. This alternative is not considered suitable for this study because the study is being done at the partner company Ericsson, with involvement of stakeholders, making technical action research a more suitable choice.

3.1.3 Ethical Considerations

The interview and survey data collected during the study is stored and processed per GDPR rules [99]. The participants are anonymous during the thesis project, and the results are presented as an aggregate in the final report so they can not be linked to a specific participant. The participation in the study was voluntary and the participants were allowed to withdraw consent at any time. The collected data is

securely stored for the duration of the thesis project with access to authorised people only, including the thesis student, supervisors, and reviewers.

The participants were contacted with help from company supervisors to participate in the study. An information letter detailing the study purpose, research tasks, and timeline were sent out to the potential participants. A consent form was also sent out to the participants detailing the ethical guidelines considered for the study and were signed by interested participants. A separate consent form was created for observations required in diagnosing stage since not all product owners who participated in the study, participated in observations.

The AI tools used in this thesis are Whisper[1] for transcription of interview recordings for RQ1.1, RQ1.2, and RQ3 after approval from company supervisors. The tool was set up locally on a work computer in a virtual environment, and the model was only used for transcription, not for training. The second AI tool used in the study is Rayyan[2] which is an AI-powered tool for conducting systematic literature reviews. The tool was used for managing research papers and paper screening during the scoping review for RQ2. The paper screening was done manually and verified manually by the thesis supervisor and the external reviewer. There is no sensitive or company data involved during this phase, and thus, the tool was approved for use by company supervisors too.

3.2 Initial Interviews and Observation

To answer RQ1.1 and RQ1.2 during diagnosing phase, 1-hour joint interviews were conducted with the five product owners along with bachelor's thesis student as a second interviewer. The interviews were followed by a full-day observation with a product owner and partial observations with another product owner who was also part of the joint interviews.

3.2.1 Data Collection

To answer RQ1.1 and RQ1.2, semi-structured interviews are considered a suitable choice because they provide an opportunity to ask follow-up questions and clarification during the interviews [33], allowing direct discussions for multiple tasks instead of focusing on one type of task. The interviews were held in-person or online on Microsoft Teams depending on participant availability. The interview sessions were recorded for transcription later.

After the interviews were complete, observations were conducted with two product owners. A full-day observation was conducted with one of the product owners for 8 hours, followed by a partial observation with another product owner for 4 hours spread over two days. The data documented during the observational study was done using the protocol described by Robson [76]. The tasks observed were given a unique ID, and the duration of doing the task was noted down every 15 seconds per minute. Any observations apart from the time taken for a task were also documented as short notes. The data in Table 3.3 below is taken from the observation data from

[1]https://github.com/openai/whisper
[2]https://www.rayyan.ai/

the full day observation. The table summarises the tasks observed for 15 minutes. The observed task were noted down every 15 seconds per minute for a total of 15 minutes in this example.

Time	0s	15s	30s	45s
1	2	2	8	8
2	8	8	8	8
3	8	8	8	8
4	8	8	8	8
5	8, 5	8, 5	8, 5	8, 5
6	8	8	8	8
7	8	8	8, 5	8, 5
8	8, 5	8, 5	8, 5	8, 5
9	8, 5	8, 5	8, 5	8, 5
10	8, 7	8, 7	8, 7	8, 7
11	8, 7	8, 7	8, 7	8, 7
12	8, 7	8	8	8, 7
13	8, 7	8, 7	8, 7	8, 7
14	8, 7	8, 7	8	8
15	8	8	8	8

Table 3.3: Observation data recorded for 15 minutes during a full day observation with a product owner. The task IDs are denoted as follows: 2 refers to replying to messages on Teams chat, 5 refers to taking down notes, 7 refers to finding and reading information, and 8 refers to attending meeting with the design team.

An alternate data collection method for RQ1.1 and RQ1.2 that can be used is surveys, because a limitation of observations is the longer time needed to collect data. Another limitation is finding participants willing to cooperate and allow observation of their workflow, which may also have inherent confidentiality issues. While data from a larger population can be collected through surveys, surveys may suffer from low participation rates if the surveys are too long or have unclear questions [50]. In such a case, open-ended questions would be a preferable choice because the purpose is to identify non-coding tasks, however, such a survey could be lengthy and time-consuming. Another limitation of the survey for RQ1 is that follow-up questions and clarifications can not be asked, which is important to understand the context in which tasks are performed and get useful answers from the participants.

3.2.2 Data Analysis

A thematic analysis of interview transcripts for RQ1.1 and RQ1.2 is conducted to identify time-consuming non-coding tasks that product owners conduct. Relevant text was highlighted and encoded using suitable labels. Themes were then identified in the labelled data. The analysed data was reviewed iteratively to split, merge or remove labels and themes that were no longer relevant. The thematic analysis is complemented with findings from observation data for RQ1.1 and RQ1.2.

3.2.3 Data Validation

To validate the findings for RQ1.1 and RQ1.2, the thematic analysis was reviewed by the thesis supervisor and company supervisors, to analyse parts of the transcripts independently or at least verify the coding scheme to strengthen its reliability. In addition, the findings were further validated using the non-coding tasks described in the company documentation about workflow processes for software development teams. Lastly, the findings were compared to the product owner's non-coding tasks identified in related works to further validate them.

3.3 Scoping Review

During the action planning phase, a scoping review was conducted to answer RQ2. Scoping reviews are systematic-like, take 1-4 weeks to complete and can be done by 1-2 reviewers [3], making them a suitable choice for this research question in the given time frame. One of the purposes of choosing a scoping review is to identify types of available evidence for a concept in a given field [68], in this case, automation of non-coding software development tasks using GenAI. While commonly used in the medical field for research [74], scoping reviews have recently been used in the software engineering field, especially for topics with limited existing research [30], [55].

An alternate research method that can be used for RQ2 is a literature review, but it may lack the rigorousness to make the findings more reliable, and the results can not be reproduced if more studies are needed for generalising the results [4]. Systematic literature reviews are another alternative, but they are too rigorous and require a more specific research question [68]. It was difficult to fit it into the current time plan with other research questions and, therefore, was not selected.

3.3.1 Review Protocol

A protocol was created to drive the scoping review detailing the steps that were followed during the study and can be found in Table 3.4 below. The first step was identifying a team to carry out the scoping review. The master's thesis study carried out the scoping review, while the university supervisor and an external reviewer from the university acted as the reviewers for it. A team of three members was considered suitable so that any disagreements could be resolved with the third team member. The review question for which the scoping review was conducted is the same as RQ2 formulated for the thesis study.

The search was conducted in four electronic databases: SCOPUS (Elsevier), ACM Digital Library, Springer Nature Link, and BTH Summon. The considered sources also covered papers from IEEE and ScienceDirect, which is why they were not included separately. Google Scholar was also not considered because the search is not repeatable if the same key terms are used to replicate the study. Keyword search strategy was used to find papers in the information sources, and the same search string could be used to find papers on all four databases. Systematic reviews of the literature were considered during the search, and the articles mentioned in the reviews were included for review. However, caution was taken to treat these studies separately.

Zotero was used for reference management, and the references were exported to Rayyan for paper screening and management. An eligibility criterion was created to identify relevant studies to be included in the review. Title and abstract screening of collected papers was conducted, followed by full-text screening. Relevant data was extracted based on the data extraction fields detailed in the protocol below and analysed using deductive thematic analysis.

Protocol for Scoping Review		
Step-1	Identifying a Team	**Primary Investigator:** Master's Thesis Student **Reviewers:** University Supervisor External Reviewer from University
Step-2	Identifying Research Question	**RQ2:** What are suitable candidates for automation based on existing evidence in the research for automating these non-coding tasks using GenAI?
Step-3	Identifying Relevant Studies	**Information Sources:** SCOPUS (Elsevier), ACM Digital Library, Springer Nature Link, BTH Summon **Search Strategy:** Keyword search and papers in systematic literature reviews. **Eligibility Criteria:** Inclusion Criteria: • Studies published from 2017 to the time of the study, i.e. February 2025. • Papers in the English language. • Published research papers, empirical study, review article, technology review, editorial opinion, discussion papers, grey literature, and student thesis. • Papers discussing the automation of non-coding tasks conducted by product owners identified during RQ1 data analysis. Exclusion Criteria: • Pre-prints of published work. • Student assignment, commercial websites, and blogs. • Papers without full text access. • Papers not relevant to the domain of software development. • Studies about the automation of coding tasks using GenAI.

Step-4	Selecting Studies to Be Included in the Review	**Data Management:** • Zotero for reference management. • Rayyan for paper screening and selection. • Separate sheets in an Excel workbook for documenting each step of the process. **Selection Process:** Abstract and title screening followed by a review by reviewers, any disagreements are resolved through discussions with the third team member. **Review Strategy:** Scoping review steps reviewed by both reviewers.
Step-5	Charting the Data	**Data Extraction:** • Summary of what the article is about. • Non-coding task automated. • GenAI models used. • Automation details. • Implementation details. • Level of evidence.
Step-6	Collating, Summarising and Reporting the Results	**Data Synthesis:** Deductive thematic analysis of extracted data, results presented as a narrative text and summarised in tables.

Table 3.4: Protocol for Scoping Review

3.3.2 Search Strategy

A concept map was created to brainstorm search terms and alternate words and phrases for them. An initial search was made using a search string developed from concept map terms before the initial interviews and observations were conducted. The concept map at this stage was generated using terms from the study scope as defined in Section 1 and brainstorming from there. However, this initial search string was not sufficient for a complete search with the generic "non-coding tasks" moniker.

The concept map was then extended with the non-coding tasks identified in initial interviews and observations. Alternate keywords and phrases were identified for each task. The key terms were combined using OR and AND connectors. A new, more specific search string was created using non-coding tasks identified from interview data and used for the search. The complete concept map is available in Table 3.5.

Topic:	GenAI Automation for Non-Coding Software Development Tasks of Product Owners
Brainstorming:	GenAI, automation, non-coding tasks, software development, product owners, LLM, ChatGPT
Ideas	**Alternate Keywords or Phrases**
GenAI	Generative Artificial Intelligence, generative Artificial Intelligence, genAI, Generative AI
Automation	Automated, Automating
Software Development	Software Process, Agile Development, Scrum, Software Engineering, Product Management
Product Owners	System Managers, Product Managers
LLM	Large Language Models, llm, LLM-Aided, LLM-Based
ChatGPT	Chatgpt, chatgpt, chat gpt, chat GPT, Chat GPT
Non-Coding Tasks	**Alternate Keywords or Phrases**
Requirements	requirement analysis, requirements engineering, RE, requirement breakdown, use case, user story, epic
Tools	Jira, Microsoft Outlook, MS Outlook, MS Suite, Microsoft Suite, Microsoft Tools, MS Tools
Meetings	meeting minutes, meeting notes, meeting invitation, meeting invite, meeting link, meeting scheduling, meeting scheduler
Reports	text generation, phrasing, report generator, background analysis, explaining text, compiling information
Source Code	code description, code explanation, understanding code, explaining code, analysing code, code analysis

Table 3.5: Concept Map for Brainstorming Keyterms for Search String

The initial and refined search strings used are available in Table 3.6 below. The initial search string was used as is for a search in all four databases. However, the refined search string was used in three of the databases, except BTH Summon. The search string was split into smaller search strings to decrease the response time for the request. The search string was split to keep the common terms in for each search and change the non-coding tasks for the search, an example of which is shown in Table 3.6.

Initial Generic Search String
("GenAI" OR "Generative Artificial Intelligence" OR "Generative AI" OR "generative Artificial Intelligence" OR "genAI") AND ("automation" OR "automated" OR "automating") AND ("software development" OR "software process" OR "agile" OR "scrum") AND ("product owners" OR "system manager" OR "product manager")
Refined Specific Search String

("GenAI" OR "Generative Artificial Intelligence" OR "Generative AI" OR “LLM-Aided” OR “LLM-Based” OR “ChatGPT”) AND (“automation" OR "automated" OR "automating") AND ("software development" OR "software process" OR "agile" OR "scrum") AND (“requirement analysis” OR “requirements engineering” OR “RE” OR “requirement breakdown” OR “use case” OR “user story” OR “epic” OR “meeting” OR “meeting minutes” OR “meeting notes” OR “meeting invitation” OR “meeting invite” OR “meeting link” OR “meeting scheduling” OR “meeting scheduler” OR “Jira” OR “Microsoft Outlook” OR “MS Outlook” OR “MS Suite” OR “Microsoft Suite” OR “Microsoft Tools” OR “MS Tools” OR “text generation” OR “phrasing” OR “report generator” OR “background analysis” OR “explaining text” OR “compiling information” OR “code description” OR “code explanation” OR “understanding code” OR “explaining code” OR “analysing code” OR “code analysis”)
Split Search String
("GenAI" OR "Generative Artificial Intelligence" OR "Generative AI" OR “LLM-Aided” OR “LLM-Based” OR “ChatGPT”) AND (“automation"" OR "automated" OR "automating") AND ("software development" OR "software process" OR "agile" OR "scrum") AND (“requirement analysis” OR “requirements engineering” OR “RE” OR “requirement breakdown” OR “use case” OR “user story” OR “epic”)

Table 3.6: Search Strings Used for Scoping Review

3.3.3 Eligibility Criteria

The papers were considered eligible if they were published between 2017 and 2025 from the year the Transformer (the ‘T’ in GPT, generative pre-trained transformer) was first announced [92] up till the time of this thesis study to capture recent developments in the field. This study only considered papers in English language. Papers discussing automation of automation of non-coding tasks conducted by product owners, as identified during the analysis of RQ1 data, were included for full-text screening.

Eligible publication types included published research papers, empirical study, review article, technology review, editorial opinions, discussion papers, grey literature, and student theses. Grey literature and student theses were included because they follow scientific research methodology and are valuable in shaping the research direction for the relatively unexplored field of GenAI for automation of non-coding software development tasks.

Some exclusion criteria were also considered such as pre-prints of published work were not included to avoid duplication. Studies not relevant to the domain of software development or about the automation of coding-related tasks were excluded because they did not fit the scope of this study. Papers without full-text access were also excluded because they could not be used during full-text screening. Lastly, student assignments, commercial websites, and blogs were excluded as these text types are not intended to be published as research.

3.3.4 Paper Screening

First, a title and abstract screening was conducted to include relevant studies based on the eligibility criteria. The papers were marked as included if they were to be included for full text screening, excluded if they were excluded from the review or maybe if the student was unsure the paper matched the eligibility criteria. The reasoning for including, excluding and uncertainty for papers was motivated by short notes within the tool.

Reviewers reviewed a random sample of included and excluded papers to validate screening done by the student and reviewed papers that were marked as "maybe" by the primary investigator. The reviewers also added their notes on these papers, and only one conflict was identified. The disagreement was resolved through discussions, and the paper was decided to be included but treated separately as it was a systematic literature review.

This was followed by full-text screening of selected papers to extract relevant data described in the review protocol. The papers were again marked as included, excluded or maybe and the data was extracted for included and papers labelled maybe. The extracted data was also reviewed by the reviewers, and the papers marked as maybe were discussed and resolved before being presented in the results.

3.4 Implementation

A prototype was developed with the intended purpose of streamlining the workflow described by product owners in initial interviews and automate parts of it to address their pain points. The prototype was developed using HTML, CSS, JavaScript for frontend and Python for backend. The prototype had integrations with Jira for fetching product requirements, Microsoft Outlook for emails, and a GPU server to access GenAI models. The prototype was deployed using Docker. The prototype integrates with the local file system of the user, specifically the OneDrive folder of the user, to store the files locally even if the Docker container is deleted.

The prototype was implemented as a Flask web application which ran locally on the user's system. The user interactions were handled via a web interface, the business logic was in the backend directory, and Flask routes acted as the communication layer between the backend logic and frontend interface. Jinja2 templates were used to render backend data dynamically on the frontend. A basic interaction flow for different tasks integrated in the prototype was that a user clicks a button on the frontend to trigger a request, the request is received by Flask route, the backend processes the request and sends a response which is rendered dynamically on the frontend as different components depending on the type of request.

3.4.1 Script-Based Automations

The prototype was developed to capture the workflow of product owners from receiving a requirement to handing it over to the design team for development. The prototype visualised the workflow of product owners in this context, taking them from one task to the next and displaying the progress. Product owners could work on multiple requirements at a time and access them in the sidebar under ongoing

requirements. The prototype was integrated with Jira using the API to retrieve the requirements from the tickets. The requirement ID and title data were used to initialise the workflow for a requirement. These values were passed through the workflow to track the status of the tasks. The user could then come back to the workflow later and continue where they left off.

The administrative tasks that product owners conduct in their workflow, which were often time-consuming, repetitive, and boring in nature were automated. The data from Jira ticket was used to populate the placeholders in the report templates with details such as requirement ID, title, and summary. The prototype then saved the generated artefacts in the user's OneDrive folder that they could access later. This included the generated draft of reports, the working document for reports and approved versions of reports. The data was also used to populate standard emails that product owners send out during the workflow. The emails were usually populated with Jira ticket information such as requirement ID, requirement title, epic ID, epic title, and link to the main requirements. The emails were also populated with aliases of recipients, subject, email body, and ending signatures based on the user's name provided at the start of the prototype setup.

3.4.2 GenAI-Based Automations

The tasks automated using GenAI were related to content generation and language review of two kinds of reports that product owners work on during their workflow, and serve as the main artefacts in the process. The tasks automated using GenAI in the workflow can be classified into three categories: text generation, text explanation, and language review. The GenAI model used for automation of tasks is *DeepSeek-R1-32B*. An example automation flow can be seen in Figure 3.2 at the end of the section.

A backend service was created to handle the logic for sending prompts to the GenAI model and handling the generated response using Python functions. The model was prompted using different prompting techniques depending on the use case. The prompting techniques used were one-shot and few-shot prompting for text generation and few-shot prompting for text explanation tasks and language review. Regardless of the prompting technique used, each prompt had the suggested prompt elements: a role, a set of instructions, context, and input data.

3.4.2.1 Text Generation

The prompt below in Table 3.7 is an example of the prompts used to generate text suggestions for different sections of the reports either using a previous report or information from Jira tickets. Each section had its own support function instead of filling the entire report at once. The text suggestions were generated for each section, similar to the above prompt and by changing the examples given in the prompt. The examples for functional requirements are not included in the example below because of their sensitive nature. The model's response was then dynamically displayed on frontend for the user, and the details of the prompt and backend service were abstracted. The text suggestions were generated for 4 sections of the first report and 11 sections of the second report during the first iteration of implementation.

You are an expert in technical writing and functional requirements engineering. "I will provide you with the content of a PowerPoint document called {Report-1}. This document is used as the base to generate a {Report-2}. Your assignment is to generate the following information exclusively based on the content of the {Report-1}." "The information to be generated is:" ** Functional requirements ** "Output requirements must not include process requirements (activities to be performed by the developing organisation)." "List detailed functional requirements. Functional requirements should be phrased as detailed statements of capabilities, behaviour, and information that the solution will need." "The requirement is in the form 'product ABC shall XYZ'. A requirement should state "The product shall (do, perform, provide, weigh, or other verb) followed by a description of what should be done." "Requirements should state WHAT is needed, NOT HOW to provide it, i.e., state the problem, not the solution. Ask, "Why do you need the requirement?" The answer may point to the real requirement." [Functional Requirement 1 Title] Text about the requirement [Functional Requirement 2 Title] Text about the requirement "Here is an example: <Example of functional requirement from older Ericsson reports>" "Do not add any introduction or closing phrase, such as: 'Here is the generated document based on the provided template and information from the One Pager' or 'I hope this meets your requirements! Let me know if you need any further assistance.'" {Report-1} Content: f"{Report-1_content} "Generate the {Report-2} functional requirements suggestion as described above, including all necessary details from the {Report-1}."

Table 3.7: Prompt for Text Generation for Functional Requirements for Report

3.4.2.2 Text Explanation

The second application of GenAI-based automation was to generate and explain a terminology list that is used as a section in the reports. The model was prompted to pick relevant terms from the description provided in the Jira ticket and explain them. The product owner could choose to add the identified abbreviations and their explanations in the report section if they found it relevant. An example prompt for one of the reports is shown below in Table 3.8, which also includes examples of Ericsson-specific abbreviations that are not included in the prompt below due to their sensitive nature.

I will provide you with the content of a Jira ticket. Your task is to generate a terminology list based on the provided information from the Jira ticket. Do a list of the terms with an explanation. Do not give any introduction or closing phrase. Use '-' as the index.

Here is an example, do not add to the response:

- LCM: Life Cycle Management
- OSS: Operations Support System

Information from Jira Ticket:
f"{requirement['description']}"

Table 3.8: Prompt for Text Explanation of Terminology in Reports

3.4.2.3 Language Review

The GenAI model was also used to conduct a language review of the reports with a focus on grammatical correctness, sentence structure, and text flow. The product owners could use this automation feature before sending out the reports to other stakeholders for review. The content of the report was parsed, and any grammatical mistakes and inconsistencies in the text were identified by the GenAI model, and suggestions for improvements were made. An example prompt is shown below in Table 3.9 that was used to conduct the language review of reports.

You are an expert technical writer and grammar specialist.
Instructions:

1. Analyse the following report content for grammatical errors, clarity, and consistency.
2. Provide detailed suggestions for improvement while preserving the original meaning.
3. Focus only on grammatical correctness, sentence structure, and text flow.
4. Do not alter the content, scientific data, or technical information.
5. Format suggestions clearly, marking the original text and your recommendations.
6. Avoid adding any introductory or concluding phrases.

** Content to analyse: **
** <START GRAMMATICAL ANALYSIS> **
f"{report_content}"
** <END GRAMMATICAL ANALYSIS> **

Examples of issues to look for:

- Subject-verb agreement
- Pronoun usage
- Modifiers placement
- Sentence clarity
- Consistent terminology

Analyse just the items described above from the document between the placeholders <START GRAMMATICAL ANALYSIS> and <END GRAMMATICAL ANALYSIS>

Table 3.9: Prompt for Language Review of Reports

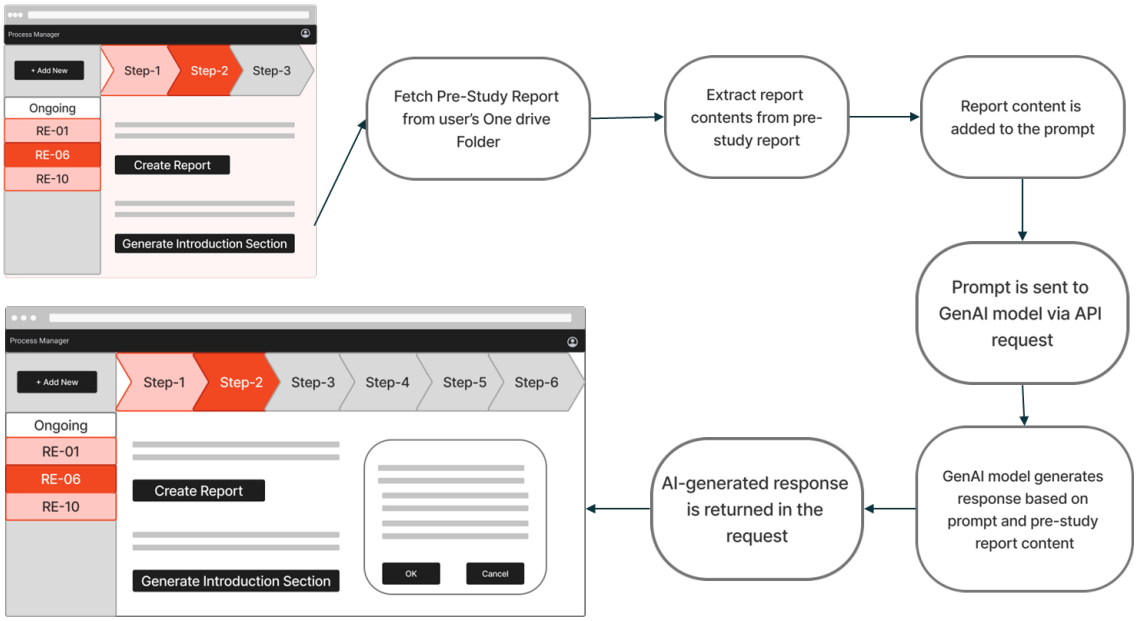

Figure 3.2: Example Automation Flow for Generation of Main Report Text Suggestions from Pre-Study Report or a Requirement

3.5 Exit Interviews and Survey

During the evaluation phase, 30-minute exit-interviews and a System Usability Scale (SUS) [18] survey were conducted to evaluate the prototype and GenAI automations within it to answer RQ3 with the same five product owners from initial interviews and observations. A demonstration was done to give product owners a walk-through of how to set up the prototype and how to use it before the exit interviews and surveys were conducted. The product owners then interacted with the prototype for almost a week till they had used most of the features to solve a task before scheduling the interviews.

3.5.1 Data Collection

The data for RQ3 was collected using the SUS survey [18], which was adapted for the prototype and the purpose of the study. The participants rated their experience on a 5-point Likert scale for 18 questions, where 1 indicated "Strongly Disagree" and 5 represented "Strongly Agree" [18]. If a participant felt they could not answer the question, they marked the centre of the scale as their choice [18]. After survey responses were received, the participants participated in the 30-minute exit interviews. For convenience, the survey was handed out right before the interview since the survey should be answered before any discussions are done for not biasing participants' evaluation [18].

The questions focused on the experience of product owners while using the prototype in their workflow and the automated support for tasks implemented using GenAI. The interviews were semi-structured with open-ended questions to allow for follow-up questions and open discussion. In-person and online interviews on Microsoft Teams were scheduled based on participant availability given that they had

interacted with the tool to solve a task in a meaningful time. The interviews were recorded for transcription and analysis later.

3.5.2 Data Analysis

For survey analysis for RQ3, Likert-type items represent similar questions combined into a single composite score/variable, i.e., whether the automated support tasks are good to use or not. Therefore, the data is analysed as interval data, making the mean the appropriate choice for measure of central tendency. A single-group t-test was conducted to identify if participants had positive or negative feedback for the prototype and automation. To analyse the rating on each question, the median is a more appropriate measure of central tendency to measure the frequency of responses since Likert items are ordinal by nature.

A thematic analysis is conducted for interview transcripts to answer RQ3 and identifying the potential and limitations of the prototype and GenAI automations in alleviating the pain points identified in initial interviews and observations. The transcripts were labelled using codes, and themes were identified. The analysis was iterative, and the codes and themes were refined with each iteration to combine, merge and split them as needed.

An alternate research method for RQ3 is conducting experiments, for instance, with a one-group pretest-posttest study design [15]. The experiments can be conducted to use appropriate metrics to measure the quality of AI-generated responses and collect quantitative data. However, interviews and survey are prioritised to collect qualitative feedback about experience with the automated support for tasks to fit with the given time frame for the study, because the emphasis is on investigating the perceived potential and limitations of GenAI.

3.5.3 Data Validation

The company supervisors reviewed the data analysis of the survey responses and thematic analysis to validate the RQ3 findings, such as parts of the transcripts, the coding scheme, and themes. The changes recommended by the reviewers were discussed and considered to refine the analysis. The survey and interview findings complemented each other to further validate the findings for RQ3.

Chapter 4

Results and Analysis

4.1 Participant Demographic

The participants for the study are product owners at the organisation with the role title of system manager, working in the same product but with different requirement areas. The product owners have been at the organisation for almost 10-30 years and have system manager experience ranging from 3-15 years. All product owners have prior experience in other roles at the organisation before becoming a system manager. They have similar educational backgrounds in computer science or telecommunications, and some product owners have a technical background with programming through their prior developer roles or education. The demographic information of participants in the study is detailed below in Table 4.1.

Participant	**Experience in PO Role**	**Educational Background**	**Requirement Area**
PO1	15 years	Telecommunications	Networking and security
PO2	3 years	Computer Science	Performance management and provisioning
PO3	12 years	Computer Science	Provisioning
PO4	4 years	Telecommunications	Robustness
PO5	15 years	Computer Science	Cloud and performance management

Table 4.1: Participant Demographic for the Study

4.2 Initial Interviews and Observations

The interview data is analysed using thematic analysis, and the short notes from observations are compiled to answer RQ1.1 and RQ1.2. Five product owners were interviewed for 45-60 minutes each using a semi-structured interview guide. Two product owners were observed for 12 hours, with one full-day observation of 8 hours and a partial observation of certain tasks, adding up to 4 hours. The time duration for initial interviews and observations with participants are summarised below in Table 4.2.

Participant	Initial Interview	Observations
PO1	45 mins	-
PO2	60 mins	8 hours
PO3	50 mins	-
PO4	55 mins	4 hours
PO5	50 mins	-

Table 4.2: Time Duration for Initial Interviews and Observations

4.2.1 Thematic Analysis

A deductive thematic analysis is conducted, and 42 codes are identified. A detailed list of identified codes, their descriptions, the number of participants mentioning them, and examples are available in Appendix A. The codes are merged to formulate 12 themes and then separated to formulate 2-5 sub-themes for 4 of the themes. The generated themes along with their description, sub-themes, and codes used to generate them are described below in Table 4.3. The thematic map created for the findings can be seen below in Figure 4.1. The findings for each theme are described in a narrative text in the next section.

Theme	Description	Sub-Themes	Codes
Process workflow	The theme describes the workflow of product owners at the organisation from receiving requirements to delivering them.		Study initiators, steps for starting the workflow, adding content to documents, review of documents, steps after approval of documents, steps during implementation of requirement
Artefacts in the workflow	The theme covers artefacts, documents, etc., produced and maintained by product owners during the workflow.		Documents produced in the process, documents maintained in the process
Involvement of teams and roles	The theme describes other teams and roles involved in the workflow and their interaction with product owners.		Involvement of other teams, involvement of other roles, stage of involvement
Tools in the workflow	The theme covers different types of tools utilised in the workflow and how they are used by product owners.	1. Productivity Tools 2. Management Tools 3. GenAI Tools	Tools used in the process, GenAI tools used by product owners, how is the tool used in the process
Existing support for tasks	This theme covers the positive and negative perception of existing tools and automation support in the process.		Positive comments about existing tools, negative comments about existing tools, previous efforts for automation

Non-coding tasks	This theme describes the different types of non-coding tasks conducted by product owners during their workflow.	1. Requirement Analysis 2. Communication 3. Compiling Information 4. Documentation 5. Artefact Management	Requirement tasks, meetings, reading and compiling information, documentation and management, answering customer questions, handling escalations
Categorisation of tasks	Under this theme, the non-coding tasks are categorised into different categories identified.	1. Time-consuming Tasks 2. Repetitive Tasks 3. High-value Tasks	Time-consuming tasks, repetitive tasks, high-value tasks, administrative tasks, boring tasks
Estimation for tasks	This theme describes the time estimate for tasks, frequency of task occurrence, and size estimates of documents.		Time estimate for time-consuming tasks, frequency of occurrence of repetitive tasks, size estimate of documents
Pain Points	The theme covers the pain points experienced by product owners during the workflow and their causes.		Pain points in workflow, causes of frustration, causes of context-switching
Perception of GenAI	This theme covers the positive and negative perception of the usage of GenAI tools in the process and the quality criteria of generated responses.		Positive comments about GenAI use, negative comments about GenAI, expectations of quality criteria
Automation suggestions	This theme describes the suggestions made by product owners about automation in the workflow with and without using GenAI.	1. Script-Based 2. GenAI-Aided	Automation suggestions, suggestions for GenAI automation, factors for choosing candidates for automation
Expectations about the impact of automation	This theme details the expectations of product owners about automated support for tasks and its impact on process, daily tasks, and pain points.		Expectations of product owners, impact of automation

Table 4.3: Themes, Description, and Codes Identified in Thematic Analysis

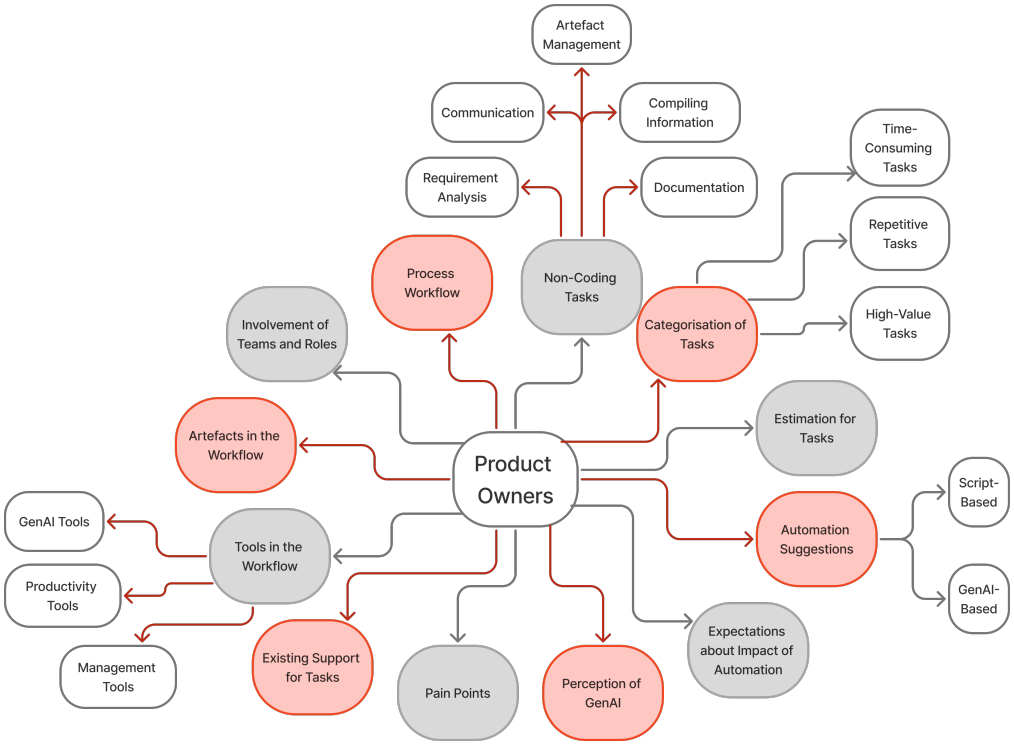

Figure 4.1: Thematic map showing themes and sub-themes identified from initial interviews

4.2.2 Findings from Initial Interviews

The findings for each theme and its sub-theme are detailed here, starting with the identification of the process workflow of product owners, the artefacts generated during this workflow, and the other teams involved in the process. This is followed by identification of tools utilised in the workflow and how well does the existing support for tasks work. The non-coding tasks conducted by product owners are described along with their classification into time-consuming, repetitive and high-value tasks, along with estimates for the same.

This is followed by findings about pain points experienced by product owners in the current implementation of the process and the causes behind it. Their perception of GenAI tools and expectations of quality criteria of generated response is also included in the findings. The suggestions made by product owners during interviews for candidates for automation and factors for choosing them are described, followed by the expectations of product owners in terms of the impact of automated support on the workflow of product owners.

Process Workflow

Product owners receive a business requirement, which could be initiated internally or by customers directly. The requirement may be new or an enhancement feature for an existing product. Product owners might do a pre-study to analyse which other components and products will be impacted by the requirement and create a report. Product owners read and compile information from different sources (Internet, stan-

dardisation documents, internal design documentation, and source code). They may also contact people in different teams that are being impacted by this requirement, as identified in the pre-study phase. The report is sent out for review and discussed in a review meeting.

After the pre-study is approved, product owners do a series of stakeholder meetings to elicit requirements, done via Microsoft Teams or in-person meetings. They, then, write another report in a Microsoft Word template. They again access and compile information from multiple sources (Internet, standardisation documents, internal design documentation, and source code). The output of this stage is refined technical requirements specified as epics and user stories. The report is also sent out 72 hours before a review meeting. The review meeting is conducted to go over the report and address any comments. The meeting is attended by, but not limited to, the architecture team, other product owners, and stakeholders.

Once the report is approved, product owners have a meeting with the design team, who are responsible for the implementation of the requirement. The meeting usually happens several months after the report is approved. In preparation for this, the main requirement is added to Ericsson's internal tool, and epic and user stories are added to Jira and synced across both tools, after which product owners call for another standard meeting. Any comments from the design team are addressed during this meeting, and an implementation proposal might be created and discussed.

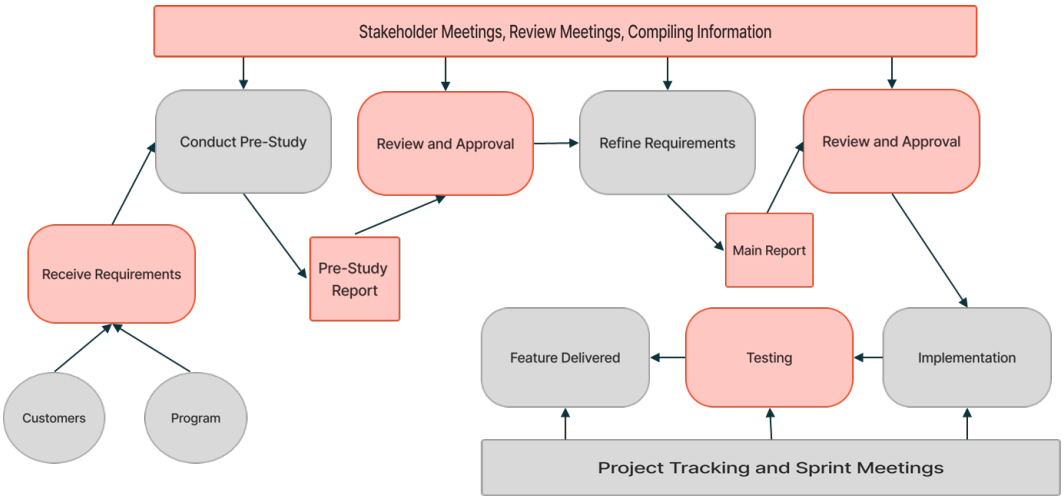

Figure 4.2: Process Workflow of Product Owners

The requirement is implemented, and meetings are called by the design team during this time with stakeholders outside the team like line managers. A meeting invite is sent before each meeting, and an approval status is sent after the meeting. The epic is then taken over by the testing team after implementation. After testing is complete, the feature is moved to complete status in Jira and considered ready for delivery. The flowchart showing the process workflow of product owners can be seen above in Figure 4.2.

Product owners are responsible for guiding the team during the implementation of a requirement. They track the status of requirements during implementation and sync it across the Excel checklist, Jira and Ericsson's internal tool for handling all requirements. They finally prepare and give a presentation and live demo for delivery.

In parallel to the above tasks, product owners are also involved in answering questions from different teams in person and on Microsoft Outlook and Teams Chat. They also answer questions from the support team, i.e., customer questions on a Q&A tool. They may also review tasks of other product owners, mainly documentation. Lastly, they maintain the artefacts such as documents, diagrams, etc., produced during the workflow.

Artefacts in the Workflow

The main documents created in the workflow are a report created during the pre-study phase and then a main report that drives the implementation of the requirement. A product owner says, "...in the last section of report we write user stories. We explain the details, what we need to implement". The other artefacts produced in the workflow are epics, user stories, checklists for requirements, and system specification documents. Another product owner explains, "It's a requirement that we break in one EPIC. So all of these requirement text breaks are in one EPIC. And this one EPIC generates multiple stories". In addition, minutes of the meeting are produced for most meetings that are considered important. The artefacts produced are maintained during the workflow of a requirement. The findings imply that the workflow has several different documents being produced that are sometimes related to each other.

Involvement of Teams and Roles

Product owners interact with initiators of the requirement and project managers responsible for release planning and business decisions in the early stages. They continuously interact with stakeholders who may be internal or external to the organisation during the workflow. As one product owner explains, "First part is just to I would say reach out and ask certain questions to relevant stakeholders, just get a better understanding of what this requirement entail, and from that point it usually renders into a series of meetings". A total of 11 different roles were identified that product owners interact with, which also relates to the findings of existing research that finds that product owner roles can interact with up to 15 different types of roles [90].

They also interact with architecture, design, and testing teams during the implementation of a requirement. They may interact with other product owners during the workflow for questions or feedback. Another product owner explains, "Once the report is approved, then we can have this meeting with the design team. That will be the team responsible for implementing the feature". Product owners interact with a configuration manager responsible for creating document collection for epic in an internal tool of the organisation. They may also interact with teams outside of their product if the requirement impacts more than one product.

Tools in the Workflow

There are several tools utilised in the process, and they are categorised into productivity, management, and GenAI tools. Productivity tools refer to tools utilised for

creating artefacts and communicating with people, whereas management tools are being used for project management and tracking. The findings show that there are several tools involved in the process, indicating that information either needs to be compiled from different data sources or synced across them.

Productivity Tools

The productivity tools utilised in workflow are mainly from the Microsoft suite, such as Microsoft Word for writing reports and documents, Microsoft Excel for sprint checklists, and Microsoft PowerPoint for the first report and presentations. A product owner mentions, "...so pretty much we have the office package there. It's very much involved in our daily activities". The informational documents and slides are shared over SharePoint. Product owners use Microsoft Teams for online meetings and messages, and Microsoft Outlook for emails. Product owners may also utilise the following tools depending on the use case: XMind for visualising mind maps, PlantUML for UML diagrams, Visual Studio Code for source code, Git for version control, and Gerrit for code review.

Management Tools

The management tools utilised in the flow are Jira for project management and maintaining epics and user stories. An internal tool is used for all requirements in the organisation that serve as the main requirement for any project. An internal Q&A tool is used for answering customer questions, and another internal tool for looking up organisation-specific information. One product owner explains, "We have this Q&A tool which is basically kind of a quick question and answer tool that customers or the customer units basically can send in some quick questions to clarify". The documents generated during a requirement workflow are uploaded to an internal tool for maintenance and approval.

GenAI Tools

The organisation only allows the use of company-controlled GenAI models with which they have agreements. Two internal GenAI tools exist that product owners make use of, one for asking questions about organisation-specific documentation and the other for more generic questions about technologies and technical concepts. Product owners say the following about existing GenAI tools in their flow, "...it gives at least some hints of which document you can find the next information rather than me going in and searching. So it saves time... ".

Existing Support for Tasks

Product owners consider the existing support for tasks through the above-mentioned tools good enough for their purposes. While the tools offer relevant functionalities, the artefacts have to be manually updated in the tools since there is no sync of tools in the workflow. One product owner reflects, "...many manual steps and copy pasting, and so on... for example, Jira to Word. Control C Control V is the tool". Only a few efforts have been made in automating the workflow so far. Some examples

include: user story is moved automatically from testing state to complete status after the testing team moves the Jira ticket, automatic generation of sequence diagrams from logs, and automatic creation of a Jira ticket from the main requirement in the internal tool. However, these automations have not had a noticeable impact on the workflow of product owners yet, because while the automations work, they are not used that frequently because they are isolated from the workflow.

Non-Coding Tasks

Several non-coding tasks are mentioned in the workflow that are categorised into sub-themes of requirement analysis, communication, compiling information, documentation, and artefact management. Requirement analysis includes tasks related to requirement elicitation and analysis, and documentation refers to creating reports and other artefacts required during the workflow to document the requirement and solution. Communication includes interactions with teams and other roles in the process, whereas artefact management is for maintaining the artefacts created during the workflow.

Requirement Analysis

Product owners may do a pre-study to understand how requirement impacts other requirements and products. They may elicit requirements from stakeholders if a pre-study is done. They refine requirements to be more specific and document them as epics and user stories for development teams. They are also responsible for analysing the impact of requirements and how to incorporate them into existing systems. As one product owner explains, "...typically one of the biggest challenges when we embark on new requirements that need to be incorporated maybe into an existing solution". They estimate man-hours for the requirement that is used to allocate budget and resources for the implementation.

Communication

Product owners communicate with other roles and teams through messages, emails, meetings and in-person discussions. They attend review meetings for documents where they present the report and discuss feedback comments to address them. They also attend meetings with different stakeholders in different kinds of meetings, such as work meetings, sprint meetings, etc. For each meeting, they may send out meeting invites, minutes of the meeting, and status approval if it was a sprint meeting. One product owner reflects, "...and you know having meetings with customers and external stakeholders, but it's also internally, can be development teams that needs guidance".

Compiling Information

The most common tasks identified for product owners are to read and collect information for pre-study, create reports, and answer queries. For example, one product owner reflects, "So I like to spend a lot of time in, I like to call it like a due diligence phase to build up the understanding both myself, but also write it down in a way that I can convey to the team, primarily to receiving it, because it's more motivating

for them as well to have a more holistic understanding of what they're building and why they're building it and what it is used for".

They may read about technologies available for the required solution on the internet or read old documents to understand what decisions were made previously. They may have to find information about other products, which may entail finding a contact person and waiting for their response. Another product owner notes, "...or else you need to go to ask the teams sometimes and then you need to find which team... it can take hours, easily". Product owners also find and understand source code in repositories through manual searching and filtering for their reports. They may also need to understand behaviour of the system, run manual tests, maintain API calls, and populate the database when preparing for live demos for the customers. They frequently communicate with teams, other product owners, and customers to answer their questions and resolve any challenges being faced by the team.

Documentation

Product owners are responsible for documenting the reports created during the workflow, which usually include sections on the background of the requirement, the domain of the problem, formulating and explaining requirements, solution, and impacted architecture components in the templates. They also fill in the administrative content, such as name, IDs, etc., in these report templates. They create epics and user stories with IDs and sync them manually across tools. They may also review documents from other product owners for editorial updated. They may also be responsible for a subset of official product documents. One product owner explains, "...so get the template for the report and then start filling in, copy and paste the requirement into the document, and so on. All these kinds of references, there is some background text and so on and then fill in all sorts of details".

Artefact Management

Product owners not only create the artefacts, they are also responsible for maintaining them during the workflow and after. This usually includes updating the status of user stories and epics in Jira and maintaining sprint checklists for user stories. They also maintain the reports created during the workflow if there is feedback from other stakeholders or if requirements change with time. They also need keep up with changes made by other teams in documents and artefacts. Lastly, they maintain schemas such as external interfaces and use cases during the workflow, which should be clear and not up for interpretation to implement the requirements correctly. As one product owner explains, "...but we want to maintain system documentation, which describes the current behaviour of the product... and that's typically the use cases, the sequence diagrams. I mean, anything that helps to describe the actual implementation is important to maintain...".

Categorisation of Tasks

The non-coding tasks are further categorised based on the type of task, which could be time-consuming, repetitive or high-value tasks. Time-consuming tasks are tasks

that take a long time to complete, repetitive tasks refer to tasks that are not necessarily time-consuming but happen frequently to take up time. High-value tasks are defined as tasks with a high impact on the successful delivery of project if not done correctly.

Time-Consuming Tasks

The tasks identified as time-consuming tasks by product owners are pre-study, when incorporating a new requirement into existing systems, especially if it impacts other products. The second task identified is compiling information and artefact management during all stages of the workflow. Some specific time-consuming tasks are filling in report templates with standard information such as name, employee ID, etc. and writing introductory sections of reports and explaining requirements. One product owner explains, “Everything that I consider administrative is very exhausting... I have to spend a lot of time to just copy that information into Jira and making sure that everything is in place".

Repetitive Tasks

The repetitive tasks identified by product owners are tasks related to meetings, such as sending out meeting invites, making meeting notes, and sending out approval status emails after a sprint meeting. They also find reviewing documents to be repetitive, but this may not be done by all product owners. They are responsible for reviewing documents within their responsibilities, or they may do it out of their own interests. They also find answering queries from teams in chat and emails, or customers in Q&A tool, a repetitive task that takes their time in total due to its frequency. Another product owner reflects, “We have this Q&A tool... So those are some things that we repeatedly use and answer questions about. And in that, again, there are things that are repeated. So one customer asks or one customer unit will ask something, but then there are already similar questions that have already been answered".

High-Value Tasks

The tasks related to requirement analysis, documentation, and artefact management are identified as high-value tasks by product owners. This is because if requirements are analysed or documented incorrectly then they are not developing the correct solution. They also believe that if artefacts are not maintained and synced across different tools, it can cause confusion and misinformation among teams about a requirement, which can also have an impact on the delivery of the requirement. As one product owner explains, “It’s crucial that the interface artefacts or interface specifications and schemas, the use cases of the output requirement, that those are crisp so they are they’re not subject to interpretation. It must be clear to anyone who is digesting that information, this is what we intend to implement basically".

Estimation for Tasks

Product owners estimate that a pre-study can take between 4 weeks to a couple of months to complete, "...everything between a couple of weeks and a couple of months". Compiling information can take between hours to a few days, while writing background and standard things in the report can take a few hours to a few days, "...depending on the size of the requirement, but I would say a minimum of three days". They highlight that artefact management is ongoing throughout the study.

Meetings happen frequently and can last from 30-60 minutes, depending on the type of meeting. For example, sync meetings are once a week, meeting requests can be received a few times a week but are only attended a few times in a month, and stakeholder meetings may be a few times a week. Reviewing documents takes a few hours but happens every other day, "...every second day I would say, and that review process I'm really going to investigate and especially if it's a new function allowed to be added so two hours maybe" and answering queries can also happen every day and may take a few hours to a couple days depending on type of request.

Pain Points

The biggest pain point experienced by product owners in their workflow is context switching when they have to move to a different task in the middle of an ongoing task. This usually happens when a question comes from development teams or customers, or someone drops by to discuss something. Other pain points experience by product owners are the maintenance of artefacts across different tools, finding information about whom to contact regarding their questions, and waiting time for information from other teams and roles that delays decisions.

In addition, they point out that manual tasks such as copy pasting information and finding documents causes frustration that drains their energy. They also find it challenging to keep up with changes in various documents for different requirements. Moreover, escalations might happen in a product that needs to be prioritised, and the rest of the work is put on hold till the escalation is resolved.

As product owners explain, "...and then the very important part is that not all questions are about the same thing so it's a lot of context switching because you might be working on something, the question is totally something else, then you need to really... it's a full context switch it takes time to really get to that level of okay understanding and then answer that properly" and "...when I go into try to find something and you have like the flow of things and then comes in some urgent questions in middle of that and it takes time to go back again, and you always have to start on square one sometimes".

Perception of GenAI

GenAI tools are being used by product owners at the organisation for researching concepts and getting quick summaries of information. A common blocker for using GenAI in their workflow is that product owners are unsure what information can be put in the prompt to avoid sending in any company-sensitive or personal information to GenAI models. One product owner says, "...if you're not directly involved in this

AI journey, you're a bit unsure what kind of information you can put in there". Another shortcoming experienced by product owners is that the current internal models can be slow and forget context after a while. They feel that they can do the task themselves in the meantime and avoid the hassle. However, they remain positive that more powerful models will be used sooner rather than later.

Product owners expect correct information in generated responses, but they perform a sanity check and cross-check information produced. Another product owner mentions, "... as long as I can do a sanity check, like a cross check, everything's okay, and I feel comfortable with the changes, I have no problem with AI helping out with that". The information is not always complete, so they have to continue looking for documents manually to get answers for their query, especially if it is specific to the organisation. However, they do see potential in GenAI for tasks related to brainstorming and writing content for documents. They believe it is a useful tool for overcoming writer's block and improving the quality and flow of text in documents.

Automation Suggestions

The product owners made suggestions for automation candidates based on how time-consuming they are, the impact they could have on their workflow if automated, and tasks with administrative overhead, "...there's a lot of things that needs to be in place which essentially requires a lot of just administrative work from the product over the system manager in question so anything that can help to to automate those kind of tasks and that would be ideal". Some other motivations for suggesting automation candidates are to get repetitive tasks out of the way and streamline the workflow.

Script-Based Automations

Some of the automation suggestions included an automation to eliminate copy pasting of content from one document or one tool to another by having some kind of link between different tools so things do not have to be updated individually. Product owners also wanted a mechanism for the maintenance of artefacts and documents produced during the study, and keeping up with updates made by others in a document. Some other automation suggestions were pre-filling and sending out meeting invites for meetings and approval status after meetings, syncing the sprint checklist to update the status of user stories automatically and preparing for live demos by testing APIs, populating databases, and running tests. One product owner explains, "...and then I can think that some people might think that, like, things in the templates being pre-filled automatically would maybe save a lot of time. But I think these minutes of meetings and... and the Jira, Excel, moving things to write status would be very good. Would just save unnecessary space in my brain".

GenAI-Aided Automations

Product owners wanted to use GenAI automation for getting quick summaries of requirements, documents, and source code and generating text for reports such as the background section and explanations of terms. Some other suggestions included

GenAI for compiling information from different sources and documents, and prompting the model to generate phrases and texts to make descriptions clear in the documents. Product owners also saw potential in GenAI for document analysis to tell where the content should be updated if requirements are changed. They also suggested the use of GenAI-assistants in meetings to get a summary of discussions and generate minutes of the meeting. Lastly, they suggested an AI-powered chatbot to answer customer questions based on previously answered questions. One of the product owner says, "...when it comes to summarising this to maybe someone who is not that maybe technically inclined or things like that, it would be ideal to have a tool that, of course, could capture the main essence of your report without you having to phrase it all over again, but in a more simplistic form."

Expectations about Impact of Automation

Product owners believe automation of their non-coding tasks and the workflow will help them save time and spend it on high-value tasks. They expect to focus on requirements that have been in the backlog for a long time, with the saved time. They expect that manual errors, such as missing steps and irregular information can be avoided by automating administrative tasks. They expect to have consistency across different tools and documents, and know they can rely on it in the future instead of looking for missing information if they come back to a requirement after a few months. As one product owner explains, "...but what I think impacts. It's like when not everyone remembers to do it. And then we got some inconsistencies when verifying the status of an epic. And well, the information can be misleading because it's a long process".

The impact of automation of time-consuming tasks will be having time for high-value tasks, such as finding a solution and making business decisions. The automation of repetitive and boring tasks will save their mental energy for other tasks during the day, "I think it would save some time and mental space". They also expect filtering out information clutter will help focus on the right things such as business solution and prioritise tasks that usually get ignored, "...it will allow us to focus more on maybe thought leadership research oriented maybe how to bring our product forward give us more time for free thinking so to speak and not just getting stuck in the development flows". They also expect to have more time for personal development.

4.2.3 Findings from Observations

The findings from observation complement the findings from initial interviews. The tasks that are not mentioned in the interviews are captured during the observation, such as product owners attending an update meeting with the study initiators to discuss the status and roadblocks of ongoing studies. The meeting usually lasts for 30 minutes and happens once a week. Product owners may have work meetings to discuss changes in reports before calling for a review meeting that can last for 30-60 minutes, and are called when necessary. Product owners also spend time on preparing for the meetings, which includes getting all the necessary documents ready for the meeting and reading required for it, which was not accounted for in time estimates for the task in interviews.

The observations also allowed for identifying patterns in the daily life of product owners that affect their workflow by being causes for context-switching. Product owners may have their employee tasks, such as employee meetings or union meetings, that can be a cause for context switching. Another cause for context-switching could be discussions that grow longer the more people get involved. The problem may not even get resolved, which means the discussion needs to be done again, which also makes communication a time-consuming task, depending on the context. However, they may get some of their work done during meetings when parts of the meeting are not related to them, emphasising that not all meetings are as important as claimed by product owners in initial interviews.

4.3 Scoping Review

During the scoping review, 1404 papers were collected from the search on four databases. Among these papers, 460 duplicates were identified, of which 142 duplicates were removed before screening. The remaining 1262 papers were screened using title and abstract screening, of which 1217 records were excluded. 44 papers remained for full-text screening. There was no full-text access for 3 of the papers, so 41 papers were assessed for eligibility. 19 papers are excluded based on eligibility criteria defined in the scoping review protocol in Chapter 3. A total of 22 papers are included in the final analysis after full-text screening. A PRISMA diagram showing the scoping review process can be seen below in Figure 4.3.

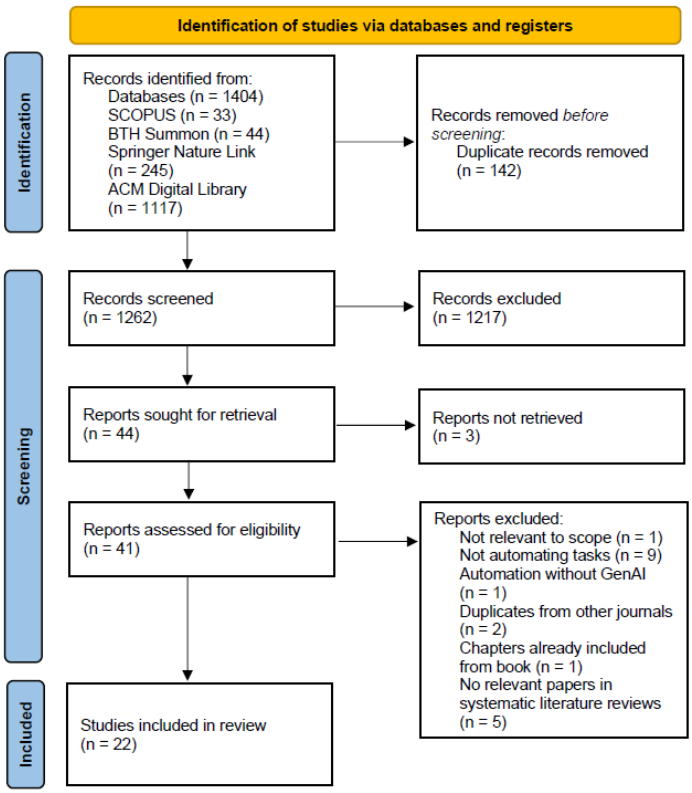

Figure 4.3: PRISMA Diagram for Scoping Literature Review

4.3.1 Findings from Scoping Review

The first finding from the scoping review is the non-coding tasks of product owners that are commonly automated, and which GenAI models are used to automate them, and what level of evidence exists for them. The complete results are detailed below in Table 4.4. The tasks automated using GenAI in order of their frequency are requirement analysis tasks (n=8) such as creating user stories, story point estimation, etc., code explainability tasks (n=6) such as explaining or summarising code snippets from source code, documentation tasks (n=4) such as brainstorming and text generation, and data modelling (n=4) for generating UML diagrams from requirements.

The most commonly used GenAI models for these tasks are OpenAI's GPT-series (n=19), followed by variants of Llama by Meta (n=5) and Bard (now Gemini) by Google (n=5). The other GenAI and LLM models identified are T5, StarCoder, NCS, Mistral-series, and txt2Model. The most commonly used GenAI tools in implementation are ChatGPT and Gemini. The existing research on automation of non-coding development tasks mainly has three levels of evidence: prototype (n=13), proof of concept (n=2), and frameworks (n=2).

The second finding from the scoping review is how the above tasks are automated and implemented. The findings show that tasks can be automated using three approaches: developing a pipeline of modules and LLMs (n=12), by accessing the GenAI models directly through API or loading them locally (n=6), and developing multi-agent systems (n=4). The automation solutions can be implemented in one of the following ways: as a web-app with a user interface for users to interact with it (n=10), executable Python programs (n=8), and using the GenAI tool's interface directly (n=3). The complete results on automation and implementation of non-coding tasks are detailed in Appendix B.

Paper	**Non-Coding Task Automated**	**GenAI Model or Tool Used**	**Level of Evidence**
Sami et al. (2025) [80]	User story generation and prioritisation by developing a multi-agent system.	GPT-3.5, GPT-4.0	Prototype
Naimi et al. (2024) [69]	Use case extraction and documentation from UML diagrams by prompting a GenAI model via an API request with XML files containing use case diagrams.	Gemini, ChatGPT	Prototype
Ronanki et al. (2024) [77]	User story quality evaluation by prompting the GenAI tool using one-shot prompting.	ChatGPT	-
Golia et al. (2024) [35]	Generation of meeting summaries by creating a pipeline of algorithms and GenAI models.	BART	Prototype
Vijayan (2024) [93]	Extraction of structured information from unstructured data by prompting GenAI models with email chains as inputs.	ChatGPT, BARD	-
Fan et al. (2024) [26]	Code search by creating a pipeline of tasks that interact with GenAI models.	T5, Bert	Prototype

Fazelnia et al. (2024) [29]	Detection of conflicting requirements by creating a pipeline of a GenAI tool and an SMT solver.	GPT-4	Prototype
Heyman et al. (2024) [40]	Text and idea generation by prompting a GenAI model via API using zero-shot and few-shot prompting.	GPT-3.5 Turbo	Prototype
Abedu et al. (2024) [5]	Extraction and analysis of repository data by creating a pipeline of processing steps built upon LLMs and GenAI models.	GPT-3.5 Turbo	Prototype
Jahan et al. (2024) [44]	Generation of sequence diagrams from user stories using NLP techniques and comparing it against the GenAI model.	GPT 3.5	Prototype
Cabrero-Daniel et al. (2024) [22]	Meeting recommendations and summarisation by developing two different AI agents.	GPT-4	Prototype
Mastropaolo et al. (2024) [60]	Code snippet summarisation by creating a pipeline of tasks built upon pre-trained models.	T5	Prototype
Li et al. (2024) [56]	Code summarisation by prompting GenAI models using few-shot prompting.	GPT4, GPT-few-shot, GPT3.5, StarCoder, Code Llama, and NCS	-
Wei (2024) [94]	Generation of object-oriented models and code from requirements by prompting a GenAI model using progressive prompting.	ChatGPT	Prototype
Zhang et al. (2024) [101]	User story quality evaluation by developing a multi-agent system.	gpt-3.5-turbo-16k, gpt-4-1106-preview	Proof of Concept
van Can et al. (2025) [91]	Identification and classification of requirements in backlog by prompting directly to the GenAI models using one-shot prompting.	BERT, RoBERTa, GPT-4, Mistral 7B, Llama 3	-
Bozyigit et al. (2024) [17]	Generation of domain models from requirements by prompting the GenAI models directly.	txt2Model, GPT-3.5	-
Li et al. (2024) [57]	Generation of acceptance criteria by creating a multi-agent system.	gpt-3.5-turbo, text-davinci-003, OpenLLaMA-7B, and OpenLLaMA-13B	Framework
Gartner et al. (2024) [37]	Detection of contradictory requirements by creating a pipeline of processing tasks and using GenAI models for generating responses.	GPT-3	Prototype
Biswas et al. (2024) [16]	Quality assurance of requirements by creating a pipeline of tasks supported by GenAI models.	Llama-2, Mistral 8x7B, ChatGPT	Framework
Zhang et al. (2025) [100]	Code generation from user requirements by creating an AI agent.	gpt-3.5-turbo, gpt-4-1106-preview	Prototype

Mattis et al. (2024) [61]	Generation of code examples and explanation by creating a pipeline of tasks and prompting GenAI models.	CodeLlama-7B, GPT-3.5, GPT-4	Prototype

Table 4.4: Results for non-coding tasks automated using GenAI models in literature

4.4 Exit-Interviews and Surveys

The survey data is analysed using a one-sample t-test and descriptive statistics to identify whether product owners preferred using the prototype and automation within it or not. The interview data is analysed using thematic analysis to complement the findings from the survey. The findings are compiled to answer RQ3. The time duration for exit-interviews and surveys is summarised below in Table 4.5.

Participant	**Survey**	**Exit-Interview**
PO1	5 mins	25 minutes
PO2	3 mins	20 minutes
PO3	2 mins	20 minutes
PO4	5 mins	20 minutes
PO5	3 mins	15 minutes

Table 4.5: Time Duration for Exit-Interviews and Survey

4.4.1 Findings from Survey

A one-sample t-test is conducted to test whether the mean rating for prototype questions and the mean rating for automation questions are significantly different from the neutral score of 3. It is a two-tailed test to detect both positive and negative deviations from neutrality. The null hypothesis is that the mean participation rating is 3 (neutral), and the alternative hypothesis is that the mean participation rating is not 3, indicating either a preference for or against the prototype and automation.

The first 17 questions of the survey are considered the prototype set because they target feedback on the overall prototype, while the last 7 questions are considered the automation set. The mean of ratings for questions per participant is calculated to be used as an input set for the t-test. This results in two datasets, one for automation and the other for the prototype. Each set has five mean values calculated for five participants. The mean and p-values for prototype and automation groups can be seen below in Table 4.6. Since both p-values are <0.05, there is a statistically significant difference from neutral. The mean is higher than 3 for the prototype and automation sets, indicating participants liked the prototype and automations.

Prototype	t=3.72	p=0.0205
Automation	t=3.31	p=0.0296

Table 4.6: Mean and p-values for prototype and automation set from t-test

The median is calculated to gain insight for each question, and the participants mainly agreed with the questions around positive aspects of the prototype and automation, while they disagreed with the negative aspects. The median for each question in the survey can be seen below in Table 4.7.

Statement	**Median**	**Response**
I think that I would like to use this prototype frequently if it is developed further.	4	Agree
I found the tasks within the prototype unnecessarily complex to complete.	2	Disagree
I thought the prototype was easy to use and follow the process.	4	Agree
I found the various tasks in process were well integrated in this prototype.	4	Agree
I thought there was too much inconsistency in the process tasks in this prototype.	2	Disagree
I would imagine that most people would learn to use this prototype very quickly.	4	Agree
I found the process very cumbersome to do using this prototype.	2	Disagree
I felt very confident using the prototype to do my process tasks.	4	Agree
I think that I would like to use this prototype frequently if it is developed further.	4	Agree
I found the tasks within the prototype unnecessarily complex to complete.	2	Disagree
I thought there was too much inconsistency in the process tasks in this prototype.	2	Disagree
I would imagine that most people would learn to use this prototype very quickly.	4	Agree
I found the process very cumbersome to do using this prototype.	2	Disagree
I felt very confident using the prototype to do my process tasks.	4	Agree
I needed to learn a lot of things before I could get going with this prototype.	2	Disagree
I had fun doing my tasks using the prototype.	4	Agree
I think that prototype is fit for the purpose of doing process tasks.	4	Agree
I think the tasks currently automated within the process scope are useful for me.	4	Agree
I found the automation that fetches requirements and information from Jira useful.	4	Agree
I found the automation that fills standard information in report drafts useful.	4	Agree
I found the automation that populates standard emails useful.	4	Agree
I found the automation that suggests text for reports useful.	4	Agree
I found the automation that explains terminology and abbreviations for reports useful.	4	Agree
I found the automation that does a language review of my reports useful.	4	Agree

Table 4.7: Data Analysis of Survey

4.4.2 Thematic Analysis

A thematic analysis of exit-interview transcripts is carried out to explore the potential and limitations of the developed prototype and GenAI automations. A total of 16 labels are identified that are discussed with their descriptions and examples in Appendix C. During the analysis, three themes are identified with two sub-themes

each and are described below in Table 4.8 and the thematic map can be seen below in Figure 4.4. The findings from the thematic analysis are discussed in a narrative text in the next section.

Themes	Description	Sub-Themes	Codes
Feedback on Prototype	This theme covers the positive and negative feedback of the prototype developed, and the potential product owners see in it for further development.	1. Perceived Potential of Prototype 2. Perceived Limitations of Prototype	Positive comment about tool, negative comment about tool, neutral comment about tool, problems in current implementation, positive impact of tool on workflow, negative impact of tool on workflow, impact of tool on context-switching
GenAI-Based Automation	This theme describes the perceived potential and limitations of GenAI-based automations and the quality of responses generated by them.	1. Perceived Potential of GenAI-Based Automation 2. Perceived Limitations of GenAI-Based Automation	Positive comment about GenAI use, negative comment about GenAI use, neutral comment about GenAI use, perceived potential of GenAI automation, perceived limitations of GenAI automation, feedback on quality of generated responses
Suggestions for Further Implementation	This theme details the suggestions made for further implementation of the prototype, including its features, integration with other tools, GenAI-based automations, and the factors for suggesting them.	1. Suggestions for Prototype 2. Suggestions for GenAI Automation	Suggestion for future prototype implementation, suggestion for GenAI automation, suggestion for non-AI automation, factors for suggestions

Table 4.8: Themes, Description, and Codes Identified in Thematic Analysis

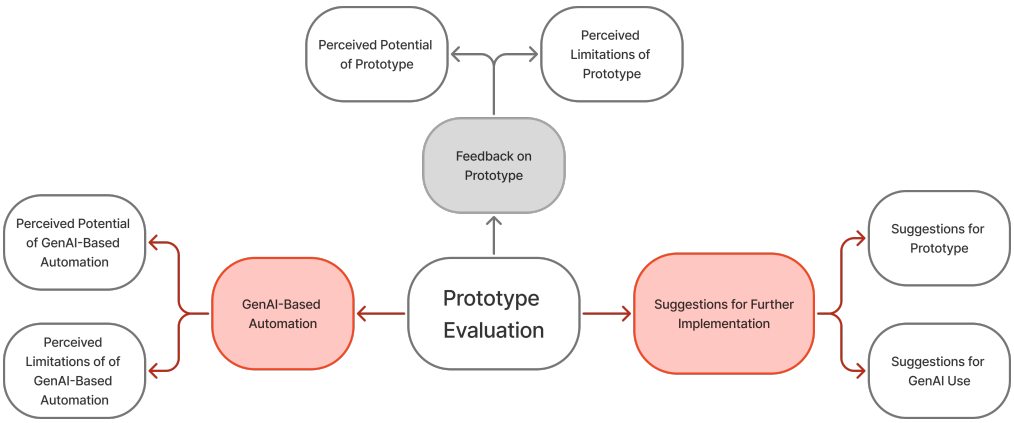

Figure 4.4: Thematic map showing themes and sub-themes identified from initial interviews

4.4.3 Findings from Exit-Interviews

The findings from themes and sub-themes generated for exit interviews are presented, starting with feedback on the prototype, including its potential if developed further and limitations of the current implementation. The product owners also share their experience with using GenAI-based automations for their tasks, including their perceived potential and limitations. The findings also include suggestions for further implementation of the prototype and GenAI-based automations.

Feedback on Prototype

Product owners report that the tool developed simplifies their administrative tasks and helps with time-consuming and repetitive tasks by getting all documents and emails in place with very little manual effort required. As a product owner said, "...usually we copy and paste things in. And then we got that optimised with this tool, and then it's kind of an improvement, certainly." They found it faster to send out emails and fill in reports since they no longer have to fill in and cross-check the details manually, and can get text suggestions from GenAI-based automations.

The prototype improved context switching to a certain extent by eliminating the need for switching between tools. However, at this stage, it does not address context switching that can be caused by changing focus to a different requirement or incoming queries from teams and customers. As a product owner explains, "...the struggle with starting a new requirement is, of course, the context switch. Because you are doing something, you have to start something else. You have to wait, find out what it really is about and try to read up a bit, and try to understand what it is".

They consider the tool to already be in a good shape, considering it is a prototype and working beyond their expectations, so they are already making use of it for their ongoing requirements. One of the product owners said, "I mean already now I could see that this helps me with a lot of the administrative work". They agree most relevant features are implemented, and it is clear and easy to follow the workflow using this tool, as one product owner claimed "...it was very clear for me what the intention was and following the interface".

Perceived Potential of Prototype

The product owners perceive that such a tool can align ways of working for product owners at the organisation if developed further. It would be easy to catch up with ongoing requirements even after some time has passed. One of the product owners says, "I think it's easy to follow what the next step should be, but I think it's a good tool to be used to align the way of working between all system managers. So we all follow the same process, we all come from the same place and have the same structure".

The prototype has the potential to become a go-to tool for product owners at the organisation and can be extended to other roles involved in the process. One product owner says, "This is something I look forward to having a productified version of... and I think that this could be given as an example for other parts of organisations as

well...". The tool could also be a good way to onboard newcomers and introduce them to the workflow of product owners in handling requirements. The product owners also highlight that this tool can enable better alignment with teams involved in the workflow since it can reduce the chance of manual error, causing misinformation and out-of-sync documents across different tools.

Perceived Limitations of Prototype

The limitations perceived by product owners are the missing integrations with internal tools of the organisation that are important for the workflow to be complete. As a product owner says, "...not without the full Jira support. That is needed to remove my pain points. It's a good start, but it's not finished as I see it". They believe these automations will add more value to the tool in streamlining the process.

Another limitation is the setup process of the prototype, since a few product owners faced challenges in setting up the tool initially. The folder structure for local file storage of generated documents is considered another limitation of the tool by product owners because it is time-consuming to fit their ongoing requirements into it, and they find the naming conventions for files very restrictive. One product owner says, "... it's very tight and fixed file name structure right now, so if we can have a little bit more relaxed approach...".

GenAI-Based Automation

The product owners also shared their experience with using GenAI-based automated support for tasks in the workflow, which included text generation, explanation, and language review of reports. The suggestions indicate that they see more value in a streamlined process before the GenAI-based automation.

Perceived Potential of GenAI-Based Automation

Product owners perceive that current GenAI-based automation is good for starting the writing process for reports. The suggestions for generic suggestions, such as background, were considered useful by product owners. They plan to extend the text with more relevant information and rephrase it before adding it to the report. A product owner says, "...what it came up with was quite good for the little information about the requirement. I think in reality, if I have created the report, I will rephrase the text, of course, and then include some more things into it".

Writing suggestions helped them save time during the introduction and summary sections. Another product owner says, "..it would really help me to bootstrap the content creation or writing of the report to at least get all of the headings in place and get some kind of inspiration for that, then maybe". The product owners perceived the language review automation to be quite good and were willing to incorporate suggested changes in their report. A product owner says, "So I think it helps with improving language, but still there is some kind of terminology that you cannot take literally from the review.".

Perceived Limitations of GenAI-Based Automation

The limitations of GenAI perceived by product owners, are that they found organisation-specific information to be insufficient for reports, which affected the quality of responses. For example, explanations for organisation-specific abbreviations were not relevant, and responses for solution-level sections were not close to what a product owner would actually write. As one product owner notes, “I think the background part was good. I mean, very general, but still information that could be used... but for example, functional requirements and solution description, it might be a little bit tricky using the AI".

They also perceive that exhaustive use of GenAI for generating content for reports might make product owners lazy in cross-checking generated text, and there might be large chunks of AI-generated text in reports. For example, a product owner notes, “...one limitation or at least the worry I have is basically it, of course, makes us a bit lazy... and the thing is, as a rule of thumb, I always try to go through the text that is generated... because sometimes I know that AI tries to create something that looks very nice on top on the surface but if you read it thoroughly, it doesn’t really make sense". They perceive that the overconfidence of GenAI models in text generation may cause problems because incorrect content may also seem correct and can be added to reports if not cross-checked. Product owners also note that the automation only provides new content and does not help in improving already written content or in structuring a section.

Suggestions for Further Implementation

Product owners perceive potential in the prototype and GenAI-based automation of tasks within it, therefore, they also make suggestions for further implementation of the prototype.

Suggestions for Prototype

The suggestions made by product owners for future implementation of the prototype are fetching all requirements from Jira and letting the user filter them, instead of only fetching requirements in progress. Some other suggestions include integration of the prototype with more internal tools of the company to capture all tasks in the workflow, and input gathering from various sources such as the internet, previous reports, etc.. A product owner recommends, “I think the Jira integration would be very nice if the epics could be created... the story and so on could be just created and injected in the report".

They also make recommendations to integrate more Jira functionality, such as automatically moving tickets to the correct status when a stage is complete, and automation of sprint checklists. The remaining suggestions are for existing features that were implemented, such as populating the OneDrive link to reports in emails, adding a team alias to review emails, adding a link to file location in local file storage to the dashboard, and introducing the ability to address review comments in the documents directly.

Suggestions for GenAI Automation

The product owners made suggestions for the use of GenAI for more tasks, such as using GenAI to make content in reports simple and break down complex sentences, generating summaries of documents and previous reports that can be used as input for ongoing requirements, and generating UML diagrams using prompts. One product owner motivates, "If someone writes a requirement or a task or something and I know what it's supposed to do, but the text doesn't say that fully, then I usually ask them to throw some AI on it to get it better... better English, better understanding, make it more simple. And that usually helps a lot".

The other suggestions are made for existing GenAI-based automation, which include adding text suggestions directly into the report and adding links in Jira tickets as context to GenAI models for generating responses. The product owners also suggest having a more organisation-specific knowledge base for GenAI models to generate relevant responses for solution-specific sections of reports. Another product owner reflects, "...and of course using more powerful models and which potentially has a knowledge base with like retrieval augmented generation based on the data sources we have, so not only use model that is trained on the internet right, but it's for good reason models are not trained on company-sensitive and propriety data".

4.5 Answering Research Questions

RQ1.1: What are the most time-consuming non-coding tasks that product owners conduct?

The time-consuming tasks conducted by product owners are related to documentation, compiling information, and artefact management, mainly due to the administrative overhead of the tasks. The specific tasks found time-consuming are filling in report templates, finding information from different sources, and syncing information in documents across different tools. The tasks that become time-consuming due to their frequent occurrence are communication-related, which are also the main cause of context-switching. For example, several meetings during the day or waiting for responses from someone before getting back to the task.

The requirement analysis-related tasks are time-consuming by nature, but product owners can not allocate as much time for them as they would like due to the time and energy spent on other tasks. Product owners want to have time to spend on coming up with solutions and understanding the impact of a requirement so they can guide teams through implementation. Therefore, the requirement analysis tasks are also considered high-value tasks because they have a high impact on the successful delivery of a feature. Most importantly, product owners note that the above-mentioned tasks are not considered time-consuming in isolation, but rather time-consuming when done together in the workflow.

RQ1.2: What are the perceptions and expectations of product owners about automation using GenAI?

The product owners expect that automated support for tasks will streamline the workflow, enabling them to focus on high-value tasks such as requirement analysis

and artefact management. They believe the automated support will help them save time in their workflow by addressing the pain points caused by administrative tasks and context switching. They also expect the automated support to standardise ways of working during the workflow so that there are no misalignments between documents and information produced during the process, different aspects of project status, and within different roles and teams.

The product owners mainly have a positive perception of using GenAI for automated support in their workflow. They believe GenAI can help them with text suggestions for documents, compile information and provide summaries, and explain technical concepts and terminologies to them. They, however, do believe human oversight is necessary to ensure that responses generated by GenAI are correct and relevant before using them. The two most common challenges that product owners expect to be taken into account are context windows and hallucinations. Regardless, product owners are interested in utilising GenAI support in their workflows.

RQ2: What non-coding tasks are suitable for automation based on existing evidence in the research using GenAI?

The existing evidence in research for automation of non-coding tasks using GenAI points towards automation of requirement analysis tasks, such as generating user stories and their quality assurance, explainability tasks, such as for explaining parts of code, and documentation tasks, such as brainstorming and generating text suggestions. The evidence also points towards using GenAI's image generation capabilities for generating UML diagrams such as use case diagrams, class diagrams, etc., from textual requirements in the form of, but not limited to, user stories. The GenAI models commonly used in literature for the automation of these tasks are GPT-series and variants of Llama.

The tasks are automated by either developing multi-agent systems for a specific task, or creating a pipeline of pre-trained modules to perform a series of tasks, or accessing the GenAI models directly using APIs. The choice depends on the type and complexity of tasks. The solutions are implemented either as a web application so users can interact with the system through an interface or as stand-alone executable programs for showing proof of concept. There may not be a specific implementation done, but instead the users interact directly with the GenAI tool's interface, such as ChatGPT.

RQ3: What are the perceived potential and limitations of product owners when using automated support for non-coding tasks?

Product owners note that the prototype developed is already capable of streamlining the process and saving time for them by syncing different tools and reports. They perceive that the prototype has the potential to align ways of working and become a go-to tool for their process if developed further. They believe a prototype helps reduce overhead from administrative, time-consuming tasks and context-switching by automating the steps that were done manually earlier and across different tools. Product owners also have a positive perception of GenAI-based automations and consider them to be enhancements for the prototype.

The product owners see potential in GenAI-based automation for improving their writing process by brainstorming with it to overcome writer's block. They find

GenAI's capability for language review of documents to be one of the better automations because of the relevance of responses. The product owners note a major limitation of the automated support, which is that GenAI models do not have an organisation-specific knowledge base, resulting in certain non-relevant text explanations for terms. The product owners do show concern for more GenAI integrations, and if that will make product owners lazy in verifying information and large chunks of AI-generated texts in reports. Overall, product owners show a positive preference for using both the prototype and automations within it in their workflows because they see potential in it to simplify the process and save time.

Chapter 5

Discussion

5.1 The Evolving Role of Product Owner

Scrum defines the product owner role, stating that the product owner is responsible for effective product backlog management and maximising value by clearly communicating product goals [83]. Bass et al. report that product owners often take on a wide array of responsibilities as a groomer, prioritiser, release manager, communicator, and so on [9]. Kadenic et al. note that the tasks and responsibilities of product owners exceed the capabilities of a single individual [46]. In practice, the role of product owner may translate into versions of this where responsibilities may be shared between different roles. Regardless, product owners constantly interact with different levels of organisation, from planning and business teams to development teams.

This is also visible from the findings of the initial interview, in which product owners describe their responsibility covering analysis and management of requirements, dealing with changing requirements, interacting with stakeholders, and guiding teams during implementation. The product owners perceive their role as that of a problem solver and communicator, and inherently challenging due to its nature. The findings also reflect that no requirement workflow is the same and require analysis and decision making with different stakeholders at every step of the way when delivering a feature.

The product owner role sits at the nexus of an agile team. In large-scale companies and projects, there are many moving parts that product owners are responsible for continually aligning. Relational coordination theory posits that high-quality communication and relationships are critical in such complexity [34]. Berntzen et al. find that product owners who prioritise frequent, high-quality exchanges, either scheduled or unscheduled, greatly improved alignment [14]. The findings of this study show that product owners at the company highly prioritise frequent communication, or there is a risk of misalignment. Product owners take part in frequent meetings with stakeholders, implementation teams, and other product owners and managers throughout the workflow. The findings from observations reflect that product owners prioritise answering questions or having discussions with co-workers over their own task that they might be doing at the moment.

Rather than a checklist of tasks, a product owner's work is best seen as the stitching together of an organisational process. Scrum may isolate planning, requirement analysis, and review as separate events, but these events are threads in the continuous cycle of product development. Especially in agile-at-scale, product owners must also coordinate across teams and knowledge domains. Berntzen et al. find that large

programs rely on product owners to coordinate knowledge about project status and goals within and across teams [14]. The same is reflected in the findings for this study, where a product is not developed in isolation, but rather can depend on other products and teams. In this reframed view, the product owner is not simply a keeper of user stories but plays an important role in the product development cycle.

5.2 Rethinking Processes, Not Just Tasks

Scrum is one of the most widespread methodologies used in companies using agile software development, but it often underemphasises how non-coding roles fit into larger organisational processes. A product owner performs many tasks beyond coding, such as requirement analysis, backlog management, stakeholder interaction, and communication with teams [9]. Yet these activities are usually treated in isolation. In reality, these responsibilities belong in a network of organisational processes spanning across product management and requirements engineering. As Kantola et al. observe, difficulties faced by product owners in release planning are ultimately the problems in planning for the organisation [48]. To understand process design, product owner tasks should be viewed as part of broader workflows, not isolated tasks.

The findings from RQ1.1 and RQ1.2 reflect the same, where product owners discuss how the tasks in isolation may not take time, but it is the amalgamation of tasks that is of importance in workflows. For doing a task, they may go back and forth between several tools and documents only to copy and paste content. And while doing it once is not that time-consuming, when it becomes an inherent step of the workflow to be done frequently, it becomes a pain point. However, this does not mean that all tasks in isolation are not time-consuming. For example, product owners note that doing a pre-study to understand the impact of requirements or coming up with a solution that can be incorporated into existing systems takes time due to the complexity of the task.

Kadenic et al. report that as agile projects scale, product owners might feel strain from constantly shifting priorities, pressure to deliver new features, and the learning curve for new project domains [46]. On the surface, writing user stories seems simple, but taken together with handling stakeholder interests, meetings, analysing the impact of the user story on existing systems, and even some design work, it can overwhelm one person. In the findings for this study, this strain turns into frustration due to the introduction of administrative overhead of doing smaller tasks manually, such as adding one's name and requirement IDs to each page of the report. The administrative tasks become time-consuming because they are seldom done, so nobody is an expert in it, and even making a small mistake in uploading a file to the tools can take several hours to fix when in reality it should only take a click or two.

The research term, process debt, describes such cumulative inefficiencies aptly. Gustavsson et al. show that certain forms of process debt, mainly role debt, where responsibilities overlap or are ill-defined, can undermine job satisfaction measurably in agile teams [36]. For example, findings of this study show that product owners maintain artefacts produced during the workflow by keeping track of changes, making updates if requirements change, and reviewing the documents. Likewise, frequent context-switching is common. A product owner might leave one meeting with a

stakeholder to attend another with the development team, then sit down to work on a report, but find themselves answering a customer's question that came in their email. Individually, each interruption seems justified, but cumulatively, they divert attention and focus from a task.

These contradictions point to inefficiencies in the process design. Agile theory favours quick, just-in-time decision making, yet organisations still need careful documentation and alignment [83]. Very often, agility and predictability pull in opposite directions. Such tensions are classic in complex organisational systems where actions like handling changing requirements create ripple effects on schedules, implementation, and expectations.

The findings from initial interviews note that requirement analysis and artefact management are high-value tasks because, if not done right, they can propagate into larger problems. For example, the documentation should be clear because teams rely on it for implementation, and if requirements change, the document should reflect that so that teams do not get misguided. The resulting debt is often only visible in retrospect, for example, burnt-out teams, but it is real. As Kadenic et al. conclude, when product owners become chronically overburdened, it can have a severe effect on their overall performance, unless organisational processes are adjusted [46].

5.3 The Productivity Paradox: When More Tools Mean Less Flow

Supporting the product owner's broad role is a barrage of tools. In practice, teams often adopt several tools such as project management and tracking tools, productivity tools, and communication tools [13]. One case study reports that some common tools used in agile processes are Jira or Trello for project management, Confluence and Microsoft suite for documentation, and Slack or Teams for chat and meetings [23]. On one hand, these tools enable coordination between distributed teams, but on the other, they fragment the process. Each tool introduces context-switch and data duplication. Product owners in this study report something similar, where they have to copy and paste user stories from Jira to reports and vice versa and have to maintain versions of reports in an internal tool, so the report has to be downloaded and uploaded for each use. Every extra login and formatting standard is overhead.

This brings up the question of whether such tool proliferation is necessary, or would a more integrated toolchain would make the process more coherent. The agile manifesto itself values "working software over comprehensive documentation" and "individuals and interactions over processes and tools", suggesting friction from tool sprawl [23]. However, in reality, few integrated platforms handle everything, and teams tend to gravitate towards familiar solutions like Jira. This means organisations have to invest in developing an internal substitute, but many organisations may not take the initiative to do it because this does not generate revenue for them. Since such tools only provide internal value, organisations may not prioritise them as much. The findings of this study also observe the same, where no or little effort has been made for the automation of the workflow previously. The results indicate only three tasks being automated that are not even used frequently because they are isolated

from the process, and their impact is not noticeable.

Moreover, this would mean that product owners become arbiters of these integrations, as seen in this study, which could mean copying data between documents, linking Jira tickets to documents, and answering queries of teams or customers on different apps. Each of these tasks is a wasted effort. If too many tools are in play, teams should ask whether each adds value or just increases administrative overhead. The goal is coherence in the process, not accumulation of technology. For example, results show that reports have to be downloaded and uploaded within an internal tool, edited in Microsoft tools, and shared using OneDrive.

The key question is whether a given tool landscape helps product owners maintain a coherent, streamlined workflow for their process. Otherwise, each additional tool risks injecting new inefficiencies. In the findings, product owners note that there have been times when requirements have become misaligned because a product owner forgot to update the status of one Jira ticket to correct the status while already approving the status of the requirement in the sprint checklist. In the end, agile's promise depends not just on writing code fast, but on designing processes around it to be as agile as the code.

5.4 Automation Dreams and Reality Checks

The current literature presents a nuanced picture. Advances in GenAI have enabled automation of tasks such as document summarisation, report writing, and early drafting of user stories [80], [35]. Pilot studies show that using AI for everyday tasks improves productivity and consistency [40], [69]. Several studies in scoping review claim they surpassed baseline models in completing the task in terms of accuracy and efficiency [35], [26]. While GenAI may perform a task correctly, judgment, negotiation, and creativity that go into software development tasks still require the experience of product owners.

Another important consideration is how applicable existing research is in a continually changing setting. Much of the initial work on automating software engineering tasks predates the most recent AI wave. Studies that were done using models that came out in 2022 to 2023 have been surpassed by newer models. For example, in findings of the scoping review, it can be seen that models used are mainly GPT-3 and GPT-3.5-Turbo [40], [5], [44], [17] with only a few studies using GPT-4 [28], [21] which also has been surpassed by newer model such as DeepSeek, Llama3.1, and Claude 3.7 Sonnet to name some.

The body of automation-focused research contains methodological constraints that limit its results. Many research studies employ limited lab experiments, simplified tasks, or simulated environments [44], [17]. For example, measuring an LLM's ability to rewrite a user story in isolation does not capture the complexities of a real backlog with dependent user stories and changing requirements. Product owners in this study note that a challenging part about their role is dealing with changing requirements as they may not know the complete picture at the beginning of the workflow, and some requirements may come up further along they are in the implementation.

The literature typically evaluates automation in terms of technical feasibility—can

AI write user stories, generate documentation, or classify requirements — but this lens can obscure the fact that the true value of the product owners is not only in producing artifacts, but also in navigating ambiguity, making trade-offs, and making decisions for the product in case of conflicts. Although some of the selected papers cover use cases of GenAI in identifying conflicting requirements or dealing with changing requirements [37], they do not fully capture the possibility of requirements being dependent on other products and teams, as pointed out by product owners in initial interviews.

Moreover, incorporating AI technologies into agile workflows in practice poses difficulties that are rarely discussed in academic settings. Although an LLM may be able to produce content for reports or summarise them, implementing such a solution requires security and privacy assurance because many projects contain confidential information that is not safe to feed into an externally hosted model, as is the case in this study. As product owners mentioned in their perceptions of GenAI use in their workflow, a common blocker for them is that they are unsure what information they can add in prompts and how much of AI-generated content they can use in their reports.

A subtle but significant realisation that comes from reading the literature is that intentionality is more important than feasibility when determining what can be automated. It should make one wonder if the task should be automated just because it can be. Therefore, the tasks selected for automation are based more on their general usefulness in supporting the workflow of product owners rather than automating every single task that is mentioned in the literature. The benefit of finding automation possibilities stems not from the automation itself, but from the opportunity to rethink process purpose. This emphasises a key point that changing processes begins not with automation, but with a critical evaluation of why we do things the way we do.

5.5 Automation Meets Articulation

As noted in the findings of this study, product owners are responsible for documenting requirements, compiling information for the same, and communicating with different teams in the process to make decisions, much of which involves articulation in language rather than code. As seen in results from the scoping review, LLMs and GenAI models can significantly support such articulative work [40]. Agile methodologies famously de-emphasise heavy documentation in favour of working software [7], but documentation is still required: product owners note that user stories must be clearly written, reports must be generated, and meetings should be documented to keep track of decisions and maintain alignment in the process. The paradox then becomes that agile discourages extensive documentation while requiring regular and high-quality written communication.

In this study, GenAI is used to automate three of the most common use cases of documentation-related tasks, text generation, explanation, and language review, that happen in workflows even outside the role of product owners. The results show that GenAI can essentially automate the preliminary drafting and brainstorming tasks that product owners typically perform manually in their workflow. Such automation

is widespread in other industries: marketing teams use AI to plan content, law offices use it to sort papers, and service departments employ chatbots to compile client complaints. Similarly, the prototype developed shows potential to streamline the workflow of product owners to free up their time for more creative tasks, such as problem-solving, that product owners find more interesting and worthwhile.

Importantly, the advantages of GenAI go beyond productivity and formatting. GenAI can ensure semantic consistency across documentation. They can ensure that terminology is aligned across user stories and reports, and that each item follows a similar narrative structure. GenAI is automatically streamlining these patterns in this study, replacing the need for a product owner to copy and paste templates or manually cross-check naming conventions. This is consistent with what Spinuzzi defined as articulation work - the invisible labour that binds different pieces of information into something usable [86].

If documentation is merely generated without curation, it risks becoming performative — polished but disconnected from reality. Teams may fall into the trap of producing meaningless content, drowning in epics produced by AI and meeting notes that no one reads. The same is perceived as a limitation of GenAI in processes where it might make product owners lazy in verifying content before adding it to the report, resulting in huge chunks of AI-generated text in reports. Therefore, it is important to note that GenAI is still a support tool, not a complete solution. The findings from exit interviews and the survey reflect that product owners value streamlining of their workflow more and consider GenAI-based automations as a plus point within it, rather than GenAI being the solution. This underscores that generative AI should be evaluated not just by what it can produce, but by what it helps others understand.

5.6 If Everything's AI, What's Left for Us? A Philosophical Pause

As product owners show a preference for adopting GenAI in their workflow and GenAI becomes more integrated into it, it is worth wondering if every artefact can be generated, what remains uniquely human in the role? The first consideration is human judgment and oversight. Product owners will still have to check AI outputs and make decisions about what to include in the documentation. Several publications emphasise that people must remain in the loop to control and navigate the tool and validate the output [11], [24], [51]. Blind faith is problematic since LLMs are prone to hallucinations, false statements that appear plausible. The product owners mention this concern in exit interviews and reflect on how they plan to verify the information, rephrase it, and add more content to the generated text before adding it to the report.

Closely related is the concern of skill degradation and overdependence on generated content for tasks. Cognitive science research warns that relying too heavily on AI assistants can negatively impact professionals' abilities [58]. They find that AI tools might accelerate skill decay among experts and hinder skill acquisition among learners. Some product owners in the study also have a similar opinion that people might become overreliant on GenAI responses to write their text and become lazy to verify it before adding it to reports, resulting in large chunks of AI-generated text

that might be completely fabricated.

One of the most insidious risks of relying on generative AI is the overconfidence it inspires in its own outputs. Product owners also reflect on this limitation in exit interviews. They note that there is a strong tendency to accept GenAI's recommendations without question because it generates writing that is syntactically fluid, and frequently more polished than what a non-expert writer would come up with, especially if on a time crunch. A user story might "sound right," a stakeholder summary might "read well," but neither may be entirely accurate or contextually aligned with the team's actual direction or needs [63]. Here, automation does not augment judgment — it replaces it. And the result is not just bad documentation, but bad decision-making.

As more non-coding tasks become automated, organisations may unintentionally confuse output with outcome. In other words, the ability to generate polished user stories or elegant meeting notes might be mistaken for product leadership. Yet the essence of product role is not in producing text, but in making decisions: selecting what matters, why it matters, and for whom. The ethical and philosophical weight of this transition lies in distinguishing artefact creation from product visioning. AI can generate hundreds of user stories — but it cannot judge which ones are feasible, how they can be incorporated in an existing system, and the trade-offs. Similarly, while LLMs can simulate empathy in text, they do not experience empathy; they cannot detect hesitations in meetings or mediate competing interests. These functions remain fundamentally human.

Automated generation is only as effective as the prompts and the data GenAI model gets. If the prompt is vague or poorly stated, AI will only produce more blur [42]. Product owners in this study acknowledge that the quality of responses can be improved by using a knowledge base that is more closely connected to the domain and organisation. The findings from the exit interview also note that product owners would like the ability to use their own prompts to generate responses from GenAI models for their reports. Homolak underlines that GenAI might produce hallucinogenic or irrelevant content: "Your answers are only as good as the questions you ask" [43]. Product owners still have to work on using good prompts, reviewing generated suggestions, and maintaining control over the process.

The product owner's attention—focused, moral, and narrative is what ensures that automation has a meaningful purpose rather than drowning in textual abundance. According to Bender et al., the "stochastic parrots" of huge language models do not understand what they produce, but their output can wield disproportionate power if not scrutinised by human actors [12]. As a result, what remains is not less work, but a different kind of work: curatorial rather than clerical, interpretive rather than descriptive, ethical rather than procedural. In doing so, we do not just preserve the human in the loop; we preserve the value of being human in that loop.

5.7 Threats to Validity

The threats to validity for this study that arise from study design, data collection methods, and data analysis techniques are discussed in this chapter, including threats to construct validity, internal validity, external validity, and reliability.

5.7.1 Construct Validity

An inherent threat to construct validity in technical action research is caused by the dual role of the primary researcher as both the researcher and implementor of the study [53]. This can cause a bias in what is emphasised in interviews, how themes are derived, or how automation candidates are selected. To counter this threat, data validation methods are clearly described and followed. The data analysis and findings are reviewed by at least one reviewer for each research question.

A common risk with interviews is recall bias, where participants may not be able to accurately recall information about their workflow or experience with automated tasks since the data will be collected after the events have occurred [70]. The missing information in interviews can mean that the results are not complete and can incomplete conclusions [70]. The initial interviews were complemented with observations as a secondary data collection method, and exit interviews were complemented with a survey.

A potential limitation of the study is that the existing literature in the subarea of automation of non-coding tasks using GenAI may be limited because the field is relatively unexplored. This limitation would affect the ability to conduct a thorough literature review for RQ2 and potentially impact the selection of tasks for automation. Moreover, tasks may not be searchable with generic "non-coding tasks" terms. Therefore, the keywords to drive the search for RQ2 are extracted from observations and initial interview findings to look for literature for specific product owner tasks.

A single-group t-test is used to analyse the Likert-scale survey data, which is ordinal but considered interval by the t-test. This is often debated [38], [71], but it is common practice when Likert-scale data are averaged. The t-test is still considered a reasonable choice because of the small size of the population to analyse the small-sample mean against a neutral benchmark. The results are treated carefully and considered preliminary instead of definitive. The survey findings are complemented with findings of the exit interview, which builds upon the survey questions and is administered after the survey has been answered by the participant.

5.7.2 Internal Validity

Due to the dual role of researcher in technical action research, the participants' behaviour and responses may be unconsciously influenced [53]. The participants might avoid criticising the prototype out of politeness, or may give more enthusiastic responses if they sense expectations. The feedback on the prototype was gathered anonymously in the surveys, and neutral questions were asked in the interviews, emphasising both the potential and limitations instead of focusing only on the positive experiences. The results are presented transparently with coding schemes and examples provided.

The observations may have suffered from observer bias, which leads to participants observing differently if they know they are being observed [49]. Repeated observations are conducted with two different product owners without interrupting them in their daily tasks to counter this threat to a certain level. Another internal threat caused during observations is that the observer may unconsciously focus on

behaviours that confirm pre-existing assumptions, especially since the observations were conducted after the interviews. An observation protocol was created before conducting observations to document the time taken for a task, complemented with short notes to counter this threat.

The survey may suffer from response style bias, which happens when some participants tend to consistently choose the same points on a scale [66]. A participant might always choose “Neutral" on Likert items to avoid taking a strong stance. Participants also tend to agree with statements regardless of content in usability surveys. To counter both threats, the survey was created with balanced scale items “Strongly Disagree", “Disagree", “Neutral", “Strongly Agree", and “Agree", and questions were formulated using reverse-worded items to detect patterns.

5.7.3 External Validity

Technical action research is usually done in a specific context [53], the product owner’s workflow at Ericsson, in this case, which can limit the generalisability of the results. The findings are put in a general context in the Discussion chapter to motivate the generalisability of results. While specific results about non-AI automations and suggestions for further implementation do not hold up in a general context, findings on pain points in fragmented workflows in an organisation and how GenAI can help with articulation tasks would hold up in a general context, even if outside the software development industry.

The workflow automated is specific to product owners at Ericsson, and so are the pain points. Moreover, only five participants participated in the study, which means the findings only reflect pain points and feedback of a subset of the actual population. This may also affect the generalisability of the results. However, it is worth noting that parts of automation and workflow design are generalisable, such as GenAI-based automation for articulation tasks and the prompt-design patterns associated with them. Some other generalisable aspects include the feedback on a streamlined process as a whole and its perceived potential, since any process with fragmented tasks and tools would benefit from being coherent.

As mentioned previously, the population size for the study is relatively small. Therefore, multiple data collection methods are used for answering the same research question, except RQ2, which does not depend on the participants. The initial interviews are complemented with observations, however, it is important to note that some tasks may have happened outside of the observation window and are not included in the findings. The exit interviews are complemented with a usability survey, however, the statistical power to generalise survey results is not sufficient due to the low population size.

5.7.4 Reliability

Technical action research involves iterative cycles [53], however, only the first cycle of iteration is conducted and presented at the time of this study. Due to the iterative nature of technical action research studies, the results can not be replicated. However, some replicable results of this study are the results from the scoping literature review due to its systematic nature and the implementation solution.

Since the product owners only simulate their requirement workflow when using the prototype for exit interviews and surveys, the findings may not reflect their complete experience as they would for using the prototype for an actual delivery of a feature. The participants may not have spent sufficient time with the prototype to solve a task as they would in reality. The findings for exit interviews and surveys are treated as such and not considered as ground-truth.

Chapter 6

Conclusions and Future Work

6.1 Conclusions

In this study, we explore the potential and limitations of GenAI for the automation of non-coding software development tasks of product owners. A technical action research is carried out with a partner company, Ericsson, in its first cycle of iteration. The study identifies the most time-consuming tasks product owners conduct, their expectations from the automated support for tasks, and perceptions of the use of GenAI in their workflow through a series of interviews and observations.

The suitable candidates for automation are identified, and GenAI models suitable for automating them based on existing evidence in scientific literature using a scoping literature review. A prototype is developed to streamline the workflow of product owners at the company, and parts of this workflow are automated using GenAI. The product owners evaluate the prototype in an exit interview and survey. The experiences of product owners with the prototype and GenAI-based automation are collected, focusing on both the potential and limitations of the tool and GenAI.

The study concludes that the most time-consuming non-coding tasks conducted by product owners are compiling information from different resources, documentation, and management of artefacts produced during the workflow. The tasks that take time because of their frequency of occurrence are communication-related. Although requirement analysis is time-consuming, the product owners are unable to dedicate sufficient time to it due to the other time-consuming tasks identified.

The product owners expect that automation of tasks within the workflow will streamline the process and help avoid mis-synchronisation of information across tools, and help save time that can be spent on high-value tasks. The product owners are open to adopting GenAI in their workflow and expect it to help with brainstorming and content generation tasks. They, however, remain sceptical about the quality of generated responses in terms of their usefulness and relevance.

The findings from the scoping literature review conclude that non-coding tasks suitable for automation using GenAI are related to requirement analysis, communication, explainability, and generating modelling diagrams. The GenAI models most commonly used for these tasks are GPT-series and variants of Llama. The tasks can be automated by developing multi-agent systems, pipelines for a series of tasks or by accessing the GenAI models directly using an API.

The findings further indicate that participants preferred to use both the prototype and automation within it. The analysis of each question also reaffirms that they found the prototype and automation useful in their process. There is a positive experience

reported by product owners, and they see potential in the prototype to streamline the software process if developed further, to help them save time on time-consuming tasks. They also have a positive perception of the use of GenAI in their tasks and believe it helps them brainstorm better and document faster. They, however, also show concern regarding the reliability and relevance of generated responses and provide suggestions for future implementation.

In conclusion, the study shows the importance of streamlined and coherent workflows with as less distractions as possible from tool chains to help product owners save time in their non-coding development workflow. The insights from existing evidence show GenAI can be successfully utilised to automate articulation tasks such as requirement analysis and documentation. Lastly, the development and evaluation of a prototype further indicate that streamlined processes reduce time spent on tasks and reduce context-switching, while GenAI-based automations add value to this streamlined process.

6.2 Future Work

The findings of this study provide insights into the importance of improving processes before automating them and the role GenAI can play in it. While this study provides findings and critical reflections on the automation of non-coding software development tasks using GenAI within a software process, it also opens avenues for future research that should go beyond technical feasibility and address the deeper questions of fit and value in real-world software development processes.

The future of automation in product owner tasks is not full autonomy, but effective human-AI collaboration. The goal of research should be to better understand how AI technologies can enhance product owners' workflow rather than replace human judgment. The future work should investigate designing workflows where AI-generated suggestions are transparent and situated within the decision-making context of agile teams. Understanding how product owners adopt, adjust, or reject AI input can help develop confidence in product owners and offer insights into the mechanics of trust.

Quality assurance is a major concern as GenAI systems are integrated more frequently in software development processes. Unlike typical automation systems, which use deterministic logic, GenAI systems produce probabilistic outputs that may appear to be correct but can be misleading, irrelevant, or biased. Future work should look into how to ensure quality assurance of AI-generated responses, the underlying causes for hallucinations, and how to improve the trustworthiness of these systems.

Future work should also move beyond automating isolated tasks and instead focus on rethinking the underlying process structures within which product owners operate. Much of the current research views product owner tasks in isolation without investigating how these tasks interact within larger organisational processes. Future studies should look into how GenAI and automation technologies can streamline workflows, rather than merely the productivity of individual tasks.

References

[1] "Can AI Truly Be Green?" Nov. 2023. [Online]. Available: https://greensoftware.foundation/articles/can-ai-truly-be-green

[2] "EU AI Act: first regulation on artificial intelligence," Jun. 2023. [Online]. Available: https://www.europarl.europa.eu/topics/en/article/20230601STO93804/eu-ai-act-first-regulation-on-artificial-intelligence

[3] "Home - Literature Review - Library Guides at Charles Sturt University," Oct. 2024. [Online]. Available: https://libguides.csu.edu.au/review

[4] "Literature Reviews - Research Methods - Research Guides at Auraria Library (CU Denver, MSU Denver, CCD)," Mar. 2025. [Online]. Available: https://guides.auraria.edu/researchmethods/literaturereviews

[5] S. Abedu, A. Abdellatif, and E. Shihab, "LLM-Based Chatbots for Mining Software Repositories: Challenges and Opportunities," in *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering*, ser. EASE '24. New York, NY, USA: Association for Computing Machinery, Jun. 2024, pp. 201–210. [Online]. Available: https://doi.org/10.1145/3661167.3661218

[6] I. L. Alberts, L. Mercolli, T. Pyka, G. Prenosil, K. Shi, A. Rominger, and A. Afshar-Oromieh, "Large language models (LLM) and ChatGPT: what will the impact on nuclear medicine be?" *European Journal of Nuclear Medicine and Molecular Imaging*, vol. 50, no. 6, pp. 1549–1552, May 2023.

[7] S. Alsaqqa, S. Sawalha, and H. Abdel-Nabi, "Agile Software Development: Methodologies and Trends," *International Journal of Interactive Mobile Technologies (iJIM)*, vol. 14, no. 11, pp. 246–270, Jul. 2020, number: 11. [Online]. Available: https://online-journals.org/index.php/i-jim/article/view/13269

[8] L. Alzubaidi, J. Zhang, A. J. Humaidi, A. Al-Dujaili, Y. Duan, O. Al-Shamma, J. Santamaría, M. A. Fadhel, M. Al-Amidie, and L. Farhan, "Review of deep learning: concepts, CNN architectures, challenges, applications, future directions," *Journal of Big Data*, vol. 8, no. 1, p. 53, Mar. 2021. [Online]. Available: https://doi.org/10.1186/s40537-021-00444-8

[9] J. M. Bass, S. Beecham, M. A. Razzak, C. N. Canna, and J. Noll, "An empirical study of the product owner role in scrum | Proceedings of the 40th International Conference on Software Engineering: Companion Proceeedings," ser. ICSE '18. New York, NY, USA: Association for Computing Machinery, May 2018, pp. 123–124. [Online]. Available: https://doi.org/10.1145/3183440.3195066

[10] T. Bazzan, B. Olojo, P. Majda, T. Kelly, M. Yilmaz, G. Marks, and P. M. Clarke, "Analysing the Role of Generative AI in Software Engineering - Results from an MLR," in *Systems, Software and Services Process Improvement*, Jul. 2024. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-031-71139-8_11

[11] H. Benbya, T. H. Davenport, and S. Pachidi, "Artificial Intelligence in Organizations: Current State and Future Opportunities," Rochester, NY, Dec. 2020. [Online]. Available: https://papers.ssrn.com/abstract=3741983

[12] E. M. Bender, T. Gebru, A. McMillan-Major, and S. Shmitchell, "On the Dangers of Stochastic Parrots: Can Language Models Be Too Big?" in *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency*, ser. FAccT '21. New York, NY, USA: Association for Computing Machinery, Mar. 2021, pp. 610–623. [Online]. Available: https://dl.acm.org/doi/10.1145/3442188.3445922

[13] M. Berntzen, S. A. Engdal, M. Gellein, and N. B. Moe, "Coordination in Agile Product Areas: A Case Study from a Large FinTech Organization," in *Agile Processes in Software Engineering and Extreme Programming*. Springer, Cham, 2024, pp. 36–52, iSSN: 1865-1356. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-031-61154-4_3

[14] M. Berntzen, N. B. Moe, and V. Stray, "The Product Owner in Large-Scale Agile: An Empirical Study Through the Lens of Relational Coordination Theory," in *Agile Processes in Software Engineering and Extreme Programming*. Springer, Cham, 2019, pp. 121–136, iSSN: 1865-1356. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-030-19034-7_8

[15] A. Bhat, "Experimental Research: What it is + Types of designs | QuestionPro." [Online]. Available: https://www.questionpro.com/blog/experimental-research/

[16] C. Biswas and S. Das, "ARIA-QA: AI-agent based requirements inspection and analysis through question answering," *Innovations in Systems and Software Engineering*, pp. 1–16, Oct. 2024, company: Springer Distributor: Springer Institution: Springer Label: Springer Publisher: Springer London. [Online]. Available: https://link.springer.com/article/10.1007/s11334-024-00589-8

[17] F. Bozyigit, T. Bardakci, A. Khalilipour, M. Challenger, G. Ramackers, O. Babur, and M. Chaudron, "Generating domain models from natural language text using NLP: a benchmark dataset and experimental comparison of tools," *Software and Systems Modeling*, vol. 23, no. 6, pp. 1493–1511, 2024. [Online]. Available: https://www.scopus.com/inward/record.uri?eid=2-s2.0-85192837407&doi=10.1007%2fs10270-024-01176-y&partnerID=40&md5=05083072b9bb803d8276a3f8ed35b8de

[18] J. Brooke, "SUS – a quick and dirty usability scale," in *Usability Evaluation in Industry*. Taylor & Francis, Jan. 1996, pp. 189–194. [Online]. Available: https://www.researchgate.net/publication/319394819_SUS_--_a_quick_and_dirty_usability_scale

[19] R. E. Browder, H. Koch, A. Long, and J. M. Hernandez, "Learning to Innovate with Big Data Analytics in Interorganizational Relationships," vol. 8, no. 1, Mar. 2022. [Online]. Available: https://doi.org/10.5465/amd.2019.0048

[20] B. Cabrero-Daniel, "Trustworthy "blackbox" Self-Adaptive Systems."

[21] B. Cabrero-Daniel, Y. Fazelidehkordi, and A. Nouri, "How Can Generative AI Enhance Software Management? Is It Better Done than Perfect?" in *Generative AI for Effective Software Development*, A. Nguyen-Duc, P. Abrahamsson, and F. Khomh, Eds. Cham: Springer Nature Switzerland, 2024, pp. 235–255. [Online]. Available: https://doi.org/10.1007/978-3-031-55642-5_11

[22] B. Cabrero-Daniel, T. Herda, V. Pichler, and M. Eder, "Exploring Human-AI Collaboration in Agile: Customised LLM Meeting Assistants," in *Agile Processes in Software Engineering and Extreme Programming*. Springer, Cham, 2024, pp. 163–178, iSSN: 1865-1356. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-031-61154-4_11

[23] F. Calefato, A. Giove, F. Lanubile, and M. Losavio, "A case study on tool support for collaboration in agile development," in *Proceedings of the 15th International Conference on Global Software Engineering*, ser. ICGSE '20. New York, NY, USA: Association for Computing Machinery, Sep. 2020, pp. 11–21. [Online]. Available: https://doi.org/10.1145/3372787.3390436

[24] M. Coutinho, L. Marques, A. Santos, M. Dahia, C. França, and R. de Souza Santos, "The Role of Generative AI in Software Development Productivity: A Pilot Case Study," in *Proceedings of the 1st ACM International Conference on AI-Powered Software*, ser. AIware 2024. New York, NY, USA: Association for Computing Machinery, Jul. 2024, pp. 131–138. [Online]. Available: https://doi.org/10.1145/3664646.3664773

[25] C. Ebert and P. Louridas, "Generative AI for Software Practitioners," *ResearchGate*, vol. 40, pp. 30–38, Jul. 2023. [Online]. Available: https://www.researchgate.net/publication/372212988_Generative_AI_for_Software_Practitioners

[26] G. Fan, S. Chen, C. Gao, J. Xiao, T. Zhang, and Z. Feng, "RAPID: Zero-Shot Domain Adaptation for Code Search with Pre-Trained Models," *ACM Trans. Softw. Eng. Methodol.*, vol. 33, no. 5, pp. 128:1–128:35, Jun. 2024. [Online]. Available: https://doi.org/10.1145/3641542

[27] I. Fatema and K. Sakib, "Factors Influencing Productivity of Agile Software Development Teamwork: A Qualitative System Dynamics Approach," in *2017 24th Asia-Pacific Software Engineering Conference (APSEC)*, Dec. 2017, pp. 737–742. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/8306011

[28] M. Fazelnia, V. Koscinski, S. Herzog, and M. Mirakhorli, "Lessons from the Use of Natural Language Inference (NLI) in Requirements Engineering Tasks," Apr. 2024, arXiv:2405.05135 [cs]. [Online]. Available: http://arxiv.org/abs/2405.05135

[29] M. Fazelnia, M. Mirakhorli, and H. Bagheri, "Translation Titans, Reasoning Challenges: Satisfiability-Aided Language Models for Detecting Conflicting Requirements," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, ser. ASE '24. New York, NY, USA: Association for Computing Machinery, Oct. 2024, pp. 2294–2298. [Online]. Available: https://doi.org/10.1145/3691620.3695302

[30] F. A. Fernandes and C. M. L. Werner, "A Scoping Review of the Metaverse for Software Engineering Education: Overview, Challenges, and Opportunities," *PRESENCE: Virtual and Augmented Reality*, vol. 31, pp. 107–146, Dec. 2022. [Online]. Available: https://doi.org/10.1162/pres_a_00371

[31] N. Forsgren, M.-A. Storey, C. Maddila, T. Zimmermann, B. Houck, and J. Butler, "The SPACE of Developer Productivity: There's more to it than you think." *Queue*, vol. 19, no. 1, pp. Pages 10:20–Pages 10:48, Mar. 2021. [Online]. Available: https://dl.acm.org/doi/10.1145/3454122.3454124

[32] F. Foukalas, "A Survey of Artificial Neural Network Computing Systems," *Cognitive Computation*, vol. 17, no. 1, pp. 1–20, Feb. 2025, company: Springer Distributor: Springer Institution: Springer Label: Springer Number: 1 Publisher: Springer US. [Online]. Available: https://link.springer.com/article/10.1007/s12559-024-10383-0

[33] T. George, "Types of Interviews in Research | Guide & Examples," Mar. 2022. [Online]. Available: https://www.scribbr.com/methodology/interviews-research/

[34] Gittell, Jody, "Relational coordination: Coordinating work through relationships of shared goals, shared knowledge and mutual respect," *ResearchGate*, Jan. 2006. [Online]. Available: https://www.researchgate.net/publication/286629377_Relational_coordination_Coordinating_work_through_relationships_of_shared_goals_shared_knowledge_and_mutual_respect

[35] L. Golia and J. Kalita, "Action-Item-Driven Summarization of Long Meeting Transcripts," in *Proceedings of the 2023 7th International Conference on Natural Language Processing and Information Retrieval*, ser. NLPIR '23. New York, NY, USA: Association for Computing Machinery, Mar. 2024, pp. 91–98. [Online]. Available: https://dl.acm.org/doi/10.1145/3639233.3639253

[36] T. Gustavsson, M. O. Ahmad, and H. Saeeda, "Job satisfaction at risk: Measuring the role of process debt in agile software development," *Journal of Systems and Software*, vol. 222, p. 112350, Apr. 2025. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0164121225000184

[37] A. E. Gärtner and D. Göhlich, "Automated requirement contradiction detection through formal logic and LLMs," *Automated Software Engg.*, vol. 31, no. 2, Jun. 2024. [Online]. Available: https://doi.org/10.1007/s10515-024-00452-x

[38] S. E. Harpe, "How to analyze Likert and other rating scale data," *Currents in Pharmacy Teaching and Learning*, vol. 7, no. 6, pp. 836–850, Nov. 2015. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1877129715200196

[39] A. Henriksen and S. A. R. Pedersen, "A qualitative case study on agile practices and project success in agile software projects," May 2017, accepted: 2017-06-08T10:35:28Z Publisher: Mundo Press. [Online]. Available: https://munin.uit.no/handle/10037/11124

[40] J. L. Heyman, S. R. Rick, G. Giacomelli, H. Wen, R. Laubacher, N. Taubenslag, M. Knicker, Y. Jeddi, P. Ragupathy, J. Curhan, and T. Malone, "Supermind Ideator: How Scaffolding Human-AI Collaboration Can Increase Creativity," in *Proceedings of the ACM Collective Intelligence Conference*, ser. CI '24. New York, NY, USA: Association for Computing Machinery, Jul. 2024, pp. 18–28. [Online]. Available: https://dl.acm.org/doi/10.1145/3643562.3672611

[41] J. Holmström and N. Carroll, "How organizations can innovate with generative AI," *Business Horizons*, Feb. 2024. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0007681324000247

[42] D. Homolak, ""Garbage In – Garbage Out!" Still Applies with Gen AI - Integreon," Sep. 2024. [Online]. Available: https://www.integreon.com/garbage-in-garbage-out-still-applies-with-gen-ai/

[43] J. Homolak, "Opportunities and risks of ChatGPT in medicine, science, and academic publishing: a modern Promethean dilemma," *Croatian Medical Journal*, vol. 64, no. 1, pp. 1–3, Feb. 2023. [Online]. Available: https://www.ncbi.nlm.nih.gov/pmc/articles/PMC10028563/

[44] M. Jahan, M. M. Hassan, R. Golpayegani, G. Ranjbaran, C. Roy, B. Roy, and K. Schneider, "Automated Derivation of UML Sequence Diagrams from User Stories: Unleashing the Power of Generative AI vs. a Rule-Based Approach," in *Proceedings of the ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems*, ser. MODELS '24. New York, NY, USA: Association for Computing Machinery, Sep. 2024, pp. 138–148. [Online]. Available: https://doi.org/10.1145/3640310.3674081

[45] M. Jonsson and J. Tholander, "Cracking the code: Co-coding with AI in creative programming education," in *Proceedings of the 14th Conference on Creativity and Cognition*, ser. C&C '22. New York, NY, USA: Association for Computing Machinery, Jun. 2022, pp. 5–14. [Online]. Available: https://dl.acm.org/doi/10.1145/3527927.3532801

[46] M. D. Kadenic, D. A. de Jesus Pacheco, K. Koumaditis, G. Tjørnehøj, and T. Tambo, "Investigating the role of Product Owner in Scrum teams: Differentiation between organisational and individual impacts and opportunities," *Journal of Systems and Software*, vol. 206, p. 111841, Dec. 2023. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0164121223002364

[47] F. Kalota, "A Primer on Generative Artificial Intelligence," *Education Sciences*, vol. 14, no. 2, p. 172, Feb. 2024, number: 2 Publisher: Multidisciplinary Digital Publishing Institute. [Online]. Available: https://www.mdpi.com/2227-7102/14/2/172

[48] K. Kantola, J. Vanhanen, and J. Tolvanen, "Mind the product owner: An action research project into agile release planning," *Information and Software Technology*, vol. 147, p. 106900, Jul. 2022. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0950584922000611

[49] B. B. Kawulich, "Participant Observation as a Data Collection Method," *Forum Qualitative Sozialforschung / Forum: Qualitative Social Research*, vol. 6, no. 2, May 2005, number: 2. [Online]. Available: https://www.qualitative-research.net/index.php/fqs/article/view/466

[50] F. Kibuacha, "Survey Response Rates: A Guide to Higher Survey Completions," Feb. 2024. [Online]. Available: https://www.geopoll.com/blog/low-survey-response-rates/

[51] D.-K. Kim, "Comparing Proficiency of ChatGPT and Bard in Software Development," in *Generative AI for Effective Software Development*. Springer Nature Switzerland, Jan. 2024, pp. 25–51. [Online]. Available: https://doi.org/10.1007/978-3-031-55642-5_2

[52] K. B. Kiran, M. Marumudi, A. V. Mudunuri, S. Kondaveeti, R. Kondapalli, K. I. Pradeep, and B. C. S. N. L. S. Sai Baba, "Multi-model Summarization on Extending T5 Transformer for Text, Audio, and Images," in *Cognitive Computing and Cyber Physical Systems*. Springer, Cham, 2025, pp. 52–64, iSSN: 1867-822X. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-031-77081-4_6

[53] N. Kock, "The three threats of action research: a discussion of methodological antidotes in the context of an information systems study," *Decision Support Systems*, vol. 37, no. 2, pp. 265–286, May 2004. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0167923603000228

[54] D. Kumlander, "Product Owner Responsibilities in the Project Assurance Process: Bridging Uncertainties Gaps," in *Innovations and Advances in Computing, Informatics, Systems Sciences, Networking and Engineering*, T. Sobh and K. Elleithy, Eds. Cham: Springer International Publishing, 2015, pp. 515–519.

[55] L. Law, "Application of generative artificial intelligence (GenAI) in language teaching and learning: A scoping literature review," *Computers and Education Open*, vol. 6, p. 100174, Jun. 2024. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2666557324000156

[56] J. Li, Y. Zhang, Z. Karas, C. McMillan, K. Leach, and Y. Huang, "Do Machines and Humans Focus on Similar Code? Exploring Explainability of Large Language Models in Code Summarization." IEEE Computer Society, Apr. 2024, pp. 47–51. [Online]. Available: https://www.computer.org/csdl/proceedings-article/icpc/2024/058600a047/1XSnQCpIdws

[57] Y. Li, J. Keung, Z. Yang, X. Ma, J. Zhang, and S. Liu, "SimAC: simulating agile collaboration to generate acceptance criteria in user story elaboration," *Automated Software Engg.*, vol. 31, no. 2, Jun. 2024. [Online]. Available: https://doi.org/10.1007/s10515-024-00448-7

[58] B. N. Macnamara, I. Berber, M. C. Çavuşoğlu, E. A. Krupinski, N. Nallapareddy, N. E. Nelson, P. J. Smith, A. L. Wilson-Delfosse, and S. Ray, "Does using artificial intelligence assistance accelerate skill decay and hinder skill development without performers' awareness?" *Cognitive Research: Principles and Implications*, vol. 9, p. 46, Jul. 2024. [Online]. Available: https://www.ncbi.nlm.nih.gov/pmc/articles/PMC11239631/

[59] E. F. Magnussen, E. Havnelid, and J. S. Molléri, "The Product Owner and its Impact on Success and Challenges in Agile Scrum Projects: An Interview Study," *Procedia Computer Science*, vol. 239, pp. 1157–1164, Jan. 2024. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1877050924015254

[60] A. Mastropaolo, M. Ciniselli, L. Pascarella, R. Tufano, E. Aghajani, and G. Bavota, "Towards Summarizing Code Snippets Using Pre-Trained Transformers," in *Proceedings of the 32nd IEEE/ACM International Conference on Program Comprehension*, ser. ICPC '24. New York, NY, USA: Association for Computing Machinery, Jun. 2024, pp. 1–12. [Online]. Available: https://dl.acm.org/doi/10.1145/3643916.3644400

[61] T. Mattis, E. Krebs, M. C. Rinard, and R. Hirschfeld, "Examples out of Thin Air: AI-Generated Dynamic Context to Assist Program Comprehension by Example," in *Companion Proceedings of the 8th International Conference on the Art, Science, and Engineering of Programming*, ser. Programming '24. New York, NY, USA: Association for Computing Machinery, Jul. 2024, pp. 99–107. [Online]. Available: https://dl.acm.org/doi/10.1145/3660829.3660845

[62] G. Matturro, F. Cordovés, and M. Solari, "The role of Product Owner from the practitioner's perspective. An exploratory study."

[63] J. Maynez, S. Narayan, B. Bohnet, and R. McDonald, "On Faithfulness and Factuality in Abstractive Summarization," May 2020, arXiv:2005.00661 [cs]. [Online]. Available: http://arxiv.org/abs/2005.00661

[64] C. Melo, D. S. Cruzes, F. Kon, and R. Conradi, "Agile Team Perceptions of Productivity Factors," in *2011 Agile Conference*, Aug. 2011, pp. 57–66. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/6005486

[65] A. N. Meyer, E. T. Barr, C. Bird, and T. Zimmermann, "Today Was a Good Day: The Daily Life of Software Developers," *IEEE Transactions on Software Engineering*, vol. 47, no. 5, pp. 863–880, May 2021. [Online]. Available: https://ieeexplore.ieee.org/document/8666786

[66] G. J. Miller, *Handbook of Research Methods in Public Administration, Second Edition*. CRC Press, Oct. 1998, google-Books-ID: awIEIBZ0fPUC.

[67] N. B. Moe, V. Stray, D. Šmite, and M. Mikalsen, "Attractive Workplaces: What Are Engineers Looking for?" *IEEE Software*, vol. 40, no. 5, pp. 85–93, Sep. 2023. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/10128875

[68] Z. Munn, M. D. J. Peters, C. Stern, C. Tufanaru, A. McArthur, and E. Aromataris, "Systematic review or scoping review? Guidance for authors

when choosing between a systematic or scoping review approach," *BMC Medical Research Methodology*, vol. 18, no. 1, p. 143, Nov. 2018. [Online]. Available: https://doi.org/10.1186/s12874-018-0611-x

[69] L. Naimi, E. M. Bouziane, A. Jakimi, R. Saadane, and A. Chehri, "Automating Software Documentation: Employing LLMs for Precise Use Case Description," *Procedia Computer Science*, vol. 246, pp. 1346–1354, Jan. 2024. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1877050924026176

[70] K. Nikolopoulou, "What Is Recall Bias? | Definition & Examples," Oct. 2022. [Online]. Available: https://www.scribbr.com/research-bias/recall-bias/

[71] G. Norman, "Likert scales, levels of measurement and the "laws" of statistics," *Advances in Health Sciences Education*, vol. 15, no. 5, pp. 625–632, Dec. 2010, company: Springer Distributor: Springer Institution: Springer Label: Springer Number: 5 Publisher: Springer Netherlands. [Online]. Available: https://link.springer.com/article/10.1007/s10459-010-9222-y

[72] Odeh, Ayman, Odeh, Nada, and Mohammed, Abdul Salam, "A Comparative Review of AI Techniques for Automated Code Generation in Software Development: Advancements, Challenges, and Future Directions," *TEM Journal*, vol. 13, no. 1, pp. 726–739, 2024, publisher: UIKTEN - Association for Information Communication Technology Education and Science. [Online]. Available: https://www.ceeol.com/search/article-detail?id=1223185

[73] A. Ott, "AI-generated content in transition – between progress and fatigue," Ph.D. dissertation, Feb. 2025. [Online]. Available: https://www.ey.com/en_ch/insights/ai/ai-generated-content-challenges-and-opportunities

[74] G. E. Park, H. Kim, and U. R. Go, "Generative artificial intelligence in nursing: A scoping review," *Collegian*, vol. 31, no. 6, pp. 428–436, Dec. 2024. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1322769624000696

[75] M. S. Raunak and D. Binkley, "Agile and other trends in software engineering," in *2017 IEEE 28th Annual Software Technology Conference (STC)*, Sep. 2017, pp. 1–7. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/8234457

[76] Robson C., *Real World Research.* Malden MA: Blackwell Publishing, 2002.

[77] K. Ronanki, B. Cabrero-Daniel, and C. Berger, "ChatGPT as a Tool for User Story Quality Evaluation: Trustworthy Out of the Box?" in *Agile Processes in Software Engineering and Extreme Programming – Workshops.* Springer, Cham, 2024, pp. 173–181, iSSN: 1865-1356. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-031-48550-3_17

[78] K. Ronanki, B. Cabrero-Daniel, J. Horkoff, and C. Berger, "Requirements Engineering Using Generative AI: Prompts and Prompting Patterns | SpringerLink," in *Generative AI for Effective Software Development.* Springer Nature Switzerland, Jan. 2024, pp. 109–127. [Online]. Available: https://doi.org/10.1007/978-3-031-55642-5_5

[79] K. S. Rubin, *Essential Scrum: A Practical Guide to the Most Popular Agile Process*, 1st ed. Addison-Wesley Professional, Jun. 2012.

[80] M. A. Sami, M. Waseem, Z. Zhang, Z. Rasheed, K. Systä, and P. Abrahamsson, "Early Results of an AI Multiagent System for Requirements Elicitation and Analysis," in *Product-Focused Software Process Improvement*. Springer, Cham, 2025, pp. 307–316, iSSN: 1611-3349. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-031-78386-9_20

[81] Sanjiv Jaggia, Alison Kelly, Kevin Lertwachara, and Leida Chen, *Business Analytics*. [Online]. Available: https://www.mheducation.com/highered/product/Business-Analytics-Jaggia.html

[82] S. Schulhoff, M. Ilie, N. Balepur, K. Kahadze, A. Liu, C. Si, Y. Li, A. Gupta, H. Han, S. Schulhoff, P. S. Dulepet, S. Vidyadhara, D. Ki, S. Agrawal, C. Pham, G. Kroiz, F. Li, H. Tao, A. Srivastava, H. D. Costa, S. Gupta, M. L. Rogers, I. Goncearenco, G. Sarli, I. Galynker, D. Peskoff, M. Carpuat, J. White, S. Anadkat, A. Hoyle, and P. Resnik, "The Prompt Report: A Systematic Survey of Prompt Engineering Techniques," Feb. 2025, arXiv:2406.06608 [cs]. [Online]. Available: http://arxiv.org/abs/2406.06608

[83] K. Schwaber, *Agile Project Management with Scrum*. Microsoft Press, Feb. 2004, google-Books-ID: 6pZCAwAAQBAJ.

[84] N. Sekitoleko, F. Evbota, E. Knauss, A. Sandberg, M. Chaudron, and H. H. Olsson, "Technical Dependency Challenges in Large-Scale Agile Software Development," in *Agile Processes in Software Engineering and Extreme Programming*. Springer, Cham, 2014, pp. 46–61, iSSN: 1865-1356. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-319-06862-6_4

[85] N. Sherje, "Enhancing Software Development Efficiency through AI-Powered Code Generation," *Research Journal of Computer Systems and Engineering*, vol. 5, no. 1, pp. 01–12, Jul. 2024, number: 1. [Online]. Available: https://technicaljournals.org/RJCSE/index.php/journal/article/view/90

[86] C. Spinuzzi, D. Altounian, G. Pogue, R. Cochran, and L. Zhu, "Articulating Problems and Markets: A Translation Analysis of Entrepreneurs' Emergent Value Propositions," *Written Communication*, vol. 35, no. 4, pp. 379–410, Oct. 2018, publisher: SAGE Publications Inc. [Online]. Available: https://doi.org/10.1177/0741088318786235

[87] H. S. Sverrisdottir, H. T. Ingason, and H. I. Jonasson, "The Role of the Product Owner in Scrum-comparison between Theory and Practices," *Procedia - Social and Behavioral Sciences*, vol. 119, pp. 257–267, Mar. 2014. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1877042814021211

[88] A. Tkalich, R. Ulfsnes, and N. B. Moe, "Toward an Agile Product Management: What Do Product Managers Do in Agile Companies?" in *Agile Processes in Software Engineering and Extreme Programming*. Springer, Cham, 2022, pp. 168–184, iSSN: 1865-1356. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-031-08169-9_11

[89] R. Ulfsnes, N. B. Moe, V. Stray, and M. Skarpen, "Transforming Software Development with Generative AI: Empirical Insights on Collaboration and Workflow | SpringerLink," Jan. 2024. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-031-55642-5_10

[90] C. Unger-Windeler, J. A.-C. Klünder, T. Reuscher, and K. Schneider, "Are Product Owners communicators? A multi-method research approach to provide a more comprehensive picture of Product Owners in practice," vol. 33, no. 1, Aug. 2020. [Online]. Available: https://doi.org/10.1002/smr.2311

[91] A. T. van Can and F. Dalpiaz, "Locating requirements in backlog items: Content analysis and experiments with large language models," *Information and Software Technology*, vol. 179, p. 107644, Mar. 2025. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0950584924002490

[92] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is All you Need," in *Advances in Neural Information Processing Systems*, vol. 30. Curran Associates, Inc., 2017. [Online]. Available: https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html

[93] A. Vijayan, "A Prompt Engineering Approach for Structured Data Extraction from Unstructured Text Using Conversational LLMs," in *Proceedings of the 2023 6th International Conference on Algorithms, Computing and Artificial Intelligence*, ser. ACAI '23. New York, NY, USA: Association for Computing Machinery, Feb. 2024, pp. 183–189. [Online]. Available: https://doi.org/10.1145/3639631.3639663

[94] B. Wei, "Requirements are All You Need: From Requirements to Code with LLMs," in *ResearchGate*, Mar. 2025. [Online]. Available: https://www.researchgate.net/publication/383285794_Requirements_are_All_You_Need_From_Requirements_to_Code_with_LLMs

[95] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. V. Le, and D. Zhou, "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," *Advances in Neural Information Processing Systems*, vol. 35, pp. 24 824–24 837, Dec. 2022. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2022/hash/9d5609613524ecf4f15af0f7b31abca4-Abstract-Conference.html?ref=https://githubhelp.com

[96] J. White, Q. Fu, S. Hays, M. Sandborn, C. Olea, H. Gilbert, A. Elnashar, J. Spencer-Smith, and D. C. Schmidt, "A Prompt Pattern Catalog to Enhance Prompt Engineering with ChatGPT," Feb. 2023, arXiv:2302.11382 [cs]. [Online]. Available: http://arxiv.org/abs/2302.11382

[97] R. J. Wieringa, *Design Science Methodology for Information Systems and Software Engineering*. Berlin, Heidelberg: Springer, 2014. [Online]. Available: https://link.springer.com/10.1007/978-3-662-43839-8

[98] ——, "Technical Action Research," in *Design Science Methodology for Infor-*

mation Systems and Software Engineering. Springer Berlin Heidelberg, Aug. 2014, pp. 269–293.

[99] B. Wolford, "What is GDPR, the EU's new data protection law?" Nov. 2018, section: GDPR Overview. [Online]. Available: https://gdpr.eu/what-is-gdpr/

[100] S. Zhang, Z. Xing, R. Guo, F. Xu, L. Chen, Z. Zhang, X. Zhang, Z. Feng, and Z. Zhuang, "Empowering Agile-Based Generative Software Development through Human-AI Teamwork," *ACM Trans. Softw. Eng. Methodol.*, Jan. 2025, just Accepted. [Online]. Available: https://dl.acm.org/doi/10.1145/3702987

[101] Z. Zhang, M. Rayhan, T. Herda, M. Goisauf, and P. Abrahamsson, "LLM-Based Agents for Automating the Enhancement of User Story Quality: An Early Report," in *Agile Processes in Software Engineering and Extreme Programming.* Springer, Cham, 2024, pp. 117–126, iSSN: 1865-1356. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-031-61154-4_8

Appendix A

Coding Scheme for Initial Interviews

Codes	Description	Examples
Requirement initiators	People who give the initial requirement for a product (n=5)	"...our technical managers, they gather information or inputs from the customers...""
Steps for starting the process	Initial steps in the workflow before any documents are created (n=5)	"...try to determine the cost or how many man hours of work would be required and which areas are going to be impacted in the product."
Adding content to documents	Steps related to creating reports in the workflow (n=5)	"...get the template for the report and then start filling in, copy and paste the requirement into the document and so on... There is some background text..."
Review of documents	Steps tied to review of reports and how they happen in the workflow (n=5)	"...we do a peer review. And in that, we, of course, provide feedback and get feedback from other product owners or other teams..."
Steps after approval of documents	Steps in the workflow after approval of reports (n=5)	"...and then we wait until the department decides which team should start to develop this based on the report"
Steps during implementation of requirement	Workflow steps during implementation of a requirement (n=5)	"We have the study phase... when LSV or testing specialised team is done with all the verification and then we are ready for market."
Documents produced in the process	Types of reports and artefacts created or written during the workflow (n=5)	"...sprint checklists...", "...system documentation..."
Documents maintained in the process	Reports and artefacts updated and revised in the workflow (n=5)	"...to maintaining, our external interface specifications or APIs..."
Involvement of other teams	External teams involved in the workflow either from a different domain or product (n=5)	"We sometimes have to discuss with the development teams..."
Involvement of other roles	Roles product owners interact with both inside and outside the team (n=5)	"...or if line manager, okay, what's missing in this..."
Stages of involvement	The stage or time at which other teams and roles get involved (n=5)	"Once the report is approved, then we can have this meeting with the design team."

Tools used in the process	Ericsson's internal and external tools used by product owners in the workflow (n=5)	"So pretty much we have the office package there... and besides that, we work a lot with Jira."
GenAI tools used by product owners	Internal GenAI tools used by product owners in the workflow (n=5)	"...which is basically uh kind of an AI tool which is aggregating information... and then you can just have a chat and ask questions to it"
How is the tool used in the process	Current usage of tool in the workflow and how product owners utilise it (n=5)	"...make proper sequence diagrams or UML diagrams with the help of PlantUML..."
Positive comments about existing tools	Positive comments made about existing support for tasks (n=5)	"I mean, it's not like we get stuck. We, of course, figure out how to get things done"
Negative comments about existing tools	Negative comments made about existing support for tasks (n=5)	"...deriving sequence diagrams from logs... it is also admin stuff you need to look, copy and paste things between systems."
Previous efforts for automation	description and reasoning for previous efforts for automation (n=5)	"It's very little automation when it comes to these things"
Requirement tasks	Tasks mentioned and described that are related to requirement analysis and management (n=5)	"...analyse the requirement, you break it down, use cases, you see what needs to be done, what the impact is in the product."
Meetings	Tasks mentioned and described that are related to meetings (n=5)	"...and sometimes it could be more than one product, so we need to have like multi-product combined meetings."
Reading and compiling information	Tasks mentioned and described that are related to reading and compiling information from different sources (n=5)	"...what did we do 10 years ago, what was the decision then, that maybe we still need to relate to data model and things like that..."
Documentation and management	Tasks mentioned and described that are related to documenting requirements and management of documents (n=5)	"...and then making sure that those are related, connected with the main requirement in other tools"
Answering customer questions	Tasks mentioned and described that are related to answering customer questions (n=5)	"...customer units basically can send in some quick questions to clarify..."
Handling escalations	Tasks mentioned and described that are related to handling sudden escalations in requirement delivery (n=5)	"...not exactly that happens every time, but it could happen that there's some escalation so we have to put everything on hold and focus on that."
Time-consuming tasks	Tasks that take time to complete and drain energy (n=5)	"...I can say that it's more than two weeks now. We are just trying to get more information to make sense of what really is required to be done."

Repetitive tasks	Tasks that happen frequently during the workflow (n=5)	"...just revisions due to editorial updates, it's quite common that you receive such review requests, but they are not very time-consuming."
High value tasks	Tasks that are important to do correctly because they have an impact on later steps in the workflow (n=5)	"...misinterpretation of a requirement..."
Administrative tasks	Tasks that are common duties and responsibilities of all product owners (n=5)	'The administrative tasks are creating the Jira tickets, Jira epics and the stories, and so on."
Boring tasks	Tasks that product owners are not interested in doing themselves (n=5)	'That would be those copy-paste things. I think that's sort of... we don't need the brain for it."
Time estimation for time-consuming tasks	Time estimated by product owners for time-consuming tasks (n=5)	"...depending on the size of the requirement, but I would say a minimum of three days."
Frequency of occurrence of repetitive tasks	Frequency of tasks estimates by product owners for repetitive tasks (n=5)	"...every second day I would say."
Size estimate of documents	Estimates made by product owners for size of documents and reports (n=5)	"20-30 pages is not uncommon."
Pain points in workflow	The frustrations and challenges faced by product owners in their workflow (n=5)	"...the lists, the repos, they can see which one's more active... and there are so many areas it takes billions of time to click them off."
Causes of frustration	The causes identified for product owners for frustrations (n=5)	"...this ping pong and trying to make them agree on this is the issue we should, how we should address it and how the solution should be."
Causes of context-switching	The causes identified for context-switching during a task (n=5)	"...the very important part is that not all questions are about the same thing, so it's a lot of context switching because you might be working on something,g the question is totally something else"
Positive comments about GenAI	Positive comments made about current use of GenAI in the workflow as product owners (n=5)	"...it gives at least some hints of which document you can find the next information in rather than me going in and searching, so it saves time."
Negative comments about GenAI	Positive comments made about current use of GenAI as product owners (n=5)	"...it's not really efficient and it's really poor in context, I would say, because it doesn't remember context."
Expectations of quality criteria	The quality criteria expected of AI-generated responses (n=5)	"...correctness would really help because then you are on the right track, of course."

Automation suggestions	Suggestions made by product owners for automation of their workflow (n=5)	"...things going to correct status, minutes of meeting being sent straight away after meeting..."
Suggestions for GenAI automation	Suggestions made by product owners for automation of their workflow using GenAI (n=5)	"I think generative AI that can help us with the phrasing, as I said, I think that would be very helpful. So it could be more, more prompt oriented."
Factors for choosing automation candidates	Factors and motivation for choosing automation candidates (n=5)	"...things that I don't need to think about, okay, now I don't need to think, did it pass, did I send the email, did I not send the email, it's just like automated, it just works..."
Expectations of product owners	Expectations of product owners from automation of tasks and GenAI-based automations (n=5)	"We are of course limited with how we think and what type of background, what type of experience we have, and so on but if it's done by AI it will actually consider a lot of factors there, and it will provide some output that is maybe you didn't even think about those things so that is kind of a lot of value I would say because that gives you a good kickstart to continue with the next item in the document."
Impact of automation	Perceived impact of automated support of tasks on product owner workflow (n=5)	"...if you want to prioritise some courses that you might have, and some kind of self-development."

Table A.1: Coding Scheme, Description, and Examples Identified in Thematic Analysis of Initial Interviews

Appendix B

Results from Scoping Review

Paper	Summary	Automation Details	Implementation
Sami et al. (2025) [80]	The study explores GPT models 3.5 and 4o to generate and prioritise user stories from an initial project description. A multi-agent system is developed to automate the process of user story generation, quality assurance and prioritisation. The results show that GPT-3.5 achieved a higher semantic similarity score than GPT-4o, but they both had consistent performance in prioritisation of requirements.	Product Owner (PO) agent is responsible for generating user stories from a given project description. These user stories are then sent to the Quality Assurance (QA) agent to check and enhance the quality of user stories using the INVEST framework. The Developer (Dev) agent, manager agent, PO and QA agents prioritise and rank user stories using the 100-dollar method.	Implemented as a web-based application using HTML, CSS, JavaScript, React and WebSocket on the frontend, with Python and the Starlette framework on the backend. The frontend communicates with the backend via API calls to OpenAI's GPT-3.5 and GPT-4.0 models.
Naimi et al. (2024) [69]	The paper presents a novel system that extracts use cases from UML use case diagrams and utilises a GenAI model to generate descriptive text for each extracted use case. The results suggest that manual labour and time required for the task can be substantially decreased while maintaining clarity and comprehensiveness in documentation.	An XML file is fed as input, which is then parsed to extract the use case diagram from the XML file. A prompt is designed to send to the GenAI model via an API request. The generated content is received and stored in an output file.	Implemented as a web-based application where users upload an XML file, choose a GenAI model, enter an API key and a custom prompt. The function calls are made using PHP.
Ronanki et al. (2024) [77]	The study explores using ChatGPT for user story quality evaluation and compares its performance with an existing benchmark. The results show that ChatGPT's evaluation aligns well with human evaluation, and a "best of three" strategy is proposed to improve output stability.	ChatGPT is presented with a set of criteria and user story pairs, and responses were recorded. A series of tests using the one-shot prompting method is done.	Prompting directly in ChatGPT interface.
Golia et al. (2024) [35]	The paper presents a novel approach to automating the generation of meeting summaries that are driven by action items identified in meeting transcripts. The results show that the pipeline achieved a BERTscore of 64.98, which is a 4.98% increase from the current state-of-the-art BART model.	First, an algorithm is used to break long meeting transcripts into chunks using topic segmentation techniques. Next, a general abstractive summary of each chunk is generated by fine-tuning Meta's BART model. Then, a BERT model is fine-tuned to classify action items in the original meeting transcript. Another algorithm is used to append the action items with context from a chunk to the end of the general summary of the same chunk to keep summaries and action items for a chunk together. This entire text is then passed to the previous BART model to condense the summary. Finally, the summary and action items of each chunk are summarised using the BART model for a summary of the entire meeting transcript and action items identified in transcripts.	The automation is implemented in Python in a Jupyter notebook. BERTscore metric is used for the evaluation of the automation. The dataset used for training and testing of models was the AMI dataset, which is a large meeting corpus consisting of 137 scenario-driven artificial meetings and their corresponding summaries.

Vijayan (2024) [93]	The paper compares different prompting techniques to extract structured information about trips from unstructured data in email chains. ChatGPT is considered the baseline, and metrics for BARD are compared against it. The results find that BARD overall performed better than ChatGPT in terms of recall, precision, and F1-scores.	Email chains are fed as input to LLMs along with using different prompt techniques - standard prompting, iterative refinement prompting, chain of thought prompting and persona pattern prompting to extract structured data from emails. The structured information is then stored in an SQL database and queried using SQL commands through an interface.	Prompting directly in ChatGPT and Bard interface.
Fan et al. (2024) [26]	The paper proposes a framework zeRo-shot domAin adaPtion with pre-traIned moDels (RAPID) to search for code in projects. The results from experiments show that RAPID outperforms CoCOSoDa and UniXcoder models by an average of 15.7% and 10% respectively, as measured by the MRR metric. In addition, RAPID is effective in code search in three real-world GitHub projects.	First, a query for code is generated using a generative model and nucleus sampling is used to ensure quality and diversity. The generated query is then employed to search similar code snippets using a negative search strategy. After filtering samples that are identical to positive samples, a final set of negative samples is found. Finally, the acquired positive and corresponding negative samples are used as training data in each batch to train the baseline models.	The implementation is in Python 3.8 with PyTorch 1.12.1, and the Huggingface Transformers library 4.27.1. The models are built on top of PLMs available in Huggingface's library.
Fazelnia et al. (2024) [28]	The paper introduces a novel framework, SAT-LLM, which integrates Satisfiability Modulo Theorems (SMT) solvers with LLMs to enhance the detection of conflicting software requirements. The early experiments provide empirical evidence that SAT-LLM is more effective than pure LLM-based methods like ChatGPT in identifying and resolving conflicting requirements with a higher recall, precision, and F1-scores.	The model initially receives a set of requirements. ChatGPT is then used to translate each natural language requirement to first-order logic using a formalisation protocol developed by researchers. The generated first-order logic expressions are then submitted to the Z3 SMT solver, which analyses if expressions are collectively satisfiable or if there are any logical inconsistencies. If there are inconsistencies identified, then a minimal unsatisfiable subset of requirements is communicated to the development team.	An SAT-LLM model is developed where the user enters a set of requirements that are passed through ChatGPT and a formalisation protocol to generate first-order logic expressions. The first-order logic expressions are then sent to Z3, which identifies conflicting (unsatisfiable) and non-conflicting (satisfiable) requirements.
Heyman et al. (2024) [40]	The paper proposes a system called Supermind Ideator that uses an LLM and adds prompts, fine-tuning, and a specialised user interface to help users reformulate their problem statements and generate possible solutions. The findings from the experimental study show that people using Supermind Ideator generated significantly more innovative ideas than those generated by people using ChatGPT or people working alone.	Zero-shot and few-shot prompting is used to prompt LLM how to respond to a given problem using a given move from the user. In a zero-shot prompt for a move, the API gives simple English instructions to the LLM for how to apply the move. The API replaces the user's problem description in the placeholder for the prompt. In a few-shot prompt for a move, the API gives LLM several examples of problems and desirable responses to those problems, followed by the user's problem.	Developed as a web application using the React framework with a user interface, API and LLM as separate layers. The LLM base model is fine-tuned using 1600 examples of real-world practices that solve problems using innovative approaches.

Abedu et al. (2024) [5]	The paper explores the potential and limitations of LLM chatbots to answer questions related to software repositories. The chatbot is evaluated on 150 software-repository related questions and only answers one question correctly. The study then explores the limitations and challenges in adopting LLMs using RAG for extracting data from a software repository. The most frequent challenge identified is inaccurate retrieval of data.	The user inputs the URL of the target repository and questions about the repository through the chatbot UI. The document processor component retrieves metadata related to the repository and converts it into JSON format documents. After the preprocessing steps, the chunks are forwarded to the embedder component, which utilises a transformer-based embedding model to generate embeddings for all chunks, which are then stored in a vector store. Then, a document retriever component searches among document embeddings that are similar to the user's query embedding. The component then returns the top four documents to the query. The response generator component then generates a response to the user query based on documents retrieved in the previous step. The component prompts the LLM with an instruction to guide the behaviour of the LLM, context containing the retrieved documents and the user query. The generated response is returned to the UI for presentation to the user.	Python's LangChain version 0.0.216 is used to implement the chatbot with GitHub's PyDriller for the GitHub API to enable the retrieval of relevant data from repositories. The embedding-ada-002 model from OpenAI is used to generate vector embeddings for document chunks and user queries.
Jahan et al. (2024) [44]	The paper presents an automated approach for generating sequence diagrams from natural language requirements expressed as user stories. The results find that the automated approach effectively produces relevant, simplified diagrams for straightforward user stories. However, ChatGPT outperforms the automated support implemented by the authors, but the LLM tends to create complex diagrams that are sometimes beyond the simplicity of the original user stories.	The user stories are preprocessed to collect noun phrases, and NLP heuristics are used to identify sender, receiver, and message. After actors, actions and benefits are extracted from user stories, the elements are refined further. The information collected from user stories is used to generate a text script compatible with PlantUML. The generated text file is then fed to the PlantUML API to create the sequence diagram as an image. This automation is compared to using ChatGPT for the same task using the zero-shot prompt. ChatGPT is prompted with individual user stories and asked to produce the PlantUML code.	An executable program is implemented using Python 3.6 and the Stanford CoreNLP API.
Cabrero-Daniel et al. (2024) [22]	The action research focuses on the integration of AI assistants in two agile software development meetings: the daily scrum and feature refinement and planning meeting. The study finds that meeting assistants reduced the time to generate and check insights to 30 minutes from 2 hours and made fewer mistakes than humans.	The First agent developed is to assist with agile release. The assistant is instantiated using a prompt and three spreadsheet files. The files contain information about features and related child user stories, average velocity of development teams per sprint, and agile release train. The assistant then analyses the data during the meeting to provide insights on the risk of over-committing, identifying features with no effort value, identifying features placed in the incorrect backlog, unplanned integration testing efforts, etc. The second assistant developed is to get real-time recommendations on the meeting process and insights on adherence to Scrum guidelines after the Daily Scrum. This assistant is also instantiated with a prompt to give general context of the intervention and instructs LLM on how to act depending on user inputs. The assistant then provides generated recommendations as pop-up messages during the meeting and a summary of problems the team faced, lists all created tickets and tips on how to improve the next Daily Scrum meeting via the meeting chat.	The assistants are instantiated by prompting Azure OpenAI Studio's GPT-4 LLM.

Mastropaolo et al. (2024) [60]	The paper explores the use of a deep learning model to identify code summaries and link them to the documented lines of code. The study finds that the model identified code summaries with 84% accuracy and recall and precision higher than 80%.	Comments from extracted source code that are longer than 5 words are extracted, lowercased and stemmed. Each comment extracted from a method featured a special token <comment></comment> to surround the comment of interest. The method and comment pair are provided as input to another model to classify comments as code summary or other and link to relevant code statements. An already pre-trained T5 model is used, and hyperparameter tuning is performed using the BLEU-4 score as the baseline performance metric.	The model is trained on the Java CodeSearchNet dataset, which features 1.6M Java methods. The model is implemented in Python in Jupyter Notebook with a Google Cloud Storage Bucket for training and fine-tuning the T5 model.
Li et al. (2024) [56]	The paper compares LLMs with humans to summarise code. Out of the LLMs selected for study, NCS generated poor summaries compared to other models. There is no significant relation found between human-generated and LLM-generated summaries.	For each model, 68 Java methods are passed along with prompts for models to output summaries of source code. Few-shot prompting is used to instruct the models to provide summaries similar to two randomly selected human-written summaries.	The LLMs are implemented with their default parameters.
Wei (2024) [94]	The paper proposes a customised ChatGPT model tailored for analysing requirements and generating code and test cases from them. This LLM uses an external knowledge base for requirements engineering and software design, including heuristics. A case study is undertaken to demonstrate the usefulness of this tailored LLM, highlighting its potential in software development.	The user uploads structured requirements to LLM as three separate Google Docs documents: project glossary, vision and scope, and use cases. A progressive prompting approach is adopted to generate intermediate artefacts instead of just the code and test cases. The model is first prompted to refine requirements into detailed functional requirements. After that, the functional requirements are used to prompt the LLM to create an object-oriented design. The design's objective is to identify objects, assign responsibilities, and interactions between them. Lastly, the LLM is asked to build test cases and code based on the functional requirements and the design created in the earlier stages.	The authors customise a base ChatGPT using GPT's feature to automate the process. The tailored GPT is provided with instructions and a knowledge base with details of what requirements engineering is, the role of a business analyst in this process, different types of requirements, guidelines of object-oriented design and how to create use cases.
Zhang et al. (2024) [101]	The authors develop an LLM-based agent to automate the enhancement of user story quality and conduct a case study at a company to evaluate the model's quality. The model shows improvement in user story clarity, comprehensibility, and alignment with business objectives. But the results also demonstrate how important human intelligence is, especially for the PO in software projects, in tracking the stories' improvements to ensure the integrity of AI-produced outputs.	The model is initialised with user stories to be enhanced, context for the user stories, including a minimum viable product document and a product vision statement. Two agent profiles are initialised, including role definition and expectation, key responsibilities, practical tips, and tone adjustment. The product owner's role is to ensure that user stories match the product vision, and the requirements engineer's role is to enhance the quality of user stories. Product owner agent refines user stories, which are enhanced by the requirements engineer agent. The product owner agent then confirms the enhanced user stories while the requirements engineer agent ensures compliance. Product owner agent reviews and approves user stories, which are then approved by the requirements engineer agent.	The implementation includes two phases - task preparation and task conduct. The goal of the task preparation phase is to create prompts that help agents understand their responsibilities and expected contributions to the job. These prompts specify the steps that each agent must take at each stage. During the task-conduction phase, agents work together dynamically, guided by prompts to complete subtasks. Agents complete subtasks sequentially, following the structured prompts.

van Can et al. (2025) [91]	The study investigates the potential of an automated approach using GenAI to identify and classify requirements in backlog items of issue tracker systems. Encoder-only models (BERT and RoBERTa) outperform decoder-only models (Llama 3, Mistral 7B, and ChatGPT with GPT 4) in extracting and identifying backlog requirements, according to experiments.	Prompting directly to models using one-shot prompting.	Python libraries used for implementation are Transformers by HuggingFace and PyTorch for BERT and RoBERTa, and GPT4All for Mistral7B and Llama 3. The GUI version of GPT-4 is used for ChatGPT.
Bozyigit et al. (2024) [17]	The study presents a detailed evaluation of two tools, txt2Model and ChatGPT, to generate domain models from software requirements. The evaluation shows that both tools get strong F-scores for modelling classes, attributes, methods, and relationships, with txt2Model outperforming ChatGPT on average.	Feeding the dataset directly into tools and prompt models to generate domain models in textual formats.	Prompting directly in the interface of GenAI tools.
Li et al. (2024) [57]	The paper proposes an LLM-based framework for automated generation of acceptance criteria for user stories. Across all tested LLMs, including the Llama and GPT-3.5 series, the results show consistent improvement. The framework greatly improves GPT-3.5-turbo's performance, obtaining improvements of 15.56% in validity and 29.48% in completeness, as well as the highest INVEST satisfaction score of 3.21/4.	SimAC has two phases: collaboration and deduplication. During the collaboration phase, an LLM is initialised with prompts that provide instructions and instructive examples to show how collaborative tasks interact. The LLM collaborates in three groups: Requirement Analyst (RA), Quality Analyst (QA), and Other team members (Others), following a prescribed create-update-update procedure. The requirement analyst first develops an initial set of US-AC based on the responsibilities for understanding and conveying requirements. The quality analyst then updates the US-AC with some testable details. Other team members then update the US-AC further in accordance with their areas of expertise.	GPT-3.5 is implemented by invoking OpenAI's API, and open-source LLMs are instantiated using their replication packages and by loading their weights from HuggingFace.
Gartner et al. (2024) [37]	The paper proposes an LLM-based system for automated identification of contradictory requirements. GenAI models were used to evaluate the datasets to decide which model to use for the system implementation. The results found that GPT-3.5 yielded a less satisfactory answer than GPT-3 and Llama. While GPT-4's responses were akin to those of GPT-3, its computational time was notably higher, and thus, GPT-3 was selected for implementation.	The system uses a pre-built classifier to identify if and what type of contradiction is present. A formal logic is created using AI that incorporates grammatical and logical rules for the identification of condition, effect, variable, and action in requirements. The system utilises GPT-3 using the frozen model approach for the generation of responses to a set of questions. The responses are used as input for the system to identify if two requirements are contradictory.	The model is accessed using OpenAI's API.
Biswas et al. (2024) [16]	The paper proposes an automated approach to check for quality assurance of natural language requirements using LLMs. The system uses a web search-based retrieval augmentation generation (RAG) mechanism. The paper evaluates both the framework and the RAG pipeline created.	The GenAI model retrieves information from pre-trained modules such as a vectoriser module for retrieving embeddings for text and a requirement analyser module for detecting incompleteness, inconsistency, and a question-answering system. The outputs from the requirements module are given to a ChatGPT-based verifier agent initialised with prompts to verify the completeness and consistency of requirements. The results are sent back to the analyser module for displaying the response to the user.	An interface is created for users to interact with the LLMs that access pre-trained models from GitHub for different tasks in the quality assurance pipeline. The GenAI models are accessed using APIs.

Zhang et al. (2025) [100]	The paper proposes an AI agent that generates code from user requirements given by the user and updates or adds new code if user requests change. This framework integrates behaviour-driven development, which requires end users to collaborate with human agents to create dependable software applications by making three decisions: requirements, scenarios, and acceptance and recommendations.	The goal of the End-user Decision-Making component is to work with end users to gather and make sense of their choices. LLMs are guided by the AgileGen Agent's analysis and transformation of user decisions to produce software code that satisfies user needs. Scenario-Design component creates various scenarios that are described in the Gherkin language in accordance with the requirement decision-making. After presenting these scenarios to end users for scenario decision-making, the selected Gherkin scenarios are returned. The Rapid Prototype Design component is primarily concerned with writing software code based on the selected Gherkin scenarios, showing the program to users for approval, getting their feedback, and making the necessary changes.	The prototype is implemented as a web app using Python for backend and HTML, CSS, JavaScript for frontend using the Flask framework.
Mattis et al. (2024) [61]	The paper proposes a prototype to automate the approach for generating examples for code and their explanations using GenAI. The study illustrates how locally hosted LLMs may be optimised and applied to this kind of activity. The framework can be used directly in example-based live programming or for new learners to provide examples of code.	Training data is created by analysing the project and base system. Using that information, a pre-trained LLM is fine-tuned to produce a low-rank adapter. Backwards program slicing is used to collect and serialise the immediate dependents of the method of interest when users request an example. The LLM receives a prompt asking it to finish an example. After the LLM output has been built and executed, the user can use the debugger or ELP tools to examine the dynamic behaviour.	Data collection is implemented in Squeak/SmallTalk. Python is used to preprocess and fine-tune the data. LoRA (rank 16) is employed for effective fine-tuning, and CodeLlama-7B serves as the pre-trained model. Fine-tuning is implemented using PyTorch, transformers, and peft libraries and trained for two epochs on 60,000 methods.

Table B.1: Results from scoping review for automation and implementation of non-coding tasks using GenAI

Appendix C

Coding Scheme for Exit Interviews

Codes	Description	Examples
Positive comment about tool	Positive comments made by product owners about the prototype (n=5)	“I think it was pretty good. I mean, we consider it a prototype, and I mean, I think most of the features were there."
Neutral comment about tool	Neutral comments made by product owners about the prototype (n=5)	“So I didn’t really see it as a tool to help me learn the process, more like a supporting tool."
Negative comment about tool	Negative comments made by product owners about the prototype (n=5)	“...not without the full integration and full Jira support. I don’t think it will... That is needed to remove my pain points."
Problems in current implementation	Flaws and issues identified by product owners in the prototype (n=5)	“...this text could just be placed in the document instead of copy and pasting it."
Positive impact of tool on workflow	Positive impact experienced by product owners in their workflow (n=5)	“...I think it’s a good tool to be used to align the way of working between all system managers so we all follow the same process".
Negative impact of tool on workflow	Negative impact experienced by product owners in their workflow (n=5)	“...but you get it back again when you copy paste things and so on, so it’s sort of plus minus zero."
Impact of tool on context-switching	Impact on context-switching perceived by product owners (n=5)	“maybe you get derailed by some other meetings or something like that and I can quickly come back to that one, okay where was I in the process and you can quickly find the document references so absolutely I see the value in that yes."
Positive comment about GenAI use	Positive comments made by product owners about the GenAI automations (n=5)	“...it gave me some insights in how to structure the document, yeah, different headings I think that helped me..."
Neutral comment about GenAI use	Neutral comments made by product owners about the automations (n=5)	“...some parts will be useful, some parts might need some adjustment...”
Negative comment about GenAI use	Negative comments made by product owners about the prototype (n=5)	“...some of the contents were maybe not that useful..."

Perceived potential of GenAI automation	Potential perceived by product owners from GenAI automations	“Because I would say that it’s of course good that it has these text suggestions. Because it is, of course, like you have the idea of what you want to achieve, but it’s hard sometimes to put it in words. So that’s where I think this helps a lot."
Perceived limitations of GenAI automation	Limitations perceived by product owners from GenAI automations	“...one limitation or at least the worry I have is basically it, of course, makes us a bit lazy."
Feedback on quality of generated responses	Comments by product owners about quality of AI-generated responses	“It was kind of funny sometimes... It’s the information the AI found itself that it had to work on the technical portion, kind of not much to build on."
Suggestion for future prototype implementation	Suggestions made by product owners for future implementation of the prototype (n=5)	“...that we maybe should have a link somewhere, so you can press it to go in the file structure and look for it."
Suggestion for GenAI automation	Suggestions made by product owners for more GenAI automations (n=5)	“...some generative ai support for creating diagrams"
Suggestion for non-AI automation	Suggestions made by product owners for more non-AI automations (n=5)	“I think the Jira integration with the... would be very nice if the epics could be created, the story and so on could be just created and injected in the technical report."
Factors for suggestions	Reasons for suggesting the automation candidates (n=5)	“... the administrative part, that’s what I want to get rid of always."

Table C.1: Coding Scheme, Description, and Examples Identified in Thematic Analysis of Exit Interviews

Faculty of Computing, Blekinge Institute of Technology, 371 79 Karlskrona, Sweden